Chapter 1 Python Primer

Contents

1.1 Python Overview 2
1.1.1 The Python Interpreter 2
1.1.2 Preview of a Python Program 3
1.2 Objects in Python 4
1.2.1 Identifiers, Objects, and the Assignment Statement 4
1.2.2 Creating and Using Objects 6
1.2.3 Python's Built-In Classes 7
1.3 Expressions, Operators, and Precedence 12
1.3.1 Compound Expressions and Operator Precedence 17
1.4 Control Flow 18
1.4.1 Conditionals 18
1.4.2 Loops 20
1.5 Functions 23
1.5.1 Information Passing 24
1.5.2 Python's Built-In Functions 28
1.6 Simple Input and Output 30
1.6.1 Console Input and Output 30
1.6.2 Files 31
1.7 Exception Handling 33
1.7.1 Raising an Exception 34
1.7.2 Catching an Exception 36
1.8 Iterators and Generators 39
1.9 Additional Python Conveniences 42
1.9.1 Conditional Expressions 42
1.9.2 Comprehension Syntax 43
1.9.3 Packing and Unpacking of Sequences 44
1.10 Scopes and Namespaces 46
1.11 Modules and the Import Statement 48
1.11.1 Existing Modules 49
1.12 Exercises 51

1.1 Python Overview

Building data structures and algorithms requires that we communicate detailed instructions to a computer. An excellent way to perform such communications is using a high-level computer language, such as Python. The Python programming language was originally developed by Guido van Rossum in the early 1990s, and has since become a prominently used language in industry and education. The second major version of the language, Python 2, was released in 2000, and the third major version, Python 3, released in 2008. We note that there are significant incompatibilities between Python 2 and Python 3. *This book is based on Python 3 (more specifically, Python 3.1 or later).* The latest version of the language is freely available at `www.python.org`, along with documentation and tutorials.

In this chapter, we provide an overview of the Python programming language, and we continue this discussion in the next chapter, focusing on object-oriented principles. We assume that readers of this book have prior programming experience, although not necessarily using Python. This book does not provide a complete description of the Python language (there are numerous language references for that purpose), but it does introduce all aspects of the language that are used in code fragments later in this book.

1.1.1 The Python Interpreter

Python is formally an ***interpreted*** language. Commands are executed through a piece of software known as the ***Python interpreter***. The interpreter receives a command, evaluates that command, and reports the result of the command. While the interpreter can be used interactively (especially when debugging), a programmer typically defines a series of commands in advance and saves those commands in a plain text file known as ***source code*** or a ***script***. For Python, source code is conventionally stored in a file named with the `.py` suffix (e.g., `demo.py`).

On most operating systems, the Python interpreter can be started by typing `python` from the command line. By default, the interpreter starts in interactive mode with a clean workspace. Commands from a predefined script saved in a file (e.g., `demo.py`) are executed by invoking the interpreter with the filename as an argument (e.g., `python demo.py`), or using an additional `-i` flag in order to execute a script and then enter interactive mode (e.g., `python -i demo.py`).

Many ***integrated development environments*** (IDEs) provide richer software development platforms for Python, including one named IDLE that is included with the standard Python distribution. IDLE provides an embedded text-editor with support for displaying and editing Python code, and a basic debugger, allowing step-by-step execution of a program while examining key variable values.

1.1.2 Preview of a Python Program

As a simple introduction, Code Fragment 1.1 presents a Python program that computes the grade-point average (GPA) for a student based on letter grades that are entered by a user. Many of the techniques demonstrated in this example will be discussed in the remainder of this chapter. At this point, we draw attention to a few high-level issues, for readers who are new to Python as a programming language.

Python's syntax relies heavily on the use of whitespace. Individual statements are typically concluded with a newline character, although a command can extend to another line, either with a concluding backslash character (\), or if an opening delimiter has not yet been closed, such as the { character in defining value_map.

Whitespace is also key in delimiting the bodies of control structures in Python. Specifically, a block of code is indented to designate it as the body of a control structure, and nested control structures use increasing amounts of indentation. In Code Fragment 1.1, the body of the **while** loop consists of the subsequent 8 lines, including a nested conditional structure.

Comments are annotations provided for human readers, yet ignored by the Python interpreter. The primary syntax for comments in Python is based on use of the # character, which designates the remainder of the line as a comment.

```
print('Welcome to the GPA calculator.')
print('Please enter all your letter grades, one per line.')
print('Enter a blank line to designate the end.')
# map from letter grade to point value
points = {'A+':4.0, 'A':4.0, 'A-':3.67, 'B+':3.33, 'B':3.0, 'B-':2.67,
          'C+':2.33, 'C':2.0, 'C':1.67, 'D+':1.33, 'D':1.0, 'F':0.0}
num_courses = 0
total_points = 0
done = False
while not done:
  grade = input( )                          # read line from user
  if grade == '':                           # empty line was entered
    done = True
  elif grade not in points:                 # unrecognized grade entered
    print("Unknown grade '{0}' being ignored".format(grade))
  else:
    num_courses += 1
    total_points += points[grade]
if num_courses > 0:                         # avoid division by zero
  print('Your GPA is {0:.3}'.format(total_points / num_courses))
```

Code Fragment 1.1: A Python program that computes a grade-point average (GPA).

1.2 Objects in Python

Python is an object-oriented language and ***classes*** form the basis for all data types. In this section, we describe key aspects of Python's object model, and we introduce Python's built-in classes, such as the **int** class for integers, the **float** class for floating-point values, and the **str** class for character strings. A more thorough presentation of object-orientation is the focus of Chapter 2.

1.2.1 Identifiers, Objects, and the Assignment Statement

The most important of all Python commands is an ***assignment statement***, such as

```
temperature = 98.6
```

This command establishes temperature as an ***identifier*** (also known as a ***name***), and then associates it with the ***object*** expressed on the right-hand side of the equal sign, in this case a floating-point object with value 98.6. We portray the outcome of this assignment in Figure 1.1.

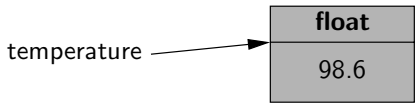


Figure 1.1: The identifier temperature references an instance of the float class having value 98.6.

Identifiers

Identifiers in Python are ***case-sensitive***, so temperature and Temperature are distinct names. Identifiers can be composed of almost any combination of letters, numerals, and underscore characters (or more general Unicode characters). The primary restrictions are that an identifier cannot begin with a numeral (thus 9lives is an illegal name), and that there are 33 specially reserved words that cannot be used as identifiers, as shown in Table 1.1.

Reserved Words								
False	as	continue	else	from	in	not	return	yield
None	assert	def	except	global	is	or	try	
True	break	del	finally	if	lambda	pass	while	
and	class	elif	for	import	nonlocal	raise	with	

Table 1.1: A listing of the reserved words in Python. These names cannot be used as identifiers.

For readers familiar with other programming languages, the semantics of a Python identifier is most similar to a reference variable in Java or a pointer variable in C++. Each identifier is implicitly associated with the ***memory address*** of the object to which it refers. A Python identifier may be assigned to a special object named None, serving a similar purpose to a null reference in Java or C++.

Unlike Java and C++, Python is a ***dynamically typed*** language, as there is no advance declaration associating an identifier with a particular data type. An identifier can be associated with any type of object, and it can later be reassigned to another object of the same (or different) type. Although an identifier has no declared type, the object to which it refers has a definite type. In our first example, the characters 98.6 are recognized as a floating-point literal, and thus the identifier temperature is associated with an instance of the float class having that value.

A programmer can establish an ***alias*** by assigning a second identifier to an existing object. Continuing with our earlier example, Figure 1.2 portrays the result of a subsequent assignment, original = temperature.

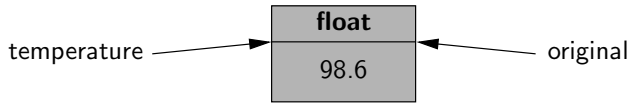


Figure 1.2: Identifiers temperature and original are aliases for the same object.

Once an alias has been established, either name can be used to access the underlying object. If that object supports behaviors that affect its state, changes enacted through one alias will be apparent when using the other alias (because they refer to the same object). However, if one of the *names* is reassigned to a new value using a subsequent assignment statement, that does not affect the aliased object, rather it breaks the alias. Continuing with our concrete example, we consider the command:

```
temperature = temperature + 5.0
```

The execution of this command begins with the evaluation of the expression on the right-hand side of the = operator. That expression, temperature + 5.0, is evaluated based on the *existing* binding of the name temperature, and so the result has value 103.6, that is, 98.6 + 5.0. That result is stored as a new floating-point instance, and only then is the name on the left-hand side of the assignment statement, temperature, (re)assigned to the result. The subsequent configuration is diagrammed in Figure 1.3. Of particular note, this last command had no effect on the value of the existing float instance that identifier original continues to reference.

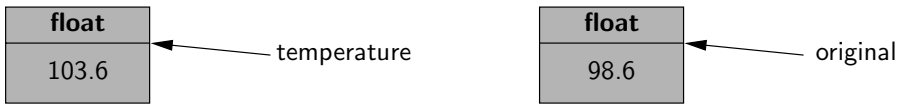


Figure 1.3: The temperature identifier has been assigned to a new value, while original continues to refer to the previously existing value.

1.2.2 Creating and Using Objects

Instantiation

The process of creating a new instance of a class is known as ***instantiation***. In general, the syntax for instantiating an object is to invoke the ***constructor*** of a class. For example, if there were a class named Widget, we could create an instance of that class using a syntax such as `w = Widget()`, assuming that the constructor does not require any parameters. If the constructor does require parameters, we might use a syntax such as `Widget(a, b, c)` to construct a new instance.

Many of Python's built-in classes (discussed in Section 1.2.3) support what is known as a ***literal*** form for designating new instances. For example, the command `temperature = 98.6` results in the creation of a new instance of the **float** class; the term `98.6` in that expression is a literal form. We discuss further cases of Python literals in the coming section.

From a programmer's perspective, yet another way to indirectly create a new instance of a class is to call a function that creates and returns such an instance. For example, Python has a built-in function named `sorted` (see Section 1.5.2) that takes a sequence of comparable elements as a parameter and returns a new instance of the **list** class containing those elements in sorted order.

Calling Methods

Python supports traditional functions (see Section 1.5) that are invoked with a syntax such as `sorted(data)`, in which case `data` is a parameter sent to the function. Python's classes may also define one or more ***methods*** (also known as ***member functions***), which are invoked on a specific instance of a class using the dot (".") operator. For example, Python's `list` class has a method named `sort` that can be invoked with a syntax such as `data.sort()`. This particular method rearranges the contents of the list so that they are sorted.

The expression to the left of the dot identifies the object upon which the method is invoked. Often, this will be an identifier (e.g., `data`), but we can use the dot operator to invoke a method upon the immediate result of some other operation. For example, if `response` identifies a string instance (we will discuss strings later in this section), the syntax `response.lower().startswith('y')` first evaluates the method call, `response.lower()`, which itself returns a new string instance, and then the `startswith('y')` method is called on that intermediate string.

When using a method of a class, it is important to understand its behavior. Some methods return information about the state of an object, but do not change that state. These are known as ***accessors***. Other methods, such as the `sort` method of the `list` class, do change the state of an object. These methods are known as ***mutators*** or ***update methods***.

1.2.3 Python's Built-In Classes

Table 1.2 provides a summary of commonly used, built-in classes in Python; we take particular note of which classes are mutable and which are immutable. A class is ***immutable*** if each object of that class has a fixed value upon instantiation that cannot subsequently be changed. For example, the float class is immutable. Once an instance has been created, its value cannot be changed (although an identifier referencing that object can be reassigned to a different value).

Class	Description	Immutable?
bool	Boolean value	✓
int	integer (arbitrary magnitude)	✓
float	floating-point number	✓
list	mutable sequence of objects	
tuple	immutable sequence of objects	✓
str	character string	✓
set	unordered set of distinct objects	
frozenset	immutable form of set class	✓
dict	associative mapping (aka dictionary)	

Table 1.2: Commonly used built-in classes for Python

In this section, we provide an introduction to these classes, discussing their purpose and presenting several means for creating instances of the classes. Literal forms (such as 98.6) exist for most of the built-in classes, and all of the classes support a traditional constructor form that creates instances that are based upon one or more existing values. Operators supported by these classes are described in Section 1.3. More detailed information about these classes can be found in later chapters as follows: lists and tuples (Chapter 5); strings (Chapters 5 and 13, and Appendix A); sets and dictionaries (Chapter 10).

The bool Class

The **bool** class is used to manipulate logical (Boolean) values, and the only two instances of that class are expressed as the literals True and False. The default constructor, bool( ), returns False, but there is no reason to use that syntax rather than the more direct literal form. Python allows the creation of a Boolean value from a nonboolean type using the syntax bool(foo) for value foo. The interpretation depends upon the type of the parameter. Numbers evaluate to False if zero, and True if nonzero. Sequences and other container types, such as strings and lists, evaluate to False if empty and True if nonempty. An important application of this interpretation is the use of a nonboolean value as a condition in a control structure.

The int Class

The **int** and **float** classes are the primary numeric types in Python. The **int** class is designed to represent integer values with arbitrary magnitude. Unlike Java and C++, which support different integral types with different precisions (e.g., int, short, long), Python automatically chooses the internal representation for an integer based upon the magnitude of its value. Typical literals for integers include 0, 137, and −23. In some contexts, it is convenient to express an integral value using binary, octal, or hexadecimal. That can be done by using a prefix of the number 0 and then a character to describe the base. Example of such literals are respectively 0b1011, 0o52, and 0x7f.

The integer constructor, int( ), returns value 0 by default. But this constructor can be used to construct an integer value based upon an existing value of another type. For example, if f represents a floating-point value, the syntax int(f) produces the *truncated* value of f. For example, both int(3.14) and int(3.99) produce the value 3, while int(−3.9) produces the value −3. The constructor can also be used to parse a string that is presumed to represent an integral value (such as one entered by a user). If s represents a string, then int(s) produces the integral value that string represents. For example, the expression int('137') produces the integer value 137. If an invalid string is given as a parameter, as in int('hello'), a ValueError is raised (see Section 1.7 for discussion of Python's exceptions). By default, the string must use base 10. If conversion from a different base is desired, that base can be indicated as a second, optional, parameter. For example, the expression int('7f', 16) evaluates to the integer 127.

The float Class

The **float** class is the sole floating-point type in Python, using a fixed-precision representation. Its precision is more akin to a double in Java or C++, rather than those languages' float type. We have already discussed a typical literal form, 98.6. We note that the floating-point equivalent of an integral number can be expressed directly as 2.0. Technically, the trailing zero is optional, so some programmers might use the expression 2. to designate this floating-point literal. One other form of literal for floating-point values uses scientific notation. For example, the literal 6.022e23 represents the mathematical value 6.022×10^{23}.

The constructor form of float( ) returns 0.0. When given a parameter, the constructor attempts to return the equivalent floating-point value. For example, the call float(2) returns the floating-point value 2.0. If the parameter to the constructor is a string, as with float('3.14'), it attempts to parse that string as a floating-point value, raising a ValueError as an exception.

Sequence Types: The list, tuple, and str Classes

The **list**, **tuple**, and **str** classes are ***sequence*** types in Python, representing a collection of values in which the order is significant. The list class is the most general, representing a sequence of arbitrary objects (akin to an "array" in other languages). The tuple class is an ***immutable*** version of the list class, benefiting from a streamlined internal representation. The str class is specially designed for representing an immutable sequence of text characters. We note that Python does not have a separate class for characters; they are just strings with length one.

The list Class

A **list** instance stores a sequence of objects. A list is a ***referential*** structure, as it technically stores a sequence of ***references*** to its elements (see Figure 1.4). Elements of a list may be arbitrary objects (including the None object). Lists are ***array-based*** sequences and are ***zero-indexed***, thus a list of length n has elements indexed from 0 to $n-1$ inclusive. Lists are perhaps the most used container type in Python and they will be extremely central to our study of data structures and algorithms. They have many valuable behaviors, including the ability to dynamically expand and contract their capacities as needed. In this chapter, we will discuss only the most basic properties of lists. We revisit the inner working of all of Python's sequence types as the focus of Chapter 5.

Python uses the characters [ ] as delimiters for a list literal, with [ ] itself being an empty list. As another example, `['red', 'green', 'blue']` is a list containing three string instances. The contents of a list literal need not be expressed as literals; if identifiers a and b have been established, then syntax [a, b] is legitimate.

The list( ) constructor produces an empty list by default. However, the constructor will accept any parameter that is of an ***iterable*** type. We will discuss iteration further in Section 1.8, but examples of iterable types include all of the standard container types (e.g., strings, list, tuples, sets, dictionaries). For example, the syntax list(`'hello'`) produces a list of individual characters, `['h', 'e', 'l', 'l', 'o']`. Because an existing list is itself iterable, the syntax backup = list(data) can be used to construct a new list instance referencing the same contents as the original.

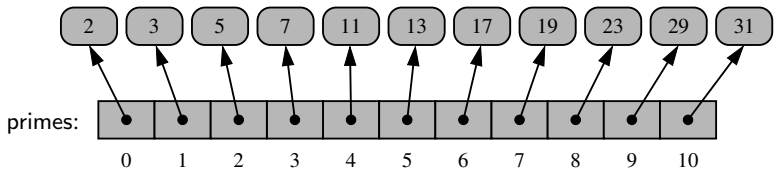


Figure 1.4: Python's internal representation of a list of integers, instantiated as prime = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31]. The implicit indices of the elements are shown below each entry.

The tuple Class

The **tuple** class provides an immutable version of a sequence, and therefore its instances have an internal representation that may be more streamlined than that of a list. While Python uses the [ ] characters to delimit a list, parentheses delimit a tuple, with ( ) being an empty tuple. There is one important subtlety. To express a tuple of length one as a literal, a comma must be placed after the element, but within the parentheses. For example, (17,) is a one-element tuple. The reason for this requirement is that, without the trailing comma, the expression (17) is viewed as a simple parenthesized numeric expression.

The str Class

Python's **str** class is specifically designed to efficiently represent an immutable sequence of characters, based upon the Unicode international character set. Strings have a more compact internal representation than the referential lists and tuples, as portrayed in Figure 1.5.

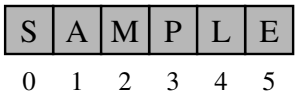


Figure 1.5: A Python string, which is an indexed sequence of characters.

String literals can be enclosed in single quotes, as in `'hello'`, or double quotes, as in `"hello"`. This choice is convenient, especially when using another of the quotation characters as an actual character in the sequence, as in `"Don't worry"`. Alternatively, the quote delimiter can be designated using a backslash as a so-called ***escape character***, as in `'Don\'t worry'`. Because the backslash has this purpose, the backslash must itself be escaped to occur as a natural character of the string literal, as in `'C:\\Python\\'`, for a string that would be displayed as `C:\Python\`. Other commonly escaped characters are `\n` for newline and `\t` for tab. Unicode characters can be included, such as `'20\u20AC'` for the string 20€.

Python also supports using the delimiter `'''` or `"""` to begin and end a string literal. The advantage of such triple-quoted strings is that newline characters can be embedded naturally (rather than escaped as `\n`). This can greatly improve the readability of long, multiline strings in source code. For example, at the beginning of Code Fragment 1.1, rather than use separate print statements for each line of introductory output, we can use a single print statement, as follows:

```
print("""Welcome to the GPA calculator.
Please enter all your letter grades, one per line.
Enter a blank line to designate the end.""")
```

The set and frozenset Classes

Python's **set** class represents the mathematical notion of a set, namely a collection of elements, without duplicates, and without an inherent order to those elements. The major advantage of using a set, as opposed to a list, is that it has a highly optimized method for checking whether a specific element is contained in the set. This is based on a data structure known as a ***hash table*** (which will be the primary topic of Chapter 10). However, there are two important restrictions due to the algorithmic underpinnings. The first is that the set does not maintain the elements in any particular order. The second is that only instances of ***immutable*** types can be added to a Python set. Therefore, objects such as integers, floating-point numbers, and character strings are eligible to be elements of a set. It is possible to maintain a set of tuples, but not a set of lists or a set of sets, as lists and sets are mutable. The **frozenset** class is an immutable form of the set type, so it is legal to have a set of frozensets.

Python uses curly braces { and } as delimiters for a set, for example, as {17} or {'red', 'green', 'blue'}. The exception to this rule is that { } does not represent an empty set; for historical reasons, it represents an empty dictionary (see next paragraph). Instead, the constructor syntax set( ) produces an empty set. If an iterable parameter is sent to the constructor, then the set of distinct elements is produced. For example, set('hello') produces {'h', 'e', 'l', 'o'}.

The dict Class

Python's **dict** class represents a ***dictionary***, or ***mapping***, from a set of distinct ***keys*** to associated ***values***. For example, a dictionary might map from unique student ID numbers, to larger student records (such as the student's name, address, and course grades). Python implements a dict using an almost identical approach to that of a set, but with storage of the associated values.

A dictionary literal also uses curly braces, and because dictionaries were introduced in Python prior to sets, the literal form { } produces an empty dictionary. A nonempty dictionary is expressed using a comma-separated series of key:value pairs. For example, the dictionary {'ga' : 'Irish', 'de' : 'German'} maps 'ga' to 'Irish' and 'de' to 'German'.

The constructor for the dict class accepts an existing mapping as a parameter, in which case it creates a new dictionary with identical associations as the existing one. Alternatively, the constructor accepts a sequence of key-value pairs as a parameter, as in dict(pairs) with pairs = [('ga', 'Irish'), ('de', 'German')].

1.3 Expressions, Operators, and Precedence

In the previous section, we demonstrated how names can be used to identify existing objects, and how literals and constructors can be used to create instances of built-in classes. Existing values can be combined into larger syntactic ***expressions*** using a variety of special symbols and keywords known as ***operators***. The semantics of an operator depends upon the type of its operands. For example, when a and b are numbers, the syntax a + b indicates addition, while if a and b are strings, the operator indicates concatenation. In this section, we describe Python's operators in various contexts of the built-in types.

We continue, in Section 1.3.1, by discussing ***compound expressions***, such as a + b * c, which rely on the evaluation of two or more operations. The order in which the operations of a compound expression are evaluated can affect the overall value of the expression. For this reason, Python defines a specific order of precedence for evaluating operators, and it allows a programmer to override this order by using explicit parentheses to group subexpressions.

Logical Operators

Python supports the following keyword operators for Boolean values:

not	unary negation
and	conditional and
or	conditional or

The **and** and **or** operators ***short-circuit***, in that they do not evaluate the second operand if the result can be determined based on the value of the first operand. This feature is useful when constructing Boolean expressions in which we first test that a certain condition holds (such as a reference not being **None**), and then test a condition that could have otherwise generated an error condition had the prior test not succeeded.

Equality Operators

Python supports the following operators to test two notions of equality:

is	same identity
is not	different identity
==	equivalent
!=	not equivalent

The expression a **is** b evaluates to **True**, precisely when identifiers a and b are aliases for the *same* object. The expression a == b tests a more general notion of equivalence. If identifiers a and b refer to the same object, then a == b should also evaluate to **True**. Yet a == b also evaluates to **True** when the identifiers refer to

different objects that happen to have values that are deemed equivalent. The precise notion of equivalence depends on the data type. For example, two strings are considered equivalent if they match character for character. Two sets are equivalent if they have the same contents, irrespective of order. In most programming situations, the equivalence tests == and != are the appropriate operators; use of **is** and **is not** should be reserved for situations in which it is necessary to detect true aliasing.

Comparison Operators

Data types may define a natural order via the following operators:

$<$	less than
$<=$	less than or equal to
$>$	greater than
$>=$	greater than or equal to

These operators have expected behavior for numeric types, and are defined lexicographically, and case-sensitively, for strings. An exception is raised if operands have incomparable types, as with `5 < 'hello'`.

Arithmetic Operators

Python supports the following arithmetic operators:

$+$	addition
$-$	subtraction
$*$	multiplication
$/$	true division
$//$	integer division
%	the modulo operator

The use of addition, subtraction, and multiplication is straightforward, noting that if both operands have type int, then the result is an int as well; if one or both operands have type float, the result will be a float.

Python takes more care in its treatment of division. We first consider the case in which both operands have type int, for example, the quantity 27 divided by 4. In mathematical notation, $27 \div 4 = 6\frac{3}{4} = 6.75$. In Python, the / operator designates ***true division***, returning the floating-point result of the computation. Thus, `27 / 4` results in the float value 6.75. Python supports the pair of operators // and % to perform the integral calculations, with expression `27 // 4` evaluating to int value 6 (the mathematical *floor* of the quotient), and expression `27 % 4` evaluating to int value 3, the remainder of the integer division. We note that languages such as C, C++, and Java do not support the // operator; instead, the / operator returns the truncated quotient when both operands have integral type, and the result of true division when at least one operand has a floating-point type.

Python carefully extends the semantics of // and % to cases where one or both operands are negative. For the sake of notation, let us assume that variables n and m represent respectively the ***dividend*** and ***divisor*** of a quotient $\frac{n}{m}$, and that q = n // m and r = n % m. Python guarantees that q * m + r will equal n. We already saw an example of this identity with positive operands, as $6*4+3=27$. When the divisor m is positive, Python further guarantees that $0 \leq r < m$. As a consequence, we find that -27 // 4 evaluates to -7 and -27 % 4 evaluates to 1, as $(-7)*4+1=-27$. When the divisor is negative, Python guarantees that $m < r \leq 0$. As an example, 27 // -4 is -7 and 27 % -4 is -1, satisfying the identity $27=(-7)*(-4)+(-1)$.

The conventions for the // and % operators are even extended to floating-point operands, with the expression q = n // m being the integral floor of the quotient, and r = n % m being the "remainder" to ensure that q * m + r equals n. For example, 8.2 // 3.14 evaluates to 2.0 and 8.2 % 3.14 evaluates to 1.92, as $2.0*3.14+1.92=8.2$.

Bitwise Operators

Python provides the following bitwise operators for integers:

~	bitwise complement (prefix unary operator)
&	bitwise and
|	bitwise or
^	bitwise exclusive-or
<<	shift bits left, filling in with zeros
>>	shift bits right, filling in with sign bit

Sequence Operators

Each of Python's built-in sequence types (**str**, **tuple**, and **list**) support the following operator syntaxes:

s[j]	element at index *j*
s[start:stop]	slice including indices [start,stop)
s[start:stop:step]	slice including indices start, start + step, start + 2*step, ..., up to but not equalling or stop
s + t	concatenation of sequences
k * s	shorthand for s + s + s + ... (k times)
val **in** s	containment check
val **not in** s	non-containment check

Python relies on ***zero-indexing*** of sequences, thus a sequence of length n has elements indexed from 0 to $n-1$ inclusive. Python also supports the use of ***negative indices***, which denote a distance from the end of the sequence; index -1 denotes the last element, index -2 the second to last, and so on. Python uses a ***slicing***

notation to describe subsequences of a sequence. Slices are described as half-open intervals, with a start index that is included, and a stop index that is excluded. For example, the syntax `data[3:8]` denotes a subsequence including the five indices: $3, 4, 5, 6, 7$. An optional "step" value, possibly negative, can be indicated as a third parameter of the slice. If a start index or stop index is omitted in the slicing notation, it is presumed to designate the respective extreme of the original sequence.

Because lists are mutable, the syntax `s[j] = val` can be used to replace an element at a given index. Lists also support a syntax, **del** `s[j]`, that removes the designated element from the list. Slice notation can also be used to replace or delete a sublist.

The notation `val` **in** `s` can be used for any of the sequences to see if there is an element equivalent to `val` in the sequence. For strings, this syntax can be used to check for a single character or for a larger substring, as with `'amp'` **in** `'example'`.

All sequences define comparison operations based on ***lexicographic order***, performing an element by element comparison until the first difference is found. For example, [5, 6, 9] < [5, 7] because of the entries at index 1. Therefore, the following operations are supported by sequence types:

`s == t`	equivalent (element by element)
`s != t`	not equivalent
`s < t`	lexicographically less than
`s <= t`	lexicographically less than or equal to
`s > t`	lexicographically greater than
`s >= t`	lexicographically greater than or equal to

Operators for Sets and Dictionaries

Sets and frozensets support the following operators:

`key` **in** `s`	containment check
`key` **not in** `s`	non-containment check
`s1 == s2`	s1 is equivalent to s2
`s1 != s2`	s1 is not equivalent to s2
`s1 <= s2`	s1 is subset of s2
`s1 < s2`	s1 is proper subset of s2
`s1 >= s2`	s1 is superset of s2
`s1 > s2`	s1 is proper superset of s2
`s1 | s2`	the union of s1 and s2
`s1 & s2`	the intersection of s1 and s2
`s1 - s2`	the set of elements in s1 but not s2
`s1 ^ s2`	the set of elements in precisely one of s1 or s2

Note well that sets do not guarantee a particular order of their elements, so the comparison operators, such as <, are not lexicographic; rather, they are based on the mathematical notion of a subset. As a result, the comparison operators define

a partial order, but not a total order, as disjoint sets are neither "less than," "equal to," or "greater than" each other. Sets also support many fundamental behaviors through named methods (e.g., add, remove); we will explore their functionality more fully in Chapter 10.

Dictionaries, like sets, do not maintain a well-defined order on their elements. Furthermore, the concept of a subset is not typically meaningful for dictionaries, so the dict class does not support operators such as $<$. Dictionaries support the notion of equivalence, with d1 $==$ d2 if the two dictionaries contain the same set of key-value pairs. The most widely used behavior of dictionaries is accessing a value associated with a particular key k with the indexing syntax, d[k]. The supported operators are as follows:

d[key]	value associated with given key
d[key] = value	set (or reset) the value associated with given key
del d[key]	remove key and its associated value from dictionary
key **in** d	containment check
key **not in** d	non-containment check
d1 == d2	d1 is equivalent to d2
d1 != d2	d1 is not equivalent to d2

Dictionaries also support many useful behaviors through named methods, which we explore more fully in Chapter 10.

Extended Assignment Operators

Python supports an extended assignment operator for most binary operators, for example, allowing a syntax such as count += 5. By default, this is a shorthand for the more verbose count = count + 5. For an immutable type, such as a number or a string, one should not presume that this syntax changes the value of the existing object, but instead that it will reassign the identifier to a newly constructed value. (See discussion of Figure 1.3.) However, it is possible for a type to redefine such semantics to mutate the object, as the list class does for the += operator.

```
alpha = [1, 2, 3]
beta = alpha                # an alias for alpha
beta += [4, 5]              # extends the original list with two more elements
beta = beta + [6, 7]        # reassigns beta to a new list [1, 2, 3, 4, 5, 6, 7]
print(alpha)                # will be [1, 2, 3, 4, 5]
```

This example demonstrates the subtle difference between the list semantics for the syntax beta += foo versus beta = beta + foo.

1.3.1 Compound Expressions and Operator Precedence

Programming languages must have clear rules for the order in which compound expressions, such as $5 + 2 * 3$, are evaluated. The formal order of precedence for operators in Python is given in Table 1.3. Operators in a category with higher precedence will be evaluated before those with lower precedence, unless the expression is otherwise parenthesized. Therefore, we see that Python gives precedence to multiplication over addition, and therefore evaluates the expression $5 + 2 * 3$ as $5 + (2 * 3)$, with value 11, but the parenthesized expression $(5 + 2) * 3$ evaluates to value 21. Operators within a category are typically evaluated from left to right, thus $5 - 2 + 3$ has value 6. Exceptions to this rule include that unary operators and exponentiation are evaluated from right to left.

Python allows a ***chained assignment***, such as `x = y = 0`, to assign multiple identifiers to the rightmost value. Python also allows the ***chaining*** of comparison operators. For example, the expression `1 <= x + y <= 10` is evaluated as the compound `(1 <= x + y)` **and** `(x + y <= 10)`, but without computing the intermediate value `x + y` twice.

Operator Precedence		
	Type	**Symbols**
1	member access	expr.member
2	function/method calls container subscripts/slices	expr(...) expr[...]
3	exponentiation	**
4	unary operators	+expr, −expr, ~expr
5	multiplication, division	*, /, //, %
6	addition, subtraction	+, −
7	bitwise shifting	<<, >>
8	bitwise-and	&
9	bitwise-xor	^
10	bitwise-or	|
11	comparisons containment	**is**, **is not**, ==, !=, <, <=, >, >= **in**, **not in**
12	logical-not	**not** expr
13	logical-and	**and**
14	logical-or	**or**
15	conditional	val1 **if** cond **else** val2
16	assignments	=, +=, −=, *=, etc.

Table 1.3: Operator precedence in Python, with categories ordered from highest precedence to lowest precedence. When stated, we use expr to denote a literal, identifier, or result of a previously evaluated expression. All operators without explicit mention of expr are binary operators, with syntax expr1 operator expr2.

1.4 Control Flow

In this section, we review Python's most fundamental control structures: conditional statements and loops. Common to all control structures is the syntax used in Python for defining blocks of code. The colon character is used to delimit the beginning of a block of code that acts as a body for a control structure. If the body can be stated as a single executable statement, it can technically placed on the same line, to the right of the colon. However, a body is more typically typeset as an ***indented block*** starting on the line following the colon. Python relies on the indentation level to designate the extent of that block of code, or any nested blocks of code within. The same principles will be applied when designating the body of a function (see Section 1.5), and the body of a class (see Section 2.3).

1.4.1 Conditionals

Conditional constructs (also known as **if** statements) provide a way to execute a chosen block of code based on the run-time evaluation of one or more Boolean expressions. In Python, the most general form of a conditional is written as follows:

```
if first_condition:
  first_body
elif second_condition:
  second_body
elif third_condition:
  third_body
else:
  fourth_body
```

Each condition is a Boolean expression, and each body contains one or more commands that are to be executed conditionally. If the first condition succeeds, the first body will be executed; no other conditions or bodies are evaluated in that case. If the first condition fails, then the process continues in similar manner with the evaluation of the second condition. The execution of this overall construct will cause precisely one of the bodies to be executed. There may be any number of **elif** clauses (including zero), and the final **else** clause is optional. As described on page 7, nonboolean types may be evaluated as Booleans with intuitive meanings. For example, if response is a string that was entered by a user, and we want to condition a behavior on this being a nonempty string, we may write

```
if response:
```

as a shorthand for the equivalent,

```
if response != '':
```

As a simple example, a robot controller might have the following logic:

```
if door_is_closed:
  open_door( )
advance( )
```

Notice that the final command, advance( ), is not indented and therefore not part of the conditional body. It will be executed unconditionally (although after opening a closed door).

We may nest one control structure within another, relying on indentation to make clear the extent of the various bodies. Revisiting our robot example, here is a more complex control that accounts for unlocking a closed door.

```
if door_is_closed:
  if door_is_locked:
    unlock_door( )
  open_door( )
advance( )
```

The logic expressed by this example can be diagrammed as a traditional ***flowchart***, as portrayed in Figure 1.6.

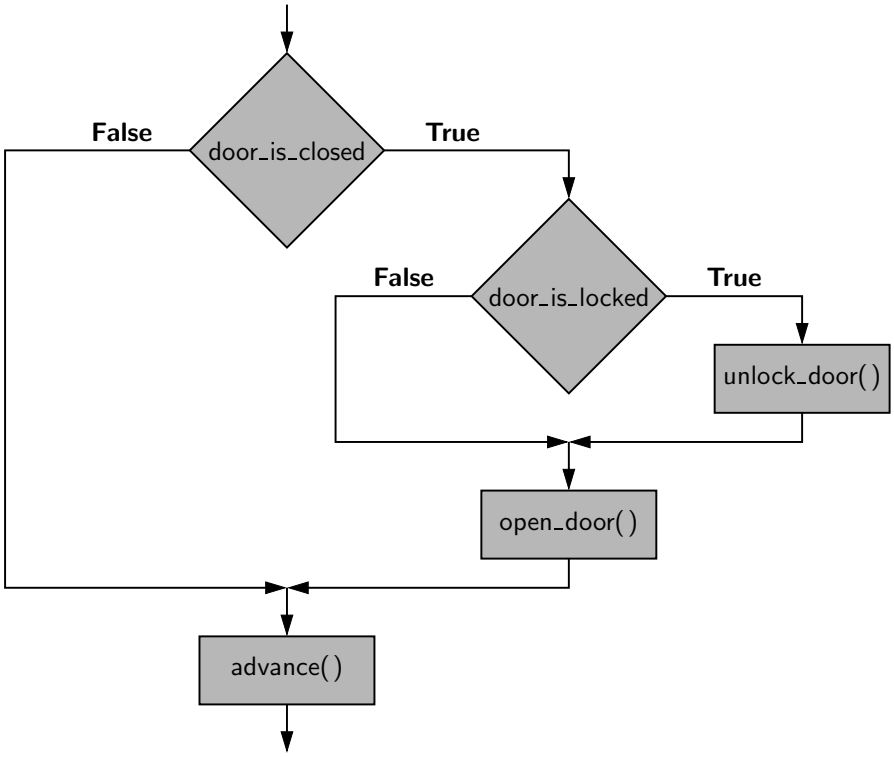


Figure 1.6: A flowchart describing the logic of nested conditional statements.

1.4.2 Loops

Python offers two distinct looping constructs. A **while** loop allows general repetition based upon the repeated testing of a Boolean condition. A **for** loop provides convenient iteration of values from a defined series (such as characters of a string, elements of a list, or numbers within a given range). We discuss both forms in this section.

While Loops

The syntax for a **while** loop in Python is as follows:

```
while condition:
  body
```

As with an **if** statement, *condition* can be an arbitrary Boolean expression, and *body* can be an arbitrary block of code (including nested control structures). The execution of a while loop begins with a test of the Boolean condition. If that condition evaluates to True, the body of the loop is performed. After each execution of the body, the loop condition is retested, and if it evaluates to True, another iteration of the body is performed. When the conditional test evaluates to False (assuming it ever does), the loop is exited and the flow of control continues just beyond the body of the loop.

As an example, here is a loop that advances an index through a sequence of characters until finding an entry with value `'X'` or reaching the end of the sequence.

```
j = 0
while j < len(data) and data[j] != 'X':
  j += 1
```

The len function, which we will introduce in Section 1.5.2, returns the length of a sequence such as a list or string. The correctness of this loop relies on the short-circuiting behavior of the **and** operator, as described on page 12. We intentionally test $j < \text{len}(\text{data})$ to ensure that j is a valid index, prior to accessing element data[j]. Had we written that compound condition with the opposite order, the evaluation of data[j] would eventually raise an IndexError when `'X'` is not found. (See Section 1.7 for discussion of exceptions.)

As written, when this loop terminates, variable j's value will be the index of the leftmost occurrence of `'X'`, if found, or otherwise the length of the sequence (which is recognizable as an invalid index to indicate failure of the search). It is worth noting that this code behaves correctly, even in the special case when the list is empty, as the condition $j < \text{len}(\text{data})$ will initially fail and the body of the loop will never be executed.

For Loops

Python's **for**-loop syntax is a more convenient alternative to a while loop when iterating through a series of elements. The for-loop syntax can be used on any type of ***iterable*** structure, such as a list, tuple str, set, dict, or file (we will discuss iterators more formally in Section 1.8). Its general syntax appears as follows.

```
for element in iterable:
  body                          # body may refer to 'element' as an identifier
```

For readers familiar with Java, the semantics of Python's for loop is similar to the "for each" loop style introduced in Java 1.5.

As an instructive example of such a loop, we consider the task of computing the sum of a list of numbers. (Admittedly, Python has a built-in function, sum, for this purpose.) We perform the calculation with a for loop as follows, assuming that data identifies the list:

```
total = 0
for val in data:
  total += val                  # note use of the loop variable, val
```

The loop body executes once for each element of the data sequence, with the identifier, val, from the for-loop syntax assigned at the beginning of each pass to a respective element. It is worth noting that val is treated as a standard identifier. If the element of the original data happens to be mutable, the val identifier can be used to invoke its methods. But a reassignment of identifier val to a new value has no affect on the original data, nor on the next iteration of the loop.

As a second classic example, we consider the task of finding the maximum value in a list of elements (again, admitting that Python's built-in max function already provides this support). If we can assume that the list, data, has at least one element, we could implement this task as follows:

```
biggest = data[0]               # as we assume nonempty list
for val in data:
  if val > biggest:
    biggest = val
```

Although we could accomplish both of the above tasks with a while loop, the for-loop syntax had an advantage of simplicity, as there is no need to manage an explicit index into the list nor to author a Boolean loop condition. Furthermore, we can use a for loop in cases for which a while loop does not apply, such as when iterating through a collection, such as a set, that does not support any direct form of indexing.

Index-Based For Loops

The simplicity of a standard for loop over the elements of a list is wonderful; however, one limitation of that form is that we do not know where an element resides within the sequence. In some applications, we need knowledge of the index of an element within the sequence. For example, suppose that we want to know *where* the maximum element in a list resides.

Rather than directly looping over the elements of the list in that case, we prefer to loop over all possible indices of the list. For this purpose, Python provides a built-in class named range that generates integer sequences. (We will discuss generators in Section 1.8.) In simplest form, the syntax range(n) generates the series of n values from 0 to $n-1$. Conveniently, these are precisely the series of valid indices into a sequence of length n. Therefore, a standard Python idiom for looping through the series of indices of a data sequence uses a syntax,

```
for j in range(len(data)):
```

In this case, identifier j is not an element of the data—it is an integer. But the expression data[j] can be used to retrieve the respective element. For example, we can find the *index* of the maximum element of a list as follows:

```
big_index = 0
for j in range(len(data)):
  if data[j] > data[big_index]:
    big_index = j
```

Break and Continue Statements

Python supports a **break** statement that immediately terminate a while or for loop when executed within its body. More formally, if applied within nested control structures, it causes the termination of the most immediately enclosing loop. As a typical example, here is code that determines whether a target value occurs in a data set:

```
found = False
for item in data:
  if item == target:
    found = True
    break
```

Python also supports a **continue** statement that causes the current *iteration* of a loop body to stop, but with subsequent passes of the loop proceeding as expected.

We recommend that the break and continue statements be used sparingly. Yet, there are situations in which these commands can be effectively used to avoid introducing overly complex logical conditions.

1.5 Functions

In this section, we explore the creation of and use of functions in Python. As we did in Section 1.2.2, we draw a distinction between ***functions*** and ***methods***. We use the general term *function* to describe a traditional, stateless function that is invoked without the context of a particular class or an instance of that class, such as sorted(data). We use the more specific term *method* to describe a member function that is invoked upon a specific object using an object-oriented message passing syntax, such as data.sort( ). In this section, we only consider pure functions; methods will be explored with more general object-oriented principles in Chapter 2.

We begin with an example to demonstrate the syntax for defining functions in Python. The following function counts the number of occurrences of a given target value within any form of iterable data set.

```
def count(data, target):
  n = 0
  for item in data:
    if item == target:                    # found a match
      n += 1
  return n
```

The first line, beginning with the keyword **def**, serves as the function's ***signature***. This establishes a new identifier as the name of the function (count, in this example), and it establishes the number of parameters that it expects, as well as names identifying those parameters (data and target, in this example). Unlike Java and C++, Python is a dynamically typed language, and therefore a Python signature does not designate the types of those parameters, nor the type (if any) of a return value. Those expectations should be stated in the function's documentation (see Section 2.2.3) and can be enforced within the body of the function, but misuse of a function will only be detected at run-time.

The remainder of the function definition is known as the ***body*** of the function. As is the case with control structures in Python, the body of a function is typically expressed as an indented block of code. Each time a function is called, Python creates a dedicated ***activation record*** that stores information relevant to the current call. This activation record includes what is known as a ***namespace*** (see Section 1.10) to manage all identifiers that have ***local scope*** within the current call. The namespace includes the function's parameters and any other identifiers that are defined locally within the body of the function. An identifier in the local scope of the function caller has no relation to any identifier with the same name in the caller's scope (although identifiers in different scopes may be aliases to the same object). In our first example, the identifier n has scope that is local to the function call, as does the identifier item, which is established as the loop variable.

Return Statement

A **return** statement is used within the body of a function to indicate that the function should immediately cease execution, and that an expressed value should be returned to the caller. If a return statement is executed without an explicit argument, the None value is automatically returned. Likewise, None will be returned if the flow of control ever reaches the end of a function body without having executed a return statement. Often, a return statement will be the final command within the body of the function, as was the case in our earlier example of a count function. However, there can be multiple return statements in the same function, with conditional logic controlling which such command is executed, if any. As a further example, consider the following function that tests if a value exists in a sequence.

```
def contains(data, target):
  for item in target:
    if item == target:                    # found a match
      return True
  return False
```

If the conditional within the loop body is ever satisfied, the return True statement is executed and the function immediately ends, with True designating that the target value was found. Conversely, if the for loop reaches its conclusion without ever finding the match, the final return False statement will be executed.

1.5.1 Information Passing

To be a successful programmer, one must have clear understanding of the mechanism in which a programming language passes information to and from a function. In the context of a function signature, the identifiers used to describe the expected parameters are known as ***formal parameters***, and the objects sent by the caller when invoking the function are the ***actual parameters***. Parameter passing in Python follows the semantics of the standard ***assignment statement***. When a function is invoked, each identifier that serves as a formal parameter is assigned, in the function's local scope, to the respective actual parameter that is provided by the caller of the function.

For example, consider the following call to our count function from page 23:

```
prizes = count(grades, 'A')
```

Just before the function body is executed, the actual parameters, grades and `'A'`, are implicitly assigned to the formal parameters, data and target, as follows:

```
data = grades
target = 'A'
```

These assignment statements establish identifier data as an alias for grades and target as a name for the string literal `'A'`. (See Figure 1.7.)



Figure 1.7: A portrayal of parameter passing in Python, for the function call count(grades, `'A'`). Identifiers data and target are formal parameters defined within the local scope of the count function.

The communication of a return value from the function back to the caller is similarly implemented as an assignment. Therefore, with our sample invocation of prizes = count(grades, `'A'`), the identifier prizes in the caller's scope is assigned to the object that is identified as n in the return statement within our function body.

An advantage to Python's mechanism for passing information to and from a function is that objects are not copied. This ensures that the invocation of a function is efficient, even in a case where a parameter or return value is a complex object.

Mutable Parameters

Python's parameter passing model has additional implications when a parameter is a mutable object. Because the formal parameter is an alias for the actual parameter, the body of the function may interact with the object in ways that change its state. Considering again our sample invocation of the count function, if the body of the function executes the command data.append(`'F'`), the new entry is added to the end of the list identified as data within the function, which is one and the same as the list known to the caller as grades. As an aside, we note that reassigning a new value to a formal parameter with a function body, such as by setting data = [ ], does not alter the actual parameter; such a reassignment simply breaks the alias.

Our hypothetical example of a count method that appends a new element to a list lacks common sense. There is no reason to expect such a behavior, and it would be quite a poor design to have such an unexpected effect on the parameter. There are, however, many legitimate cases in which a function may be designed (and clearly documented) to modify the state of a parameter. As a concrete example, we present the following implementation of a method named scale that's primary purpose is to multiply all entries of a numeric data set by a given factor.

```
def scale(data, factor):
  for j in range(len(data)):
    data[j] *= factor
```

Default Parameter Values

Python provides means for functions to support more than one possible calling signature. Such a function is said to be ***polymorphic*** (which is Greek for "many forms"). Most notably, functions can declare one or more default values for parameters, thereby allowing the caller to invoke a function with varying numbers of actual parameters. As an artificial example, if a function is declared with signature

def foo(a, b=15, c=27):

there are three parameters, the last two of which offer default values. A caller is welcome to send three actual parameters, as in foo(4, 12, 8), in which case the default values are not used. If, on the other hand, the caller only sends one parameter, foo(4), the function will execute with parameters values a=4, b=15, c=27. If a caller sends two parameters, they are assumed to be the first two, with the third being the default. Thus, foo(8, 20) executes with a=8, b=20, c=27. However, it is illegal to define a function with a signature such as bar(a, b=15, c) with b having a default value, yet not the subsequent c; if a default parameter value is present for one parameter, it must be present for all further parameters.

As a more motivating example for the use of a default parameter, we revisit the task of computing a student's GPA (see Code Fragment 1.1). Rather than assume direct input and output with the console, we prefer to design a function that computes and returns a GPA. Our original implementation uses a fixed mapping from each letter grade (such as a B−) to a corresponding point value (such as 2.67). While that point system is somewhat common, it may not agree with the system used by all schools. (For example, some may assign an `'A+'` grade a value higher than 4.0.) Therefore, we design a compute_gpa function, given in Code Fragment 1.2, which allows the caller to specify a custom mapping from grades to values, while offering the standard point system as a default.

```
def compute_gpa(grades, points={'A+':4.0, 'A':4.0, 'A-':3.67, 'B+':3.33,
                                'B':3.0, 'B-':2.67,'C+':2.33, 'C':2.0,
                                'C':1.67, 'D+':1.33, 'D':1.0, 'F':0.0}):
  num_courses = 0
  total_points = 0
  for g in grades:
    if g in points:                          # a recognizable grade
      num_courses += 1
      total_points += points[g]
  return total_points / num_courses
```

Code Fragment 1.2: A function that computes a student's GPA with a point value system that can be customized as an optional parameter.

As an additional example of an interesting polymorphic function, we consider Python's support for range. (Technically, this is a constructor for the range class, but for the sake of this discussion, we can treat it as a pure function.) Three calling syntaxes are supported. The one-parameter form, range(n), generates a sequence of integers from 0 up to but not including *n*. A two-parameter form, range(start,stop) generates integers from start up to, but not including, stop. A three-parameter form, range(start, stop, step), generates a similar range as range(start, stop), but with increments of size step rather than 1.

This combination of forms seems to violate the rules for default parameters. In particular, when a single parameter is sent, as in range(n), it serves as the stop value (which is the second parameter); the value of start is effectively 0 in that case. However, this effect can be achieved with some sleight of hand, as follows:

```
def range(start, stop=None, step=1):
  if stop is None:
    stop = start
    start = 0
  ...
```

From a technical perspective, when range(n) is invoked, the actual parameter n will be assigned to formal parameter start. Within the body, if only one parameter is received, the start and stop values are reassigned to provide the desired semantics.

Keyword Parameters

The traditional mechanism for matching the actual parameters sent by a caller, to the formal parameters declared by the function signature is based on the concept of ***positional arguments***. For example, with signature foo(a=10, b=20, c=30), parameters sent by the caller are matched, in the given order, to the formal parameters. An invocation of foo(5) indicates that a=5, while b and c are assigned their default values.

Python supports an alternate mechanism for sending a parameter to a function known as a ***keyword argument***. A keyword argument is specified by explicitly assigning an actual parameter to a formal parameter by name. For example, with the above definition of function foo, a call foo(c=5) will invoke the function with parameters a=10, b=20, c=5.

A function's author can require that certain parameters be sent only through the keyword-argument syntax. We never place such a restriction in our own function definitions, but we will see several important uses of keyword-only parameters in Python's standard libraries. As an example, the built-in max function accepts a keyword parameter, coincidentally named key, that can be used to vary the notion of "maximum" that is used.

By default, max operates based upon the natural order of elements according to the $<$ operator for that type. But the maximum can be computed by comparing some other aspect of the elements. This is done by providing an auxiliary *function* that converts a natural element to some other value for the sake of comparison. For example, if we are interested in finding a numeric value with *magnitude* that is maximal (i.e., considering -35 to be larger than $+20$), we can use the calling syntax max(a, b, key=abs). In this case, the built-in abs function is itself sent as the value associated with the keyword parameter key. (Functions are first-class objects in Python; see Section 1.10.) When max is called in this way, it will compare abs(a) to abs(b), rather than a to b. The motivation for the keyword syntax as an alternate to positional arguments is important in the case of max. This function is polymorphic in the number of arguments, allowing a call such as max(a,b,c,d); therefore, it is not possible to designate a key function as a traditional positional element. Sorting functions in Python also support a similar key parameter for indicating a nonstandard order. (We explore this further in Section 9.4 and in Section 12.6.1, when discussing sorting algorithms).

1.5.2 Python's Built-In Functions

Table 1.4 provides an overview of common functions that are automatically available in Python, including the previously discussed abs, max, and range. When choosing names for the parameters, we use identifiers x, y, z for arbitrary numeric types, k for an integer, and a, b, and c for arbitrary comparable types. We use the identifier, iterable, to represent an instance of any iterable type (e.g., str, list, tuple, set, dict); we will discuss iterators and iterable data types in Section 1.8. A sequence represents a more narrow category of indexable classes, including str, list, and tuple, but neither set nor dict. Most of the entries in Table 1.4 can be categorized according to their functionality as follows:

Input/Output: print, input, and open will be more fully explained in Section 1.6.

Character Encoding: ord and chr relate characters and their integer code points. For example, ord('A') is 65 and chr(65) is 'A'.

Mathematics: abs, divmod, pow, round, and sum provide common mathematical functionality; an additional math module will be introduced in Section 1.11.

Ordering: max and min apply to any data type that supports a notion of comparison, or to any collection of such values. Likewise, sorted can be used to produce an ordered list of elements drawn from any existing collection.

Collections/Iterations: range generates a new sequence of numbers; len reports the length of any existing collection; functions reversed, all, any, and map operate on arbitrary iterations as well; iter and next provide a general framework for iteration through elements of a collection, and are discussed in Section 1.8.

Common Built-In Functions	
Calling Syntax	**Description**
abs(x)	Return the absolute value of a number.
all(iterable)	Return True if bool(e) is True for each element e.
any(iterable)	Return True if bool(e) is True for at least one element e.
chr(integer)	Return a one-character string with the given Unicode code point.
divmod(x, y)	Return (x // y, x % y) as tuple, if x and y are integers.
hash(obj)	Return an integer hash value for the object (see Chapter 10).
id(obj)	Return the unique integer serving as an "identity" for the object.
input(prompt)	Return a string from standard input; the prompt is optional.
isinstance(obj, cls)	Determine if obj is an instance of the class (or a subclass).
iter(iterable)	Return a new iterator object for the parameter (see Section 1.8).
len(iterable)	Return the number of elements in the given iteration.
map(f, iter1, iter2, ...)	Return an iterator yielding the result of function calls f(e1, e2, ...) for respective elements e1 $\in$ iter1, e2 $\in$ iter2, ...
max(iterable)	Return the largest element of the given iteration.
max(a, b, c, ...)	Return the largest of the arguments.
min(iterable)	Return the smallest element of the given iteration.
min(a, b, c, ...)	Return the smallest of the arguments.
next(iterator)	Return the next element reported by the iterator (see Section 1.8).
open(filename, mode)	Open a file with the given name and access mode.
ord(char)	Return the Unicode code point of the given character.
pow(x, y)	Return the value x^y (as an integer if x and y are integers); equivalent to x ** y.
pow(x, y, z)	Return the value $(x^y \bmod z)$ as an integer.
print(obj1, obj2, ...)	Print the arguments, with separating spaces and trailing newline.
range(stop)	Construct an iteration of values $0, 1, \ldots,$ stop -1.
range(start, stop)	Construct an iteration of values start, start $+1, \ldots,$ stop -1.
range(start, stop, step)	Construct an iteration of values start, start + step, start + 2*step, ...
reversed(sequence)	Return an iteration of the sequence in reverse.
round(x)	Return the nearest int value (a tie is broken toward the even value).
round(x, k)	Return the value rounded to the nearest 10^{-k} (return-type matches x).
sorted(iterable)	Return a list containing elements of the iterable in sorted order.
sum(iterable)	Return the sum of the elements in the iterable (must be numeric).
type(obj)	Return the class to which the instance obj belongs.

Table 1.4: Commonly used built-in function in Python.

1.6 Simple Input and Output

In this section, we address the basics of input and output in Python, describing standard input and output through the user console, and Python's support for reading and writing text files.

1.6.1 Console Input and Output

The print Function

The built-in function, print, is used to generate standard output to the console. In its simplest form, it prints an arbitrary sequence of arguments, separated by spaces, and followed by a trailing newline character. For example, the command print('maroon', 5) outputs the string 'maroon 5\n'. Note that arguments need not be string instances. A nonstring argument x will be displayed as str(x). Without any arguments, the command print( ) outputs a single newline character.

The print function can be customized through the use of the following keyword parameters (see Section 1.5 for a discussion of keyword parameters):

- By default, the print function inserts a separating space into the output between each pair of arguments. The separator can be customized by providing a desired separating string as a keyword parameter, sep. For example, colon-separated output can be produced as print(a, b, c, sep=':'). The separating string need not be a single character; it can be a longer string, and it can be the empty string, sep='', causing successive arguments to be directly concatenated.
- By default, a trailing newline is output after the final argument. An alternative trailing string can be designated using a keyword parameter, end. Designating the empty string end='' suppresses all trailing characters.
- By default, the print function sends its output to the standard console. However, output can be directed to a file by indicating an output file stream (see Section 1.6.2) using file as a keyword parameter.

The input Function

The primary means for acquiring information from the user console is a built-in function named input. This function displays a prompt, if given as an optional parameter, and then waits until the user enters some sequence of characters followed by the return key. The formal return value of the function is the string of characters that were entered strictly before the return key (i.e., no newline character exists in the returned string).

When reading a numeric value from the user, a programmer must use the input function to get the string of characters, and then use the int or float syntax to construct the numeric value that character string represents. That is, if a call to response = input( ) reports that the user entered the characters, '2013', the syntax int(response) could be used to produce the integer value 2013. It is quite common to combine these operations with a syntax such as

```
year = int(input('In what year were you born? '))
```

if we assume that the user will enter an appropriate response. (In Section 1.7 we discuss error handling in such a situation.)

Because input returns a string as its result, use of that function can be combined with the existing functionality of the string class, as described in Appendix A. For example, if the user enters multiple pieces of information on the same line, it is common to call the split method on the result, as in

```
reply = input('Enter x and y, separated by spaces: ')
pieces = reply.split( )    # returns a list of strings, as separated by spaces
x = float(pieces[0])
y = float(pieces[1])
```

A Sample Program

Here is a simple, but complete, program that demonstrates the use of the input and print functions. The tools for formatting the final output is discussed in Appendix A.

```
age = int(input('Enter your age in years: '))
max_heart_rate = 206.9 - (0.67 * age)    # as per Med Sci Sports Exerc.
target = 0.65 * max_heart_rate
print('Your target fat-burning heart rate is', target)
```

1.6.2 Files

Files are typically accessed in Python beginning with a call to a built-in function, named open, that returns a proxy for interactions with the underlying file. For example, the command, fp = open('sample.txt'), attempts to open a file named sample.txt, returning a proxy that allows read-only access to the text file.

The open function accepts an optional second parameter that determines the access mode. The default mode is 'r' for reading. Other common modes are 'w' for writing to the file (causing any existing file with that name to be overwritten), or 'a' for appending to the end of an existing file. Although we focus on use of text files, it is possible to work with binary files, using access modes such as 'rb' or 'wb'.

When processing a file, the proxy maintains a current position within the file as an offset from the beginning, measured in number of bytes. When opening a file with mode `'r'` or `'w'`, the position is initially 0; if opened in append mode, `'a'`, the position is initially at the end of the file. The syntax fp.close( ) closes the file associated with proxy fp, ensuring that any written contents are saved. A summary of methods for reading and writing a file is given in Table 1.5

Calling Syntax	Description
fp.read( )	Return the (remaining) contents of a readable file as a string.
fp.read(k)	Return the next k bytes of a readable file as a string.
fp.readline( )	Return (remainder of) the current line of a readable file as a string.
fp.readlines( )	Return all (remaining) lines of a readable file as a list of strings.
for line **in** fp:	Iterate all (remaining) lines of a readable file.
fp.seek(k)	Change the current position to be at the kth byte of the file.
fp.tell( )	Return the current position, measured as byte-offset from the start.
fp.write(string)	Write given string at current position of the writable file.
fp.writelines(seq)	Write each of the strings of the given sequence at the current position of the writable file. This command does *not* insert any newlines, beyond those that are embedded in the strings.
print(..., file=fp)	Redirect output of print function to the file.

Table 1.5: Behaviors for interacting with a text file via a file proxy (named fp).

Reading from a File

The most basic command for reading via a proxy is the read method. When invoked on file proxy fp, as fp.read(k), the command returns a string representing the next k bytes of the file, starting at the current position. Without a parameter, the syntax fp.read( ) returns the remaining contents of the file in entirety. For convenience, files can be read a line at a time, using the readline method to read one line, or the readlines method to return a list of all remaining lines. Files also support the for-loop syntax, with iteration being line by line (e.g., **for** line **in** fp:).

Writing to a File

When a file proxy is writable, for example, if created with access mode `'w'` or `'a'`, text can be written using methods write or writelines. For example, if we define fp = open(`'results.txt'`, `'w'`), the syntax fp.write(`'Hello World.\n'`) writes a single line to the file with the given string. Note well that write does not explicitly add a trailing newline, so desired newline characters must be embedded directly in the string parameter. Recall that the output of the print method can be redirected to a file using a keyword parameter, as described in Section 1.6.

1.7 Exception Handling

Exceptions are unexpected events that occur during the execution of a program. An exception might result from a logical error or an unanticipated situation. In Python, ***exceptions*** (also known as ***errors***) are objects that are ***raised*** (or ***thrown***) by code that encounters an unexpected circumstance. The Python interpreter can also raise an exception should it encounter an unexpected condition, like running out of memory. A raised error may be ***caught*** by a surrounding context that "handles" the exception in an appropriate fashion. If uncaught, an exception causes the interpreter to stop executing the program and to report an appropriate message to the console. In this section, we examine the most common error types in Python, the mechanism for catching and handling errors that have been raised, and the syntax for raising errors from within user-defined blocks of code.

Common Exception Types

Python includes a rich hierarchy of exception classes that designate various categories of errors; Table 1.6 shows many of those classes. The Exception class serves as a base class for most other error types. An instance of the various subclasses encodes details about a problem that has occurred. Several of these errors may be raised in exceptional cases by behaviors introduced in this chapter. For example, use of an undefined identifier in an expression causes a NameError, and errant use of the dot notation, as in foo.bar( ), will generate an AttributeError if object foo does not support a member named bar.

Class	Description
Exception	A base class for most error types
AttributeError	Raised by syntax obj.foo, if obj has no member named foo
EOFError	Raised if "end of file" reached for console or file input
IOError	Raised upon failure of I/O operation (e.g., opening file)
IndexError	Raised if index to sequence is out of bounds
KeyError	Raised if nonexistent key requested for set or dictionary
KeyboardInterrupt	Raised if user types ctrl-C while program is executing
NameError	Raised if nonexistent identifier used
StopIteration	Raised by next(iterator) if no element; see Section 1.8
TypeError	Raised when wrong type of parameter is sent to a function
ValueError	Raised when parameter has invalid value (e.g., sqrt(−5))
ZeroDivisionError	Raised when any division operator used with 0 as divisor

Table 1.6: Common exception classes in Python

Sending the wrong number, type, or value of parameters to a function is another common cause for an exception. For example, a call to abs('hello') will raise a TypeError because the parameter is not numeric, and a call to abs(3, 5) will raise a TypeError because one parameter is expected. A ValueError is typically raised when the correct number and type of parameters are sent, but a value is illegitimate for the context of the function. For example, the int constructor accepts a string, as with int('137'), but a ValueError is raised if that string does not represent an integer, as with int('3.14') or int('hello').

Python's sequence types (e.g., list, tuple, and str) raise an IndexError when syntax such as data[k] is used with an integer k that is not a valid index for the given sequence (as described in Section 1.2.3). Sets and dictionaries raise a KeyError when an attempt is made to access a nonexistent element.

1.7.1 Raising an Exception

An exception is thrown by executing the **raise** statement, with an appropriate instance of an exception class as an argument that designates the problem. For example, if a function for computing a square root is sent a negative value as a parameter, it can raise an exception with the command:

```
raise ValueError('x cannot be negative')
```

This syntax raises a newly created instance of the ValueError class, with the error message serving as a parameter to the constructor. If this exception is not caught within the body of the function, the execution of the function immediately ceases and the exception is propagated to the calling context (and possibly beyond).

When checking the validity of parameters sent to a function, it is customary to first verify that a parameter is of an appropriate type, and then to verify that it has an appropriate value. For example, the sqrt function in Python's math library performs error-checking that might be implemented as follows:

```
def sqrt(x):
  if not isinstance(x, (int, float)):
    raise TypeError('x must be numeric')
  elif x < 0:
    raise ValueError('x cannot be negative')
  # do the real work here...
```

Checking the type of an object can be performed at run-time using the built-in function, isinstance. In simplest form, isinstance(obj, cls) returns True if object, obj, is an instance of class, cls, or any subclass of that type. In the above example, a more general form is used with a tuple of allowable types indicated with the second parameter. After confirming that the parameter is numeric, the function enforces an expectation that the number be nonnegative, raising a ValueError otherwise.

How much error-checking to perform within a function is a matter of debate. Checking the type and value of each parameter demands additional execution time and, if taken to an extreme, seems counter to the nature of Python. Consider the built-in sum function, which computes a sum of a collection of numbers. An implementation with rigorous error-checking might be written as follows:

```
def sum(values):
  if not isinstance(values, collections.Iterable):
    raise TypeError('parameter must be an iterable type')
  total = 0
  for v in values:
    if not isinstance(v, (int, float)):
      raise TypeError('elements must be numeric')
    total = total+ v
  return total
```

The abstract base class, collections.Iterable, includes all of Python's iterable containers types that guarantee support for the for-loop syntax (e.g., list, tuple, set); we discuss iterables in Section 1.8, and the use of modules, such as collections, in Section 1.11. Within the body of the for loop, each element is verified as numeric before being added to the total. A far more direct and clear implementation of this function can be written as follows:

```
def sum(values):
  total = 0
  for v in values:
    total = total + v
  return total
```

Interestingly, this simple implementation performs exactly like Python's built-in version of the function. Even without the explicit checks, appropriate exceptions are raised naturally by the code. In particular, if values is not an iterable type, the attempt to use the for-loop syntax raises a TypeError reporting that the object is not iterable. In the case when a user sends an iterable type that includes a nonnumerical element, such as sum([3.14, 'oops']), a TypeError is naturally raised by the evaluation of expression total + v. The error message

```
unsupported operand type(s) for +:  'float' and 'str'
```

should be sufficiently informative to the caller. Perhaps slightly less obvious is the error that results from sum(['alpha', 'beta']). It will technically report a failed attempt to add an int and str, due to the initial evaluation of total + 'alpha', when total has been initialized to 0.

In the remainder of this book, we tend to favor the simpler implementations in the interest of clean presentation, performing minimal error-checking in most situations.

1.7.2 Catching an Exception

There are several philosophies regarding how to cope with possible exceptional cases when writing code. For example, if a division x/y is to be computed, there is clear risk that a ZeroDivisionError will be raised when variable y has value 0. In an ideal situation, the logic of the program may dictate that y has a nonzero value, thereby removing the concern for error. However, for more complex code, or in a case where the value of y depends on some external input to the program, there remains some possibility of an error.

One philosophy for managing exceptional cases is to ***"look before you leap."*** The goal is to entirely avoid the possibility of an exception being raised through the use of a proactive conditional test. Revisiting our division example, we might avoid the offending situation by writing:

```
if y != 0:
  ratio = x / y
else:
  ... do something else ...
```

A second philosophy, often embraced by Python programmers, is that ***"it is easier to ask for forgiveness than it is to get permission."*** This quote is attributed to Grace Hopper, an early pioneer in computer science. The sentiment is that we need not spend extra execution time safeguarding against every possible exceptional case, as long as there is a mechanism for coping with a problem after it arises. In Python, this philosophy is implemented using a ***try-except*** control structure. Revising our first example, the division operation can be guarded as follows:

```
try:
  ratio = x / y
except ZeroDivisionError:
  ... do something else ...
```

In this structure, the "try" block is the primary code to be executed. Although it is a single command in this example, it can more generally be a larger block of indented code. Following the try-block are one or more "except" cases, each with an identified error type and an indented block of code that should be executed if the designated error is raised within the try-block.

The relative advantage of using a try-except structure is that the non-exceptional case runs efficiently, without extraneous checks for the exceptional condition. However, handling the exceptional case requires slightly more time when using a try-except structure than with a standard conditional statement. For this reason, the try-except clause is best used when there is reason to believe that the exceptional case is relatively unlikely, or when it is prohibitively expensive to proactively evaluate a condition to avoid the exception.

Exception handling is particularly useful when working with user input, or when reading from or writing to files, because such interactions are inherently less predictable. In Section 1.6.2, we suggest the syntax, fp = open('sample.txt'), for opening a file with read access. That command may raise an IOError for a variety of reasons, such as a non-existent file, or lack of sufficient privilege for opening a file. It is significantly easier to attempt the command and catch the resulting error than it is to accurately predict whether the command will succeed.

We continue by demonstrating a few other forms of the try-except syntax. Exceptions are objects that can be examined when caught. To do so, an identifier must be established with a syntax as follows:

```
try:
  fp = open('sample.txt')
except IOError as e:
  print('Unable to open the file:', e)
```

In this case, the name, e, denotes the instance of the exception that was thrown, and printing it causes a detailed error message to be displayed (e.g., "file not found").

A try-statement may handle more than one type of exception. For example, consider the following command from Section 1.6.1:

```
age = int(input('Enter your age in years: '))
```

This command could fail for a variety of reasons. The call to input will raise an EOFError if the console input fails. If the call to input completes successfully, the int constructor raises a ValueError if the user has not entered characters representing a valid integer. If we want to handle two or more types of errors in the same way, we can use a single except-statement, as in the following example:

```
age = -1                    # an initially invalid choice
while age <= 0:
  try:
    age = int(input('Enter your age in years: '))
    if age <= 0:
      print('Your age must be positive')
  except (ValueError, EOFError):
    print('Invalid response')
```

We use the tuple, (ValueError, EOFError), to designate the types of errors that we wish to catch with the except-clause. In this implementation, we catch either error, print a response, and continue with another pass of the enclosing while loop. We note that when an error is raised within the try-block, the remainder of that body is immediately skipped. In this example, if the exception arises within the call to input, or the subsequent call to the int constructor, the assignment to age never occurs, nor the message about needing a positive value. Because the value of age

will be unchanged, the while loop will continue. If we preferred to have the while loop continue without printing the `'Invalid response'` message, we could have written the exception-clause as

```
except (ValueError, EOFError):
  pass
```

The keyword, **pass**, is a statement that does nothing, yet it can serve syntactically as a body of a control structure. In this way, we quietly catch the exception, thereby allowing the surrounding while loop to continue.

In order to provide different responses to different types of errors, we may use two or more except-clauses as part of a try-structure. In our previous example, an EOFError suggests a more insurmountable error than simply an errant value being entered. In that case, we might wish to provide a more specific error message, or perhaps to allow the exception to interrupt the loop and be propagated to a higher context. We could implement such behavior as follows:

```
age = -1                    # an initially invalid choice
while age <= 0:
  try:
    age = int(input('Enter your age in years: '))
    if age <= 0:
      print('Your age must be positive')
  except ValueError:
    print('That is an invalid age specification')
  except EOFError:
    print('There was an unexpected error reading input.')
    raise                   # let's re-raise this exception
```

In this implementation, we have separate except-clauses for the ValueError and EOFError cases. The body of the clause for handling an EOFError relies on another technique in Python. It uses the raise statement without any subsequent argument, to re-raise the same exception that is currently being handled. This allows us to provide our own response to the exception, and then to interrupt the while loop and propagate the exception upward.

In closing, we note two additional features of try-except structures in Python. It is permissible to have a final except-clause without any identified error types, using syntax **except**:, to catch any other exceptions that occurred. However, this technique should be used sparingly, as it is difficult to suggest how to handle an error of an unknown type. A try-statement can have a **finally** clause, with a body of code that will always be executed in the standard or exceptional cases, even when an uncaught or re-raised exception occurs. That block is typically used for critical cleanup work, such as closing an open file.

1.8 Iterators and Generators

In Section 1.4.2, we introduced the for-loop syntax beginning as:

for *element* **in** *iterable*:

and we noted that there are many types of objects in Python that qualify as being iterable. Basic container types, such as list, tuple, and set, qualify as iterable types. Furthermore, a string can produce an iteration of its characters, a dictionary can produce an iteration of its keys, and a file can produce an iteration of its lines. User-defined types may also support iteration. In Python, the mechanism for iteration is based upon the following conventions:

- An ***iterator*** is an object that manages an iteration through a series of values. If variable, i, identifies an iterator object, then each call to the built-in function, next(i), produces a subsequent element from the underlying series, with a StopIteration exception raised to indicate that there are no further elements.
- An ***iterable*** is an object, obj, that produces an *iterator* via the syntax iter(obj).

By these definitions, an instance of a list is an iterable, but not itself an iterator. With data = [1, 2, 4, 8], it is not legal to call next(data). However, an iterator object can be produced with syntax, i = iter(data), and then each subsequent call to next(i) will return an element of that list. The for-loop syntax in Python simply automates this process, creating an iterator for the give iterable, and then repeatedly calling for the next element until catching the StopIteration exception.

More generally, it is possible to create multiple iterators based upon the same iterable object, with each iterator maintaining its own state of progress. However, iterators typically maintain their state with indirect reference back to the original collection of elements. For example, calling iter(data) on a list instance produces an instance of the list_iterator class. That iterator does not store its own copy of the list of elements. Instead, it maintains a current *index* into the original list, representing the next element to be reported. Therefore, if the contents of the original list are modified after the iterator is constructed, but before the iteration is complete, the iterator will be reporting the *updated* contents of the list.

Python also supports functions and classes that produce an implicit iterable series of values, that is, without constructing a data structure to store all of its values at once. For example, the call range(1000000) does *not* return a list of numbers; it returns a range object that is iterable. This object generates the million values one at a time, and only as needed. Such a ***lazy evaluation*** technique has great advantage. In the case of range, it allows a loop of the form, **for** j **in** range(1000000):, to execute without setting aside memory for storing one million values. Also, if such a loop were to be interrupted in some fashion, no time will have been spent computing unused values of the range.

We see lazy evaluation used in many of Python's libraries. For example, the dictionary class supports methods keys( ), values( ), and items( ), which respectively produce a "view" of all keys, values, or (key,value) pairs within a dictionary. None of these methods produces an explicit list of results. Instead, the views that are produced are iterable objects based upon the actual contents of the dictionary. An explicit list of values from such an iteration can be immediately constructed by calling the list class constructor with the iteration as a parameter. For example, the syntax list(range(1000)) produces a list instance with values from 0 to 999, while the syntax list(d.values( )) produces a list that has elements based upon the current values of dictionary d. We can similarly construct a tuple or set instance based upon a given iterable.

Generators

In Section 2.3.4, we will explain how to define a class whose instances serve as iterators. However, the most convenient technique for creating iterators in Python is through the use of ***generators***. A generator is implemented with a syntax that is very similar to a function, but instead of returning values, a **yield** statement is executed to indicate each element of the series. As an example, consider the goal of determining all factors of a positive integer. For example, the number 100 has factors 1, 2, 4, 5, 10, 20, 25, 50, 100. A traditional function might produce and return a list containing all factors, implemented as:

```
def factors(n):                    # traditional function that computes factors
  results = [ ]                    # store factors in a new list
  for k in range(1,n+1):
    if n % k == 0:                 # divides evenly, thus k is a factor
      results.append(k)            # add k to the list of factors
  return results                   # return the entire list
```

In contrast, an implementation of a *generator* for computing those factors could be implemented as follows:

```
def factors(n):                    # generator that computes factors
  for k in range(1,n+1):
    if n % k == 0:                 # divides evenly, thus k is a factor
      yield k                      # yield this factor as next result
```

Notice use of the keyword **yield** rather than **return** to indicate a result. This indicates to Python that we are defining a generator, rather than a traditional function. It is illegal to combine yield and return statements in the same implementation, other than a zero-argument return statement to cause a generator to end its execution. If a programmer writes a loop such as **for** factor **in** factors(100):, an instance of our generator is created. For each iteration of the loop, Python executes our procedure

until a yield statement indicates the next value. At that point, the procedure is temporarily interrupted, only to be resumed when another value is requested. When the flow of control naturally reaches the end of our procedure (or a zero-argument return statement), a StopIteration exception is automatically raised. Although this particular example uses a single yield statement in the source code, a generator can rely on multiple yield statements in different constructs, with the generated series determined by the natural flow of control. For example, we can greatly improve the efficiency of our generator for computing factors of a number, n, by only testing values up to the square root of that number, while reporting the factor $n//k$ that is associated with each k (unless $n//k$ equals k). We might implement such a generator as follows:

```
def factors(n):                # generator that computes factors
  k = 1
  while k * k < n:             # while k < sqrt(n)
    if n % k == 0:
      yield k
      yield n // k
    k += 1
  if k * k == n:               # special case if n is perfect square
    yield k
```

We should note that this generator differs from our first version in that the factors are not generated in strictly increasing order. For example, factors(100) generates the series $1, 100, 2, 50, 4, 25, 5, 20, 10$.

In closing, we wish to emphasize the benefits of lazy evaluation when using a generator rather than a traditional function. The results are only computed if requested, and the entire series need not reside in memory at one time. In fact, a generator can effectively produce an infinite series of values. As an example, the Fibonacci numbers form a classic mathematical sequence, starting with value 0, then value 1, and then each subsequent value being the sum of the two preceding values. Hence, the Fibonacci series begins as: $0, 1, 1, 2, 3, 5, 8, 13, \ldots$. The following generator produces this infinite series.

```
def fibonacci( ):
  a = 0
  b = 1
  while True:                  # keep going...
    yield a                    # report value, a, during this pass
    future = a + b
    a = b                      # this will be next value reported
    b = future                 # and subsequently this
```

1.9 Additional Python Conveniences

In this section, we introduce several features of Python that are particularly convenient for writing clean, concise code. Each of these syntaxes provide functionality that could otherwise be accomplished using functionality that we have introduced earlier in this chapter. However, at times, the new syntax is a more clear and direct expression of the logic.

1.9.1 Conditional Expressions

Python supports a ***conditional expression*** syntax that can replace a simple control structure. The general syntax is an expression of the form:

expr1 **if** *condition* **else** *expr2*

This compound expression evaluates to *expr1* if the condition is true, and otherwise evaluates to *expr2*. For those familiar with Java or C++, this is equivalent to the syntax, *condition* ? *expr1* : *expr2*, in those languages.

As an example, consider the goal of sending the absolute value of a variable, n, to a function (and without relying on the built-in abs function, for the sake of example). Using a traditional control structure, we might accomplish this as follows:

```
if n >= 0:
    param = n
else:
    param = -n
result = foo(param)                # call the function
```

With the conditional expression syntax, we can directly assign a value to variable, param, as follows:

```
param = n if n >= 0 else -n        # pick the appropriate value
result = foo(param)                # call the function
```

In fact, there is no need to assign the compound expression to a variable. A conditional expression can itself serve as a parameter to the function, written as follows:

```
result = foo(n if n >= 0 else -n)
```

Sometimes, the mere shortening of source code is advantageous because it avoids the distraction of a more cumbersome control structure. However, we recommend that a conditional expression be used only when it improves the readability of the source code, and when the first of the two options is the more "natural" case, given its prominence in the syntax. (We prefer to view the alternative value as more exceptional.)

1.9.2 Comprehension Syntax

A very common programming task is to produce one series of values based upon the processing of another series. Often, this task can be accomplished quite simply in Python using what is known as a ***comprehension syntax***. We begin by demonstrating ***list comprehension***, as this was the first form to be supported by Python. Its general form is as follows:

```
[ expression for value in iterable if condition ]
```

We note that both *expression* and *condition* may depend on *value*, and that the if-clause is optional. The evaluation of the comprehension is logically equivalent to the following traditional control structure for computing a resulting list:

```
result = [ ]
for value in iterable:
  if condition:
    result.append(expression)
```

As a concrete example, a list of the squares of the numbers from 1 to n, that is $[1, 4, 9, 16, 25, \ldots, n^2]$, can be created by traditional means as follows:

```
squares = [ ]
for k in range(1, n+1):
  squares.append(k*k)
```

With list comprehension, this logic is expressed as follows:

```
squares = [k*k for k in range(1, n+1)]
```

As a second example, Section 1.8 introduced the goal of producing a list of factors for an integer n. That task is accomplished with the following list comprehension:

```
factors = [k for k in range(1,n+1) if n % k == 0]
```

Python supports similar comprehension syntaxes that respectively produce a set, generator, or dictionary. We compare those syntaxes using our example for producing the squares of numbers.

```
[ k*k for k in range(1, n+1) ]          list comprehension
{ k*k for k in range(1, n+1) }          set comprehension
( k*k for k in range(1, n+1) )          generator comprehension
{ k : k*k for k in range(1, n+1) }      dictionary comprehension
```

The generator syntax is particularly attractive when results do not need to be stored in memory. For example, to compute the sum of the first n squares, the generator syntax, `total = sum(k*k for k in range(1, n+1))`, is preferred to the use of an explicitly instantiated list comprehension as the parameter.

1.9.3 Packing and Unpacking of Sequences

Python provides two additional conveniences involving the treatment of tuples and other sequence types. The first is rather cosmetic. If a series of comma-separated expressions are given in a larger context, they will be treated as a single tuple, even if no enclosing parentheses are provided. For example, the assignment

```
data = 2, 4, 6, 8
```

results in identifier, data, being assigned to the tuple (2, 4, 6, 8). This behavior is called ***automatic packing*** of a tuple. One common use of packing in Python is when returning multiple values from a function. If the body of a function executes the command,

```
return x, y
```

it will be formally returning a single object that is the tuple (x, y).

As a dual to the packing behavior, Python can automatically ***unpack*** a sequence, allowing one to assign a series of individual identifiers to the elements of sequence. As an example, we can write

```
a, b, c, d = range(7, 11)
```

which has the effect of assigning a=7, b=8, c=9, and d=10, as those are the four values in the sequence returned by the call to range. For this syntax, the right-hand side expression can be any *iterable* type, as long as the number of variables on the left-hand side is the same as the number of elements in the iteration.

This technique can be used to unpack tuples returned by a function. For example, the built-in function, divmod(a, b), returns the pair of values (a // b, a % b) associated with an integer division. Although the caller can consider the return value to be a single tuple, it is possible to write

```
quotient, remainder = divmod(a, b)
```

to separately identify the two entries of the returned tuple. This syntax can also be used in the context of a for loop, when iterating over a sequence of iterables, as in

```
for x, y in [ (7, 2), (5, 8), (6, 4) ]:
```

In this example, there will be three iterations of the loop. During the first pass, x=7 and y=2, and so on. This style of loop is quite commonly used to iterate through key-value pairs that are returned by the items( ) method of the dict class, as in:

```
for k, v in mapping.items( ):
```

Simultaneous Assignments

The combination of automatic packing and unpacking forms a technique known as ***simultaneous assignment***, whereby we explicitly assign a series of values to a series of identifiers, using a syntax:

```
x, y, z = 6, 2, 5
```

In effect, the right-hand side of this assignment is automatically packed into a tuple, and then automatically unpacked with its elements assigned to the three identifiers on the left-hand side.

When using a simultaneous assignment, all of the expressions are evaluated on the right-hand side before any of the assignments are made to the left-hand variables. This is significant, as it provides a convenient means for swapping the values associated with two variables:

```
j, k = k, j
```

With this command, j will be assigned to the *old* value of k, and k will be assigned to the *old* value of j. Without simultaneous assignment, a swap typically requires more delicate use of a temporary variable, such as

```
temp = j
j = k
k = temp
```

With the simultaneous assignment, the unnamed tuple representing the packed values on the right-hand side implicitly serves as the temporary variable when performing such a swap.

The use of simultaneous assignments can greatly simplify the presentation of code. As an example, we reconsider the generator on page 41 that produces the Fibonacci series. The original code requires separate initialization of variables a and b to begin the series. Within each pass of the loop, the goal was to reassign a and b, respectively, to the values of b and a+b. At the time, we accomplished this with brief use of a third variable. With simultaneous assignments, that generator can be implemented more directly as follows:

```
def fibonacci( ):
  a, b = 0, 1
  while True:
    yield a
    a, b = b, a+b
```

1.10 Scopes and Namespaces

When computing a sum with the syntax x + y in Python, the names x and y must have been previously associated with objects that serve as values; a NameError will be raised if no such definitions are found. The process of determining the value associated with an identifier is known as ***name resolution***.

Whenever an identifier is assigned to a value, that definition is made with a specific ***scope***. Top-level assignments are typically made in what is known as ***global*** scope. Assignments made within the body of a function typically have scope that is ***local*** to that function call. Therefore, an assignment, x = 5, within a function has no effect on the identifier, x, in the broader scope.

Each distinct scope in Python is represented using an abstraction known as a ***namespace***. A namespace manages all identifiers that are currently defined in a given scope. Figure 1.8 portrays two namespaces, one being that of a caller to our count function from Section 1.5, and the other being the local namespace during the execution of that function.

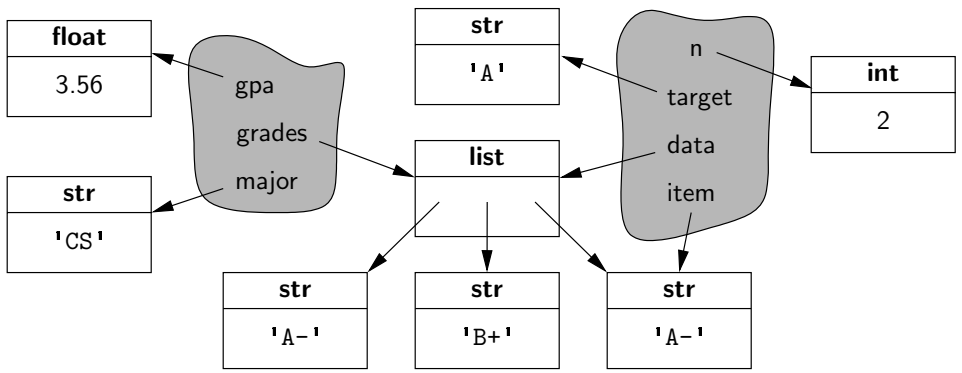


Figure 1.8: A portrayal of the two namespaces associated with a user's call count(grades, 'A'), as defined in Section 1.5. The left namespace is the caller's and the right namespace represents the local scope of the function.

Python implements a namespace with its own dictionary that maps each identifying string (e.g., 'n') to its associated value. Python provides several ways to examine a given namespace. The function, dir, reports the names of the identifiers in a given namespace (i.e., the keys of the dictionary), while the function, vars, returns the full dictionary. By default, calls to dir( ) and vars( ) report on the most locally enclosing namespace in which they are executed.

When an identifier is indicated in a command, Python searches a series of namespaces in the process of name resolution. First, the most locally enclosing scope is searched for a given name. If not found there, the next outer scope is searched, and so on. We will continue our examination of namespaces, in Section 2.5, when discussing Python's treatment of object-orientation. We will see that each object has its own namespace to store its attributes, and that classes each have a namespace as well.

First-Class Objects

In the terminology of programming languages, ***first-class objects*** are instances of a type that can be assigned to an identifier, passed as a parameter, or returned by a function. All of the data types we introduced in Section 1.2.3, such as int and list, are clearly first-class types in Python. In Python, functions and classes are also treated as first-class objects. For example, we could write the following:

```
scream = print       # assign name 'scream' to the function denoted as 'print'
scream('Hello')     # call that function
```

In this case, we have not created a new function, we have simply defined scream as an alias for the existing print function. While there is little motivation for precisely this example, it demonstrates the mechanism that is used by Python to allow one function to be passed as a parameter to another. On page 28, we noted that the built-in function, max, accepts an optional keyword parameter to specify a non-default order when computing a maximum. For example, a caller can use the syntax, max(a, b, key=abs), to determine which value has the larger absolute value. Within the body of that function, the formal parameter, key, is an identifier that will be assigned to the actual parameter, abs.

In terms of namespaces, an assignment such as scream = print, introduces the identifier, scream, into the current namespace, with its value being the object that represents the built-in function, print. The same mechanism is applied when a user-defined function is declared. For example, our count function from Section 1.5 beings with the following syntax:

```
def count(data, target):
  ...
```

Such a declaration introduces the identifier, count, into the current namespace, with the value being a function instance representing its implementation. In similar fashion, the name of a newly defined class is associated with a representation of that class as its value. (Class definitions will be introduced in the next chapter.)

1.11 Modules and the Import Statement

We have already introduced many functions (e.g., max) and classes (e.g., list) that are defined within Python's built-in namespace. Depending on the version of Python, there are approximately 130–150 definitions that were deemed significant enough to be included in that built-in namespace.

Beyond the built-in definitions, the standard Python distribution includes perhaps tens of thousands of other values, functions, and classes that are organized in additional libraries, known as ***modules***, that can be ***imported*** from within a program. As an example, we consider the math module. While the built-in namespace includes a few mathematical functions (e.g., abs, min, max, round), many more are relegated to the math module (e.g., sin, cos, sqrt). That module also defines approximate values for the mathematical constants, pi and e.

Python's **import** statement loads definitions from a module into the current namespace. One form of an import statement uses a syntax such as the following:

```
from math import pi, sqrt
```

This command adds both pi and sqrt, as defined in the math module, into the current namespace, allowing direct use of the identifier, pi, or a call of the function, sqrt(2). If there are many definitions from the same module to be imported, an asterisk may be used as a wild card, as in, **from** math **import** $*$, but this form should be used sparingly. The danger is that some of the names defined in the module may conflict with names already in the current namespace (or being imported from another module), and the import causes the new definitions to replace existing ones.

Another approach that can be used to access many definitions from the same module is to import the module itself, using a syntax such as:

```
import math
```

Formally, this adds the identifier, math, to the current namespace, with the module as its value. (Modules are also first-class objects in Python.) Once imported, individual definitions from the module can be accessed using a fully-qualified name, such as math.pi or math.sqrt(2).

Creating a New Module

To create a new module, one simply has to put the relevant definitions in a file named with a `.py` suffix. Those definitions can be imported from any other `.py` file within the same project directory. For example, if we were to put the definition of our count function (see Section 1.5) into a file named `utility.py`, we could import that function using the syntax, **from** utility **import** count.

It is worth noting that top-level commands with the module source code are executed when the module is first imported, almost as if the module were its own script. There is a special construct for embedding commands within the module that will be executed if the module is directly invoked as a script, but not when the module is imported from another script. Such commands should be placed in a body of a conditional statement of the following form,

```
if __name__ == '__main__':
```

Using our hypothetical `utility.py` module as an example, such commands will be executed if the interpreter is started with a command `python utility.py`, but not when the utility module is imported into another context. This approach is often used to embed what are known as ***unit tests*** within the module; we will discuss unit testing further in Section 2.2.4.

1.11.1 Existing Modules

Table 1.7 provides a summary of a few available modules that are relevant to a study of data structures. We have already discussed the math module briefly. In the remainder of this section, we highlight another module that is particularly important for some of the data structures and algorithms that we will study later in this book.

Existing Modules	
Module Name	**Description**
array	Provides compact array storage for primitive types.
collections	Defines additional data structures and abstract base classes involving collections of objects.
copy	Defines general functions for making copies of objects.
heapq	Provides heap-based priority queue functions (see Section 9.3.7).
math	Defines common mathematical constants and functions.
os	Provides support for interactions with the operating system.
random	Provides random number generation.
re	Provides support for processing regular expressions.
sys	Provides additional level of interaction with the Python interpreter.
time	Provides support for measuring time, or delaying a program.

Table 1.7: Some existing Python modules relevant to data structures and algorithms.

Pseudo-Random Number Generation

Python's random module provides the ability to generate pseudo-random numbers, that is, numbers that are statistically random (but not necessarily truly random). A ***pseudo-random number generator*** uses a deterministic formula to generate the

next number in a sequence based upon one or more past numbers that it has generated. Indeed, a simple yet popular pseudo-random number generator chooses its next number based solely on the most recently chosen number and some additional parameters using the following formula.

```
next = (a*current + b) % n;
```

where a, b, and n are appropriately chosen integers. Python uses a more advanced technique known as a ***Mersenne twister***. It turns out that the sequences generated by these techniques can be proven to be statistically uniform, which is usually good enough for most applications requiring random numbers, such as games. For applications, such as computer security settings, where one needs unpredictable random sequences, this kind of formula should not be used. Instead, one should ideally sample from a source that is actually random, such as radio static coming from outer space.

Since the next number in a pseudo-random generator is determined by the previous number(s), such a generator always needs a place to start, which is called its ***seed***. The sequence of numbers generated for a given seed will always be the same. One common trick to get a different sequence each time a program is run is to use a seed that will be different for each run. For example, we could use some timed input from a user or the current system time in milliseconds.

Python's random module provides support for pseudo-random number generation by defining a Random class; instances of that class serve as generators with independent state. This allows different aspects of a program to rely on their own pseudo-random number generator, so that calls to one generator do not affect the sequence of numbers produced by another. For convenience, all of the methods supported by the Random class are also supported as stand-alone functions of the random module (essentially using a single generator instance for all top-level calls).

Syntax	Description
seed(hashable)	Initializes the pseudo-random number generator based upon the hash value of the parameter
random( )	Returns a pseudo-random floating-point value in the interval $[0.0, 1.0)$.
randint(a,b)	Returns a pseudo-random integer in the closed interval $[a, b]$.
randrange(start, stop, step)	Returns a pseudo-random integer in the standard Python range indicated by the parameters.
choice(seq)	Returns an element of the given sequence chosen pseudo-randomly.
shuffle(seq)	Reorders the elements of the given sequence pseudo-randomly.

Table 1.8: Methods supported by instances of the Random class, and as top-level functions of the random module.

1.12 Exercises

For help with exercises, please visit the site, www.wiley.com/college/goodrich.

Reinforcement

R-1.1 Write a short Python function, is_multiple(n, m), that takes two integer values and returns True if n is a multiple of m, that is, $n = mi$ for some integer i, and False otherwise.

R-1.2 Write a short Python function, is_even(k), that takes an integer value and returns True if k is even, and **False** otherwise. However, your function cannot use the multiplication, modulo, or division operators.

R-1.3 Write a short Python function, minmax(data), that takes a sequence of one or more numbers, and returns the smallest and largest numbers, in the form of a tuple of length two. Do not use the built-in functions min or max in implementing your solution.

R-1.4 Write a short Python function that takes a positive integer n and returns the sum of the squares of all the positive integers smaller than n.

R-1.5 Give a single command that computes the sum from Exercise R-1.4, relying on Python's comprehension syntax and the built-in sum function.

R-1.6 Write a short Python function that takes a positive integer n and returns the sum of the squares of all the odd positive integers smaller than n.

R-1.7 Give a single command that computes the sum from Exercise R-1.6, relying on Python's comprehension syntax and the built-in sum function.

R-1.8 Python allows negative integers to be used as indices into a sequence, such as a string. If string s has length n, and expression s[k] is used for index $-n \leq k < 0$, what is the equivalent index $j \geq 0$ such that s[j] references the same element?

R-1.9 What parameters should be sent to the range constructor, to produce a range with values 50, 60, 70, 80?

R-1.10 What parameters should be sent to the range constructor, to produce a range with values 8, 6, 4, 2, 0, -2, -4, -6, -8?

R-1.11 Demonstrate how to use Python's list comprehension syntax to produce the list [1, 2, 4, 8, 16, 32, 64, 128, 256].

R-1.12 Python's random module includes a function choice(data) that returns a random element from a non-empty sequence. The random module includes a more basic function randrange, with parameterization similar to the built-in range function, that return a random choice from the given range. Using only the randrange function, implement your own version of the choice function.

Creativity

C-1.13 Write a pseudo-code description of a function that reverses a list of n integers, so that the numbers are listed in the opposite order than they were before, and compare this method to an equivalent Python function for doing the same thing.

C-1.14 Write a short Python function that takes a sequence of integer values and determines if there is a distinct pair of numbers in the sequence whose product is odd.

C-1.15 Write a Python function that takes a sequence of numbers and determines if all the numbers are different from each other (that is, they are distinct).

C-1.16 In our implementation of the scale function (page 25), the body of the loop executes the command data[j] *= factor. We have discussed that numeric types are immutable, and that use of the *= operator in this context causes the creation of a new instance (not the mutation of an existing instance). How is it still possible, then, that our implementation of scale changes the actual parameter sent by the caller?

C-1.17 Had we implemented the scale function (page 25) as follows, does it work properly?

```
def scale(data, factor):
  for val in data:
    val *= factor
```

Explain why or why not.

C-1.18 Demonstrate how to use Python's list comprehension syntax to produce the list [0, 2, 6, 12, 20, 30, 42, 56, 72, 90].

C-1.19 Demonstrate how to use Python's list comprehension syntax to produce the list ['a', 'b', 'c', ..., 'z'], but without having to type all 26 such characters literally.

C-1.20 Python's random module includes a function shuffle(data) that accepts a list of elements and randomly reorders the elements so that each possible order occurs with equal probability. The random module includes a more basic function randint(a, b) that returns a uniformly random integer from a to b (including both endpoints). Using only the randint function, implement your own version of the shuffle function.

C-1.21 Write a Python program that repeatedly reads lines from standard input until an EOFError is raised, and then outputs those lines in reverse order (a user can indicate end of input by typing ctrl-D).

C-1.22 Write a short Python program that takes two arrays a and b of length n storing **int** values, and returns the dot product of a and b. That is, it returns an array c of length n such that $c[i] = a[i] \cdot b[i]$, for $i = 0, \ldots, n-1$.

C-1.23 Give an example of a Python code fragment that attempts to write an element to a list based on an index that may be out of bounds. If that index is out of bounds, the program should catch the exception that results, and print the following error message:
"`Don't try buffer overflow attacks in Python!`"

C-1.24 Write a short Python function that counts the number of vowels in a given character string.

C-1.25 Write a short Python function that takes a string s, representing a sentence, and returns a copy of the string with all punctuation removed. For example, if given the string `"Let's try, Mike."`, this function would return `"Lets try Mike"`.

C-1.26 Write a short program that takes as input three integers, a, b, and c, from the console and determines if they can be used in a correct arithmetic formula (in the given order), like "$a+b=c$," "$a=b-c$," or "$a*b=c$."

C-1.27 In Section 1.8, we provided three different implementations of a generator that computes factors of a given integer. The third of those implementations, from page 41, was the most efficient, but we noted that it did not yield the factors in increasing order. Modify the generator so that it reports factors in increasing order, while maintaining its general performance advantages.

C-1.28 The ***p-norm*** of a vector $v = (v_1, v_2, \ldots, v_n)$ in n-dimensional space is defined as

$$\|v\| = \sqrt[p]{v_1^p + v_2^p + \cdots + v_n^p}.$$

For the special case of $p = 2$, this results in the traditional ***Euclidean norm***, which represents the length of the vector. For example, the Euclidean norm of a two-dimensional vector with coordinates $(4,3)$ has a Euclidean norm of $\sqrt{4^2+3^2} = \sqrt{16+9} = \sqrt{25} = 5$. Give an implementation of a function named `norm` such that `norm(v, p)` returns the p-norm value of v and `norm(v)` returns the Euclidean norm of v. You may assume that v is a list of numbers.

Projects

P-1.29 Write a Python program that outputs all possible strings formed by using the characters `'c'`, `'a'`, `'t'`, `'d'`, `'o'`, and `'g'` exactly once.

P-1.30 Write a Python program that can take a positive integer greater than 2 as input and write out the number of times one must repeatedly divide this number by 2 before getting a value less than 2.

P-1.31 Write a Python program that can "make change." Your program should take two numbers as input, one that is a monetary amount charged and the other that is a monetary amount given. It should then return the number of each kind of bill and coin to give back as change for the difference between the amount given and the amount charged. The values assigned to the bills and coins can be based on the monetary system of any current or former government. Try to design your program so that it returns as few bills and coins as possible.

P-1.32 Write a Python program that can simulate a simple calculator, using the console as the exclusive input and output device. That is, each input to the calculator, be it a number, like `12.34` or `1034`, or an operator, like `+` or `=`, can be done on a separate line. After each such input, you should output to the Python console what would be displayed on your calculator.

P-1.33 Write a Python program that simulates a handheld calculator. Your program should process input from the Python console representing buttons that are "pushed," and then output the contents of the screen after each operation is performed. Minimally, your calculator should be able to process the basic arithmetic operations and a reset/clear operation.

P-1.34 A common punishment for school children is to write out a sentence multiple times. Write a Python stand-alone program that will write out the following sentence one hundred times: "I will never spam my friends again." Your program should number each of the sentences and it should make eight different random-looking typos.

P-1.35 The ***birthday paradox*** says that the probability that two people in a room will have the same birthday is more than half, provided n, the number of people in the room, is more than 23. This property is not really a paradox, but many people find it surprising. Design a Python program that can test this paradox by a series of experiments on randomly generated birthdays, which test this paradox for $n = 5, 10, 15, 20, \ldots, 100$.

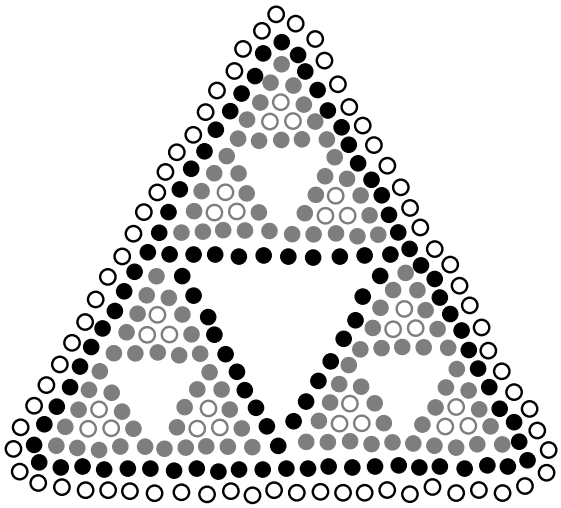
P-1.36 Write a Python program that inputs a list of words, separated by whitespace, and outputs how many times each word appears in the list. You need not worry about efficiency at this point, however, as this topic is something that will be addressed later in this book.

Chapter Notes

The official Python Web site (http://www.python.org) has a wealth of information, including a tutorial and full documentation of the built-in functions, classes, and standard modules. The Python interpreter is itself a useful reference, as the interactive command help(foo) provides documentation for any function, class, or module that foo identifies.

Books providing an introduction to programming in Python include titles authored by Campbell *et al.* [22], Cedar [25], Dawson [32], Goldwasser and Letscher [43], Lutz [72], Perkovic [82], and Zelle [105]. More complete reference books on Python include titles by Beazley [12], and Summerfield [91].

Chapter 4 Recursion



Contents

4.1 Illustrative Examples 150
4.1.1 The Factorial Function 150
4.1.2 Drawing an English Ruler 152
4.1.3 Binary Search 155
4.1.4 File Systems 157
4.2 Analyzing Recursive Algorithms 161
4.3 Recursion Run Amok 165
4.3.1 Maximum Recursive Depth in Python 168
4.4 Further Examples of Recursion 169
4.4.1 Linear Recursion 169
4.4.2 Binary Recursion 174
4.4.3 Multiple Recursion 175
4.5 Designing Recursive Algorithms 177
4.6 Eliminating Tail Recursion 178
4.7 Exercises 180

One way to describe repetition within a computer program is the use of loops, such as Python's while-loop and for-loop constructs described in Section 1.4.2. An entirely different way to achieve repetition is through a process known as ***recursion***.

Recursion is a technique by which a function makes one or more calls to itself during execution, or by which a data structure relies upon smaller instances of the very same type of structure in its representation. There are many examples of recursion in art and nature. For example, fractal patterns are naturally recursive. A physical example of recursion used in art is in the Russian Matryoshka dolls. Each doll is either made of solid wood, or is hollow and contains another Matryoshka doll inside it.

In computing, recursion provides an elegant and powerful alternative for performing repetitive tasks. In fact, a few programming languages (e.g., Scheme, Smalltalk) do not explicitly support looping constructs and instead rely directly on recursion to express repetition. Most modern programming languages support functional recursion using the identical mechanism that is used to support traditional forms of function calls. When one invocation of the function make a recursive call, that invocation is suspended until the recursive call completes.

Recursion is an important technique in the study of data structures and algorithms. We will use it prominently in several later chapters of this book (most notably, Chapters 8 and 12). In this chapter, we begin with the following four illustrative examples of the use of recursion, providing a Python implementation for each.

- The ***factorial function*** (commonly denoted as $n!$) is a classic mathematical function that has a natural recursive definition.
- An ***English ruler*** has a recursive pattern that is a simple example of a fractal structure.
- ***Binary search*** is among the most important computer algorithms. It allows us to efficiently locate a desired value in a data set with upwards of billions of entries.
- The ***file system*** for a computer has a recursive structure in which directories can be nested arbitrarily deeply within other directories. Recursive algorithms are widely used to explore and manage these file systems.

We then describe how to perform a formal analysis of the running time of a recursive algorithm and we discuss some potential pitfalls when defining recursions. In the balance of the chapter, we provide many more examples of recursive algorithms, organized to highlight some common forms of design.

4.1 Illustrative Examples

4.1.1 The Factorial Function

To demonstrate the mechanics of recursion, we begin with a simple mathematical example of computing the value of the ***factorial function***. The factorial of a positive integer n, denoted $n!$, is defined as the product of the integers from 1 to n. If $n = 0$, then $n!$ is defined as 1 by convention. More formally, for any integer $n \geq 0$,

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot (n-1) \cdot (n-2) \cdots 3 \cdot 2 \cdot 1 & \text{if } n \geq 1. \end{cases}$$

For example, $5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 120$. The factorial function is important because it is known to equal the number of ways in which n distinct items can be arranged into a sequence, that is, the number of ***permutations*** of n items. For example, the three characters `a`, `b`, and `c` can be arranged in $3! = 3 \cdot 2 \cdot 1 = 6$ ways: `abc`, `acb`, `bac`, `bca`, `cab`, and `cba`.

There is a natural recursive definition for the factorial function. To see this, observe that $5! = 5 \cdot (4 \cdot 3 \cdot 2 \cdot 1) = 5 \cdot 4!$. More generally, for a positive integer n, we can define $n!$ to be $n \cdot (n-1)!$. This ***recursive definition*** can be formalized as

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot (n-1)! & \text{if } n \geq 1. \end{cases}$$

This definition is typical of many recursive definitions. First, it contains one or more ***base cases***, which are defined nonrecursively in terms of fixed quantities. In this case, $n = 0$ is the base case. It also contains one or more ***recursive cases***, which are defined by appealing to the definition of the function being defined.

A Recursive Implementation of the Factorial Function

Recursion is not just a mathematical notation; we can use recursion to design a Python implementation of a factorial function, as shown in Code Fragment 4.1.

```
1 def factorial(n):
2   if n == 0:
3     return 1
4   else:
5     return n * factorial(n-1)
```

Code Fragment 4.1: A recursive implementation of the factorial function.

This function does not use any explicit loops. Repetition is provided by the repeated recursive invocations of the function. There is no circularity in this definition, because each time the function is invoked, its argument is smaller by one, and when a base case is reached, no further recursive calls are made.

We illustrate the execution of a recursive function using a ***recursion trace***. Each entry of the trace corresponds to a recursive call. Each new recursive function call is indicated by a downward arrow to a new invocation. When the function returns, an arrow showing this return is drawn and the return value may be indicated alongside this arrow. An example of such a trace for the factorial function is shown in Figure 4.1.

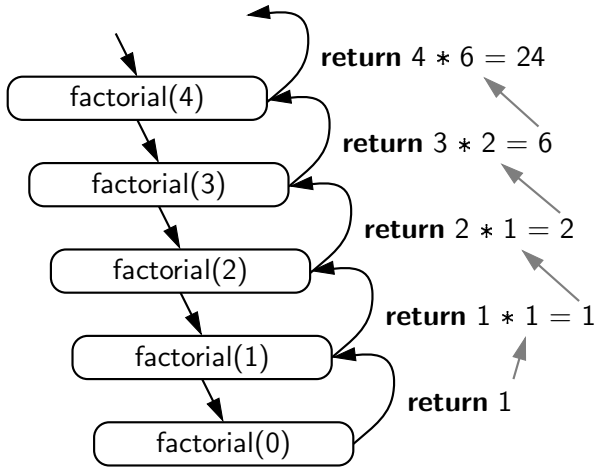


Figure 4.1: A recursion trace for the call factorial(5).

A recursion trace closely mirrors the programming language's execution of the recursion. In Python, each time a function (recursive or otherwise) is called, a structure known as an ***activation record*** or ***frame*** is created to store information about the progress of that invocation of the function. This activation record includes a namespace for storing the function call's parameters and local variables (see Section 1.10 for a discussion of namespaces), and information about which command in the body of the function is currently executing.

When the execution of a function leads to a nested function call, the execution of the former call is suspended and its activation record stores the place in the source code at which the flow of control should continue upon return of the nested call. This process is used both in the standard case of one function calling a different function, or in the recursive case in which a function invokes itself. The key point is that there is a different activation record for each active call.

4.1.2 Drawing an English Ruler

In the case of computing a factorial, there is no compelling reason for preferring recursion over a direct iteration with a loop. As a more complex example of the use of recursion, consider how to draw the markings of a typical English ruler. For each inch, we place a tick with a numeric label. We denote the length of the tick designating a whole inch as the ***major tick length***. Between the marks for whole inches, the ruler contains a series of ***minor ticks***, placed at intervals of $1/2$ inch, $1/4$ inch, and so on. As the size of the interval decreases by half, the tick length decreases by one. Figure 4.2 demonstrates several such rulers with varying major tick lengths (although not drawn to scale).

```
---- 0
-
--
-
---
-
--
-
---- 1
-
--
-
---
-
--
-
---- 2
(a)
```

```
----- 0
-
--
-
---
-
--
-
----
-
--
-
---
-
--
-
----- 1
(b)
```

```
--- 0
-
--
-
--- 1
-
--
-
--- 2
-
--
-
--- 3
(c)
```

Figure 4.2: Three sample outputs of an English ruler drawing: (a) a 2-inch ruler with major tick length 4; (b) a 1-inch ruler with major tick length 5; (c) a 3-inch ruler with major tick length 3.

A Recursive Approach to Ruler Drawing

The English ruler pattern is a simple example of a ***fractal***, that is, a shape that has a self-recursive structure at various levels of magnification. Consider the rule with major tick length 5 shown in Figure 4.2(b). Ignoring the lines containing 0 and 1, let us consider how to draw the sequence of ticks lying between these lines. The central tick (at 1/2 inch) has length 4. Observe that the two patterns of ticks above and below this central tick are identical, and each has a central tick of length 3.

In general, an interval with a central tick length $L \geq 1$ is composed of:

- An interval with a central tick length $L-1$
- A single tick of length L
- An interval with a central tick length $L-1$

Although it is possible to draw such a ruler using an iterative process (see Exercise P-4.25), the task is considerably easier to accomplish with recursion. Our implementation consists of three functions, as shown in Code Fragment 4.2. The main function, draw_ruler, manages the construction of the entire ruler. Its arguments specify the total number of inches in the ruler and the major tick length. The utility function, draw_line, draws a single tick with a specified number of dashes (and an optional string label, that is printed after the tick).

The interesting work is done by the recursive draw_interval function. This function draws the sequence of minor ticks within some interval, based upon the length of the interval's central tick. We rely on the intuition shown at the top of this page, and with a base case when $L = 0$ that draws nothing. For $L \geq 1$, the first and last steps are performed by recursively calling draw_interval$(L-1)$. The middle step is performed by calling the function draw_line(L).

```
1  def draw_line(tick_length, tick_label=''):
2    """Draw one line with given tick length (followed by optional label)."""
3    line = '-' * tick_length
4    if tick_label:
5      line += ' ' + tick_label
6    print(line)
7
8  def draw_interval(center_length):
9    """Draw tick interval based upon a central tick length."""
10   if center_length > 0:                    # stop when length drops to 0
11     draw_interval(center_length - 1)       # recursively draw top ticks
12     draw_line(center_length)               # draw center tick
13     draw_interval(center_length - 1)       # recursively draw bottom ticks
14
15 def draw_ruler(num_inches, major_length):
16   """Draw English ruler with given number of inches, major tick length."""
17   draw_line(major_length, '0')             # draw inch 0 line
18   for j in range(1, 1 + num_inches):
19     draw_interval(major_length - 1)        # draw interior ticks for inch
20     draw_line(major_length, str(j))        # draw inch j line and label
```

Code Fragment 4.2: A recursive implementation of a function that draws a ruler.

Illustrating Ruler Drawing Using a Recursion Trace

The execution of the recursive draw_interval function can be visualized using a recursion trace. The trace for draw_interval is more complicated than in the factorial example, however, because each instance makes two recursive calls. To illustrate this, we will show the recursion trace in a form that is reminiscent of an outline for a document. See Figure 4.3.

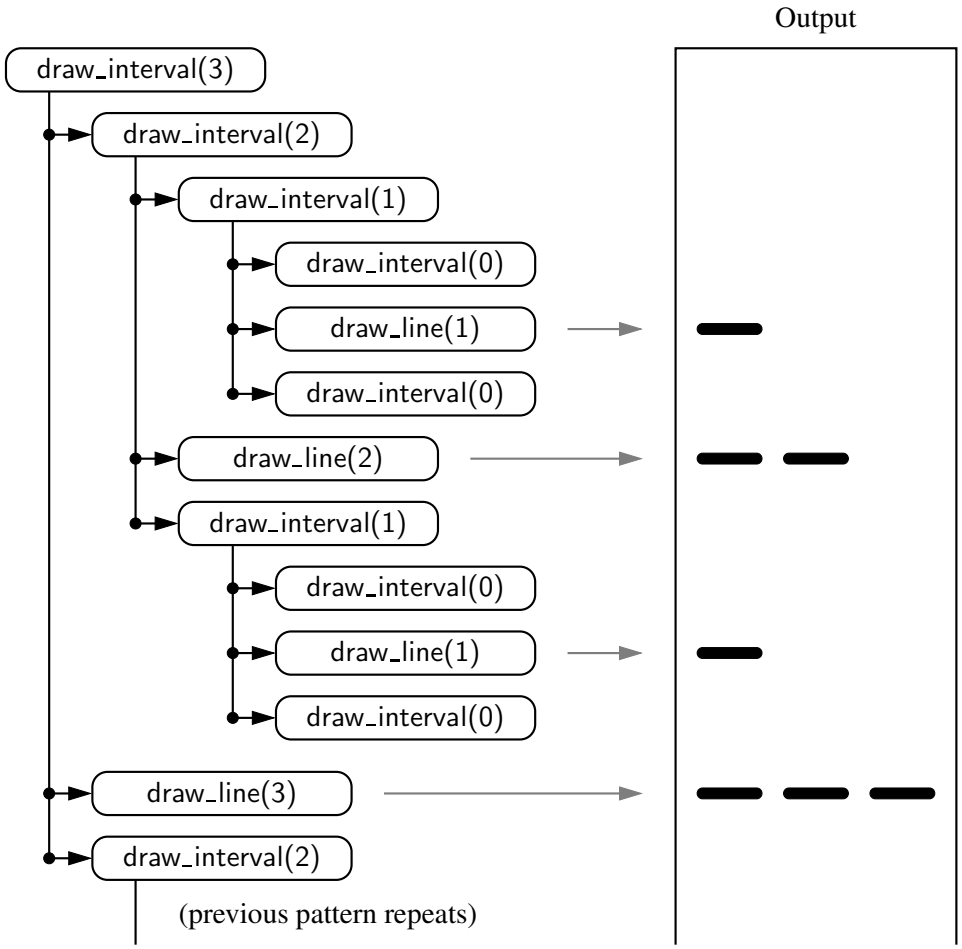


Figure 4.3: A partial recursion trace for the call draw_interval(3). The second pattern of calls for draw_interval(2) is not shown, but it is identical to the first.

4.1.3 Binary Search

In this section, we describe a classic recursive algorithm, ***binary search***, that is used to efficiently locate a target value within a sorted sequence of n elements. This is among the most important of computer algorithms, and it is the reason that we so often store data in sorted order (as in Figure 4.4).

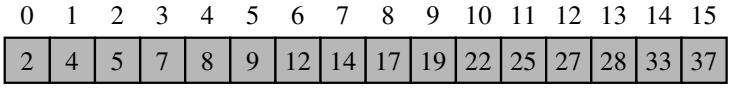


Figure 4.4: Values stored in sorted order within an indexable sequence, such as a Python list. The numbers at top are the indices.

When the sequence is ***unsorted***, the standard approach to search for a target value is to use a loop to examine every element, until either finding the target or exhausting the data set. This is known as the ***sequential search*** algorithm. This algorithm runs in $O(n)$ time (i.e., linear time) since every element is inspected in the worst case.

When the sequence is ***sorted*** and ***indexable***, there is a much more efficient algorithm. (For intuition, think about how you would accomplish this task by hand!) For any index j, we know that all the values stored at indices $0,\ldots,j-1$ are less than or equal to the value at index j, and all the values stored at indices $j+1,\ldots,n-1$ are greater than or equal to that at index j. This observation allows us to quickly "home in" on a search target using a variant of the children's game "high-low." We call an element of the sequence a ***candidate*** if, at the current stage of the search, we cannot rule out that this item matches the target. The algorithm maintains two parameters, low and high, such that all the candidate entries have index at least low and at most high. Initially, $\mathsf{low} = 0$ and $\mathsf{high} = n-1$. We then compare the target value to the median candidate, that is, the item data[mid] with index

$$\mathsf{mid} = \lfloor (\mathsf{low} + \mathsf{high})/2 \rfloor .$$

We consider three cases:

- If the target equals data[mid], then we have found the item we are looking for, and the search terminates successfully.
- If target $<$ data[mid], then we recur on the first half of the sequence, that is, on the interval of indices from low to $\mathsf{mid}-1$.
- If target $>$ data[mid], then we recur on the second half of the sequence, that is, on the interval of indices from $\mathsf{mid}+1$ to high.

An unsuccessful search occurs if low $>$ high, as the interval $[\mathsf{low}, \mathsf{high}]$ is empty.

This algorithm is known as ***binary search***. We give a Python implementation in Code Fragment 4.3, and an illustration of the execution of the algorithm in Figure 4.5. Whereas sequential search runs in $O(n)$ time, the more efficient binary search runs in $O(\log n)$ time. This is a significant improvement, given that if n is one billion, $\log n$ is only 30. (We defer our formal analysis of binary search's running time to Proposition 4.2 in Section 4.2.)

```
1  def binary_search(data, target, low, high):
2    """Return True if target is found in indicated portion of a Python list.
3
4    The search only considers the portion from data[low] to data[high] inclusive.
5    """
6    if low > high:
7      return False                          # interval is empty; no match
8    else:
9      mid = (low + high) // 2
10     if target == data[mid]:               # found a match
11       return True
12     elif target < data[mid]:
13       # recur on the portion left of the middle
14       return binary_search(data, target, low, mid - 1)
15     else:
16       # recur on the portion right of the middle
17       return binary_search(data, target, mid + 1, high)
```

Code Fragment 4.3: An implementation of the binary search algorithm.

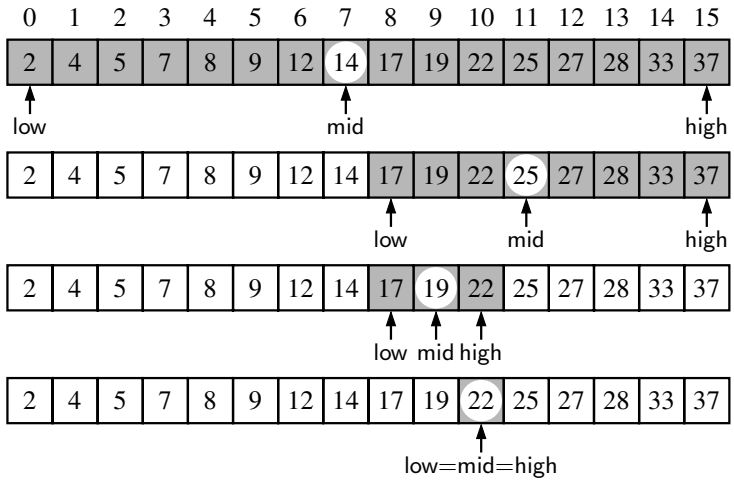


Figure 4.5: Example of a binary search for target value 22.

4.1.4 File Systems

Modern operating systems define file-system directories (which are also sometimes called "folders") in a recursive way. Namely, a file system consists of a top-level directory, and the contents of this directory consists of files and other directories, which in turn can contain files and other directories, and so on. The operating system allows directories to be nested arbitrarily deep (as long as there is enough space in memory), although there must necessarily be some base directories that contain only files, not further subdirectories. A representation of a portion of such a file system is given in Figure 4.6.

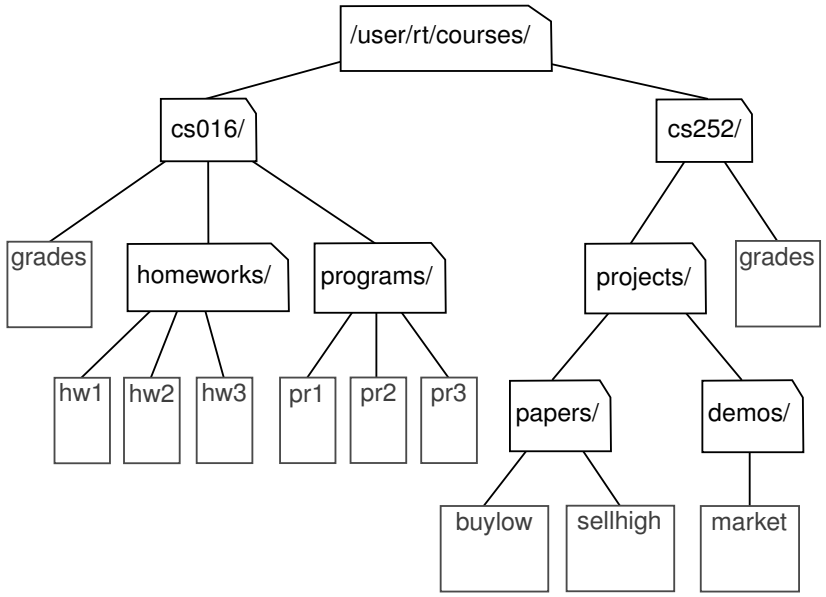


Figure 4.6: A portion of a file system demonstrating a nested organization.

Given the recursive nature of the file-system representation, it should not come as a surprise that many common behaviors of an operating system, such as copying a directory or deleting a directory, are implemented with recursive algorithms. In this section, we consider one such algorithm: computing the total disk usage for all files and directories nested within a particular directory.

For illustration, Figure 4.7 portrays the disk space being used by all entries in our sample file system. We differentiate between the *immediate* disk space used by each entry and the *cumulative* disk space used by that entry and all nested features. For example, the `cs016` directory uses only 2K of immediate space, but a total of 249K of cumulative space.

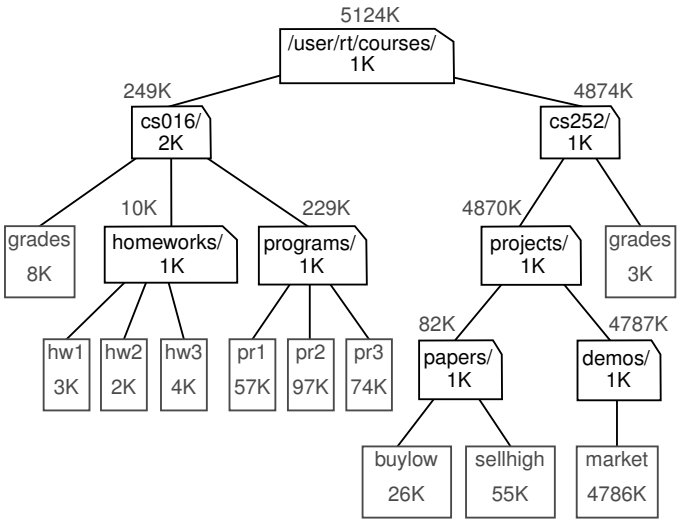


Figure 4.7: The same portion of a file system given in Figure 4.6, but with additional annotations to describe the amount of disk space that is used. Within the icon for each file or directory is the amount of space directly used by that artifact. Above the icon for each directory is an indication of the *cumulative* disk space used by that directory and all its (recursive) contents.

The cumulative disk space for an entry can be computed with a simple recursive algorithm. It is equal to the immediate disk space used by the entry plus the sum of the *cumulative* disk space usage of any entries that are stored directly within the entry. For example, the cumulative disk space for `cs016` is 249K because it uses 2K itself, 8K cumulatively in `grades`, 10K cumulatively in `homeworks`, and 229K cumulatively in `programs`. Pseudo-code for this algorithm is given in Code Fragment 4.4.

Algorithm DiskUsage(path):
Input: A string designating a path to a file-system entry
Output: The cumulative disk space used by that entry and any nested entries

```
total = size(path)                          {immediate disk space used by the entry}
if path represents a directory then
  for each child entry stored within directory path do
    total = total + DiskUsage(child)        {recursive call}
return total
```

Code Fragment 4.4: An algorithm for computing the cumulative disk space usage nested at a file-system entry. Function size returns the immediate disk space of an entry.

Python's os Module

To provide a Python implementation of a recursive algorithm for computing disk usage, we rely on Python's os module, which provides robust tools for interacting with the operating system during the execution of a program. This is an extensive library, but we will only need the following four functions:

- **os.path.getsize(path)**
  Return the immediate disk usage (measured in bytes) for the file or directory that is identified by the string path (e.g., `/user/rt/courses`).
- **os.path.isdir(path)**
  Return True if entry designated by string path is a directory; False otherwise.
- **os.listdir(path)**
  Return a list of strings that are the names of all entries within a directory designated by string path. In our sample file system, if the parameter is `/user/rt/courses`, this returns the list `['cs016', 'cs252']`.
- **os.path.join(path, filename)**
  Compose the path string and filename string using an appropriate operating system separator between the two (e.g., the / character for a Unix/Linux system, and the \ character for Windows). Return the string that represents the full path to the file.

Python Implementation

With use of the os module, we now convert the algorithm from Code Fragment 4.4 into the Python implementation of Code Fragment 4.5.

```
1  import os
2
3  def disk_usage(path):
4    """Return the number of bytes used by a file/folder and any descendents."""
5    total = os.path.getsize(path)                # account for direct usage
6    if os.path.isdir(path):                      # if this is a directory,
7      for filename in os.listdir(path):          # then for each child:
8        childpath = os.path.join(path, filename) # compose full path to child
9        total += disk_usage(childpath)           # add child's usage to total
10
11   print ('{0:<7}'.format(total), path)         # descriptive output (optional)
12   return total                                 # return the grand total
```

Code Fragment 4.5: A recursive function for reporting disk usage of a file system.

Recursion Trace

To produce a different form of a recursion trace, we have included an extraneous print statement within our Python implementation (line 11 of Code Fragment 4.5). The precise format of that output intentionally mirrors output that is produced by a classic Unix/Linux utility named du (for "disk usage"). It reports the amount of disk space used by a directory and all contents nested within, and can produce a verbose report, as given in Figure 4.8.

Our implementation of the disk_usage function produces an identical result, when executed on the sample file system portrayed in Figure 4.7. During the execution of the algorithm, exactly one recursive call is made for each entry in the portion of the file system that is considered. Because the print statement is made just before returning from a recursive call, the output shown in Figure 4.8 reflects the order in which the recursive calls are completed. In particular, we begin and end a recursive call for each entry that is nested below another entry, computing the nested cumulative disk space before we can compute and report the cumulative disk space for the containing entry. For example, we do not know the cumulative total for entry /user/rt/courses/cs016 until after the recursive calls regarding contained entries grades, homeworks, and programs complete.

```
8       /user/rt/courses/cs016/grades
3       /user/rt/courses/cs016/homeworks/hw1
2       /user/rt/courses/cs016/homeworks/hw2
4       /user/rt/courses/cs016/homeworks/hw3
10      /user/rt/courses/cs016/homeworks
57      /user/rt/courses/cs016/programs/pr1
97      /user/rt/courses/cs016/programs/pr2
74      /user/rt/courses/cs016/programs/pr3
229     /user/rt/courses/cs016/programs
249     /user/rt/courses/cs016
26      /user/rt/courses/cs252/projects/papers/buylow
55      /user/rt/courses/cs252/projects/papers/sellhigh
82      /user/rt/courses/cs252/projects/papers
4786    /user/rt/courses/cs252/projects/demos/market
4787    /user/rt/courses/cs252/projects/demos
4870    /user/rt/courses/cs252/projects
3       /user/rt/courses/cs252/grades
4874    /user/rt/courses/cs252
5124    /user/rt/courses/
```

Figure 4.8: A report of the disk usage for the file system shown in Figure 4.7, as generated by the Unix/Linux utility du (with command-line options -ak), or equivalently by our disk_usage function from Code Fragment 4.5.

4.2 Analyzing Recursive Algorithms

In Chapter 3, we introduced mathematical techniques for analyzing the efficiency of an algorithm, based upon an estimate of the number of primitive operations that are executed by the algorithm. We use notations such as big-Oh to summarize the relationship between the number of operations and the input size for a problem. In this section, we demonstrate how to perform this type of running-time analysis to recursive algorithms.

With a recursive algorithm, we will account for each operation that is performed based upon the particular ***activation*** of the function that manages the flow of control at the time it is executed. Stated another way, for each invocation of the function, we only account for the number of operations that are performed within the body of that activation. We can then account for the overall number of operations that are executed as part of the recursive algorithm by taking the sum, over all activations, of the number of operations that take place during each individual activation. (As an aside, this is also the way we analyze a nonrecursive function that calls other functions from within its body.)

To demonstrate this style of analysis, we revisit the four recursive algorithms presented in Sections 4.1.1 through 4.1.4: factorial computation, drawing an English ruler, binary search, and computation of the cumulative size of a file system. In general, we may rely on the intuition afforded by a ***recursion trace*** in recognizing how many recursive activations occur, and how the parameterization of each activation can be used to estimate the number of primitive operations that occur within the body of that activation. However, each of these recursive algorithms has a unique structure and form.

Computing Factorials

It is relatively easy to analyze the efficiency of our function for computing factorials, as described in Section 4.1.1. A sample recursion trace for our factorial function was given in Figure 4.1. To compute factorial(n), we see that there are a total of $n+1$ activations, as the parameter decreases from n in the first call, to $n-1$ in the second call, and so on, until reaching the base case with parameter 0.

It is also clear, given an examination of the function body in Code Fragment 4.1, that each individual activation of factorial executes a constant number of operations. Therefore, we conclude that the overall number of operations for computing factorial(n) is $O(n)$, as there are $n+1$ activations, each of which accounts for $O(1)$ operations.

Drawing an English Ruler

In analyzing the English ruler application from Section 4.1.2, we consider the fundamental question of how many total lines of output are generated by an initial call to draw_interval(c), where c denotes the center length. This is a reasonable benchmark for the overall efficiency of the algorithm as each line of output is based upon a call to the draw_line utility, and each recursive call to draw_interval with nonzero parameter makes exactly one direct call to draw_line.

Some intuition may be gained by examining the source code and the recursion trace. We know that a call to draw_interval(c) for $c > 0$ spawns two calls to draw_interval(c−1) and a single call to draw_line. We will rely on this intuition to prove the following claim.

Proposition 4.1: *For $c \geq 0$, a call to draw_interval(c) results in precisely $2^c - 1$ lines of output.*

Justification: We provide a formal proof of this claim by ***induction*** (see Section 3.4.3). In fact, induction is a natural mathematical technique for proving the correctness and efficiency of a recursive process. In the case of the ruler, we note that an application of draw_interval(0) generates no output, and that $2^0 - 1 = 1 - 1 = 0$. This serves as a base case for our claim.

More generally, the number of lines printed by draw_interval(c) is one more than twice the number generated by a call to draw_interval(c−1), as one center line is printed between two such recursive calls. By induction, we have that the number of lines is thus $1 + 2 \cdot (2^{c-1} - 1) = 1 + 2^c - 2 = 2^c - 1$. ■

This proof is indicative of a more mathematically rigorous tool, known as a ***recurrence equation*** that can be used to analyze the running time of a recursive algorithm. That technique is discussed in Section 12.2.4, in the context of recursive sorting algorithms.

Performing a Binary Search

Considering the running time of the binary search algorithm, as presented in Section 4.1.3, we observe that a constant number of primitive operations are executed at each recursive call of method of a binary search. Hence, the running time is proportional to the number of recursive calls performed. We will show that at most $\lfloor \log n \rfloor + 1$ recursive calls are made during a binary search of a sequence having n elements, leading to the following claim.

Proposition 4.2: *The binary search algorithm runs in $O(\log n)$ time for a sorted sequence with n elements.*

Justification: To prove this claim, a crucial fact is that with each recursive call the number of candidate entries still to be searched is given by the value

$$\mathsf{high} - \mathsf{low} + 1.$$

Moreover, the number of remaining candidates is reduced by at least one half with each recursive call. Specifically, from the definition of mid, the number of remaining candidates is either

$$(\mathsf{mid} - 1) - \mathsf{low} + 1 = \left\lfloor \frac{\mathsf{low} + \mathsf{high}}{2} \right\rfloor - \mathsf{low} \leq \frac{\mathsf{high} - \mathsf{low} + 1}{2}$$

or

$$\mathsf{high} - (\mathsf{mid} + 1) + 1 = \mathsf{high} - \left\lfloor \frac{\mathsf{low} + \mathsf{high}}{2} \right\rfloor \leq \frac{\mathsf{high} - \mathsf{low} + 1}{2}.$$

Initially, the number of candidates is n; after the first call in a binary search, it is at most $n/2$; after the second call, it is at most $n/4$; and so on. In general, after the jth call in a binary search, the number of candidate entries remaining is at most $n/2^j$. In the worst case (an unsuccessful search), the recursive calls stop when there are no more candidate entries. Hence, the maximum number of recursive calls performed, is the smallest integer r such that

$$\frac{n}{2^r} < 1.$$

In other words (recalling that we omit a logarithm's base when it is 2), $r > \log n$. Thus, we have

$$r = \lfloor \log n \rfloor + 1,$$

which implies that binary search runs in $O(\log n)$ time. ■

Computing Disk Space Usage

Our final recursive algorithm from Section 4.1 was that for computing the overall disk space usage in a specified portion of a file system. To characterize the "problem size" for our analysis, we let n denote the number of file-system entries in the portion of the file system that is considered. (For example, the file system portrayed in Figure 4.6 has $n = 19$ entries.)

To characterize the cumulative time spent for an initial call to the disk_usage function, we must analyze the total number of recursive invocations that are made, as well as the number of operations that are executed within those invocations.

We begin by showing that there are precisely n recursive invocations of the function, in particular, one for each entry in the relevant portion of the file system. Intuitively, this is because a call to disk_usage for a particular entry e of the file system is only made from within the for loop of Code Fragment 4.5 when processing the entry for the unique directory that contains e, and that entry will only be explored once.

To formalize this argument, we can define the ***nesting level*** of each entry such that the entry on which we begin has nesting level 0, entries stored directly within it have nesting level 1, entries stored within those entries have nesting level 2, and so on. We can prove by induction that there is exactly one recursive invocation of disk_usage upon each entry at nesting level k. As a base case, when $k = 0$, the only recursive invocation made is the initial one. As the inductive step, once we know there is exactly one recursive invocation for each entry at nesting level k, we can claim that there is exactly one invocation for each entry e at nesting level k, made within the for loop for the entry at level k that contains e.

Having established that there is one recursive call for each entry of the file system, we return to the question of the overall computation time for the algorithm. It would be great if we could argue that we spend $O(1)$ time in any single invocation of the function, but that is not the case. While there are a constant number of steps reflect in the call to os.path.getsize to compute the disk usage directly at that entry, when the entry is a directory, the body of the disk_usage function includes a for loop that iterates over all entries that are contained within that directory. In the worst case, it is possible that one entry includes $n - 1$ others.

Based on this reasoning, we could conclude that there are $O(n)$ recursive calls, each of which runs in $O(n)$ time, leading to an overall running time that is $O(n^2)$. While this upper bound is technically true, it is not a tight upper bound. Remarkably, we can prove the stronger bound that the recursive algorithm for disk_usage completes in $O(n)$ time! The weaker bound was pessimistic because it assumed a worst-case number of entries for each directory. While it is possible that some directories contain a number of entries proportional to n, they cannot all contain that many. To prove the stronger claim, we choose to consider the *overall* number of iterations of the for loop across all recursive calls. We claim there are precisely $n - 1$ such iteration of that loop overall. We base this claim on the fact that each iteration of that loop makes a recursive call to disk_usage, and yet we have already concluded that there are a total of n calls to disk_usage (including the original call). We therefore conclude that there are $O(n)$ recursive calls, each of which uses $O(1)$ time outside the loop, and that the *overall* number of operations due to the loop is $O(n)$. Summing all of these bounds, the overall number of operations is $O(n)$.

The argument we have made is more advanced than with the earlier examples of recursion. The idea that we can sometimes get a tighter bound on a series of operations by considering the cumulative effect, rather than assuming that each achieves a worst case is a technique called ***amortization***; we will see a further example of such analysis in Section 5.3. Furthermore, a file system is an implicit example of a data structure known as a ***tree***, and our disk usage algorithm is really a manifestation of a more general algorithm known as a ***tree traversal***. Trees will be the focus of Chapter 8, and our argument about the $O(n)$ running time of the disk usage algorithm will be generalized for tree traversals in Section 8.4.

4.3 Recursion Run Amok

Although recursion is a very powerful tool, it can easily be misused in various ways. In this section, we examine several problems in which a poorly implemented recursion causes drastic inefficiency, and we discuss some strategies for recognizing and avoid such pitfalls.

We begin by revisiting the ***element uniqueness problem***, defined on page 135 of Section 3.3.3. We can use the following recursive formulation to determine if all n elements of a sequence are unique. As a base case, when $n = 1$, the elements are trivially unique. For $n \geq 2$, the elements are unique if and only if the first $n-1$ elements are unique, the last $n-1$ items are unique, and the first and last elements are different (as that is the only pair that was not already checked as a subcase). A recursive implementation based on this idea is given in Code Fragment 4.6, named unique3 (to differentiate it from unique1 and unique2 from Chapter 3).

```
1  def unique3(S, start, stop):
2    """Return True if there are no duplicate elements in slice S[start:stop]."""
3    if stop - start <= 1: return True                 # at most one item
4    elif not unique(S, start, stop-1): return False   # first part has duplicate
5    elif not unique(S, start+1, stop): return False   # second part has duplicate
6    else: return S[start] != S[stop-1]                # do first and last differ?
```

Code Fragment 4.6: Recursive unique3 for testing element uniqueness.

Unfortunately, this is a terribly inefficient use of recursion. The nonrecursive part of each call uses $O(1)$ time, so the overall running time will be proportional to the total number of recursive invocations. To analyze the problem, we let n denote the number of entries under consideration, that is, let n= stop − start.

If $n = 1$, then the running time of unique3 is $O(1)$, since there are no recursive calls for this case. In the general case, the important observation is that a single call to unique3 for a problem of size n may result in two recursive calls on problems of size $n-1$. Those two calls with size $n-1$ could in turn result in four calls (two each) with a range of size $n-2$, and thus eight calls with size $n-3$ and so on. Thus, in the worst case, the total number of function calls is given by the geometric summation

$$1 + 2 + 4 + \cdots + 2^{n-1},$$

which is equal to $2^n - 1$ by Proposition 3.5. Thus, the running time of function unique3 is $O(2^n)$. This is an incredibly inefficient function for solving the element uniqueness problem. Its inefficiency comes not from the fact that it uses recursion—it comes from the fact that it uses recursion poorly, which is something we address in Exercise C-4.11.

An Inefficient Recursion for Computing Fibonacci Numbers

In Section 1.8, we introduced a process for generating the Fibonacci numbers, which can be defined recursively as follows:

$$\begin{aligned} F_0 &= 0 \\ F_1 &= 1 \\ F_n &= F_{n-2} + F_{n-1} \quad \text{for } n > 1. \end{aligned}$$

Ironically, a direct implementation based on this definition results in the function bad_fibonacci shown in Code Fragment 4.7, which computes the sequence of Fibonacci numbers by making two recursive calls in each non-base case.

```
1  def bad_fibonacci(n):
2    """Return the nth Fibonacci number."""
3    if n <= 1:
4      return n
5    else:
6      return bad_fibonacci(n-2) + bad_fibonacci(n-1)
```

Code Fragment 4.7: Computing the nth Fibonacci number using binary recursion.

Unfortunately, such a direct implementation of the Fibonacci formula results in a terribly inefficient function. Computing the nth Fibonacci number in this way requires an exponential number of calls to the function. Specifically, let c_n denote the number of calls performed in the execution of bad_fibonacci(n). Then, we have the following values for the c_n's:

$$\begin{aligned} c_0 &= 1 \\ c_1 &= 1 \\ c_2 &= 1 + c_0 + c_1 = 1 + 1 + 1 = 3 \\ c_3 &= 1 + c_1 + c_2 = 1 + 1 + 3 = 5 \\ c_4 &= 1 + c_2 + c_3 = 1 + 3 + 5 = 9 \\ c_5 &= 1 + c_3 + c_4 = 1 + 5 + 9 = 15 \\ c_6 &= 1 + c_4 + c_5 = 1 + 9 + 15 = 25 \\ c_7 &= 1 + c_5 + c_6 = 1 + 15 + 25 = 41 \\ c_8 &= 1 + c_6 + c_7 = 1 + 25 + 41 = 67 \end{aligned}$$

If we follow the pattern forward, we see that the number of calls more than doubles for each two consecutive indices. That is, c_4 is more than twice c_2, c_5 is more than twice c_3, c_6 is more than twice c_4, and so on. Thus, $c_n > 2^{n/2}$, which means that bad_fibonacci(n) makes a number of calls that is exponential in n.

An Efficient Recursion for Computing Fibonacci Numbers

We were tempted into using the bad recursion formulation because of the way the nth Fibonacci number, F_n, depends on the two previous values, F_{n-2} and F_{n-1}. But notice that after computing F_{n-2}, the call to compute F_{n-1} requires its own recursive call to compute F_{n-2}, as it does not have knowledge of the value of F_{n-2} that was computed at the earlier level of recursion. That is duplicative work. Worse yet, both of those calls will need to (re)compute the value of F_{n-3}, as will the computation of F_{n-1}. This snowballing effect is what leads to the exponential running time of bad_recursion.

We can compute F_n much more efficiently using a recursion in which each invocation makes only one recursive call. To do so, we need to redefine the expectations of the function. Rather than having the function return a single value, which is the nth Fibonacci number, we define a recursive function that returns a pair of consecutive Fibonacci numbers (F_n, F_{n-1}), using the convention $F_{-1} = 0$. Although it seems to be a greater burden to report two consecutive Fibonacci numbers instead of one, passing this extra information from one level of the recursion to the next makes it much easier to continue the process. (It allows us to avoid having to recompute the second value that was already known within the recursion.) An implementation based on this strategy is given in Code Fragment 4.8.

```
1  def good_fibonacci(n):
2    """Return pair of Fibonacci numbers, F(n) and F(n-1)."""
3    if n <= 1:
4      return (n,0)
5    else:
6      (a, b) = good_fibonacci(n-1)
7      return (a+b, a)
```

Code Fragment 4.8: Computing the nth Fibonacci number using linear recursion.

In terms of efficiency, the difference between the bad recursion and the good recursion for this problem is like night and day. The bad_fibonacci function uses exponential time. We claim that the execution of function good_fibonacci(n) takes $O(n)$ time. Each recursive call to good_fibonacci decreases the argument n by 1; therefore, a recursion trace includes a series of n function calls. Because the nonrecursive work for each call uses constant time, the overall computation executes in $O(n)$ time.

4.3.1 Maximum Recursive Depth in Python

Another danger in the misuse of recursion is known as ***infinite recursion***. If each recursive call makes another recursive call, without ever reaching a base case, then we have an infinite series of such calls. This is a fatal error. An infinite recursion can quickly swamp computing resources, not only due to rapid use of the CPU, but because each successive call creates an activation record requiring additional memory. A blatant example of an ill-formed recursion is the following:

```
def fib(n):
  return fib(n)                    # fib(n) equals fib(n)
```

However, there are far more subtle errors that can lead to an infinite recursion. Revisiting our implementation of binary search in Code Fragment 4.3, in the final case (line 17) we make a recursive call on the right portion of the sequence, in particular going from index mid+1 to high. Had that line instead been written as

```
    return binary_search(data, target, mid, high)      # note the use of mid
```

this could result in an infinite recursion. In particular, when searching a range of two elements, it becomes possible to make a recursive call on the identical range.

A programmer should ensure that each recursive call is in some way progressing toward a base case (for example, by having a parameter value that decreases with each call). However, to combat against infinite recursions, the designers of Python made an intentional decision to limit the overall number of function activations that can be simultaneously active. The precise value of this limit depends upon the Python distribution, but a typical default value is 1000. If this limit is reached, the Python interpreter raises a RuntimeError with a message, `maximum recursion depth exceeded`.

For many legitimate applications of recursion, a limit of 1000 nested function calls suffices. For example, our binary_search function (Section 4.1.3) has $O(\log n)$ recursive depth, and so for the default recursive limit to be reached, there would need to be 2^{1000} elements (far, far more than the estimated number of atoms in the universe). However, in the next section we discuss several algorithms that have recursive depth proportional to n. Python's artificial limit on the recursive depth could disrupt such otherwise legitimate computations.

Fortunately, the Python interpreter can be dynamically reconfigured to change the default recursive limit. This is done through use of a module named sys, which supports a getrecursionlimit function and a setrecursionlimit. Sample usage of those functions is demonstrated as follows:

```
import sys
old = sys.getrecursionlimit( )       # perhaps 1000 is typical
sys.setrecursionlimit(1000000)       # change to allow 1 million nested calls
```

4.4 Further Examples of Recursion

In the remainder of this chapter, we provide additional examples of the use of recursion. We organize our presentation by considering the maximum number of recursive calls that may be started from within the body of a single activation.

- If a recursive call starts at most one other, we call this a ***linear recursion***.
- If a recursive call may start two others, we call this a ***binary recursion***.
- If a recursive call may start three or more others, this is ***multiple recursion***.

4.4.1 Linear Recursion

If a recursive function is designed so that each invocation of the body makes at most one new recursive call, this is know as ***linear recursion***. Of the recursions we have seen so far, the implementation of the factorial function (Section 4.1.1) and the good_fibonacci function (Section 4.3) are clear examples of linear recursion. More interestingly, the binary search algorithm (Section 4.1.3) is also an example of *linear recursion*, despite the "binary" terminology in the name. The code for binary search (Code Fragment 4.3) includes a case analysis with two branches that lead to recursive calls, but only one of those calls can be reached during a particular execution of the body.

A consequence of the definition of linear recursion is that any recursion trace will appear as a single sequence of calls, as we originally portrayed for the factorial function in Figure 4.1 of Section 4.1.1. Note that the *linear recursion* terminology reflects the structure of the recursion trace, not the asymptotic analysis of the running time; for example, we have seen that binary search runs in $O(\log n)$ time.

Summing the Elements of a Sequence Recursively

Linear recursion can be a useful tool for processing a data sequence, such as a Python list. Suppose, for example, that we want to compute the sum of a sequence, S, of n integers. We can solve this summation problem using linear recursion by observing that the sum of all n integers in S is trivially 0, if $n = 0$, and otherwise that it is the sum of the first $n-1$ integers in S plus the last element in S. (See Figure 4.9.)

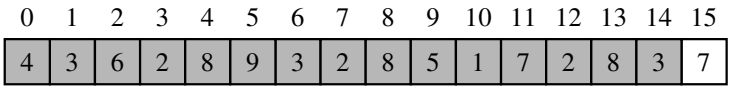


Figure 4.9: Computing the sum of a sequence recursively, by adding the last number to the sum of the first $n-1$.

A recursive algorithm for computing the sum of a sequence of numbers based on this intuition is implemented in Code Fragment 4.9.

```
1  def linear_sum(S, n):
2    """Return the sum of the first n numbers of sequence S."""
3    if n == 0:
4      return 0
5    else:
6      return linear_sum(S, n−1) + S[n−1]
```

Code Fragment 4.9: Summing the elements of a sequence using linear recursion.

A recursion trace of the linear_sum function for a small example is given in Figure 4.10. For an input of size n, the linear_sum algorithm makes $n+1$ function calls. Hence, it will take $O(n)$ time, because it spends a constant amount of time performing the nonrecursive part of each call. Moreover, we can also see that the memory space used by the algorithm (in addition to the sequence S) is also $O(n)$, as we use a constant amount of memory space for each of the $n+1$ activation records in the trace at the time we make the final recursive call (with $n = 0$).

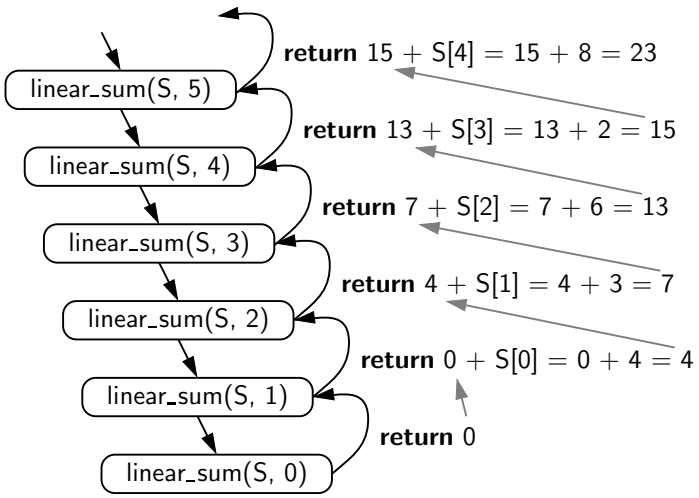


Figure 4.10: Recursion trace for an execution of linear_sum(S, 5) with input parameter S = [4, 3, 6, 2, 8].

Reversing a Sequence with Recursion

Next, let us consider the problem of reversing the n elements of a sequence, S, so that the first element becomes the last, the second element becomes second to the last, and so on. We can solve this problem using linear recursion, by observing that the reversal of a sequence can be achieved by swapping the first and last elements and then recursively reversing the remaining elements. We present an implementation of this algorithm in Code Fragment 4.10, using the convention that the first time we call this algorithm we do so as reverse(S, 0, len(S)).

```
1  def reverse(S, start, stop):
2    """Reverse elements in implicit slice S[start:stop]."""
3    if start < stop - 1:                    # if at least 2 elements:
4      S[start], S[stop-1] = S[stop-1], S[start]    # swap first and last
5      reverse(S, start+1, stop-1)                  # recur on rest
```

Code Fragment 4.10: Reversing the elements of a sequence using linear recursion.

Note that there are two implicit base case scenarios: When start == stop, the implicit range is empty, and when start == stop−1, the implicit range has only one element. In either of these cases, there is no need for action, as a sequence with zero elements or one element is trivially equal to its reversal. When otherwise invoking recursion, we are guaranteed to make progress towards a base case, as the difference, stop−start, decreases by two with each call (see Figure 4.11). If n is even, we will eventually reach the start == stop case, and if n is odd, we will eventually reach the start == stop − 1 case.

The above argument implies that the recursive algorithm of Code Fragment 4.10 is guaranteed to terminate after a total of $1 + \lfloor \frac{n}{2} \rfloor$ recursive calls. Since each call involves a constant amount of work, the entire process runs in $O(n)$ time.

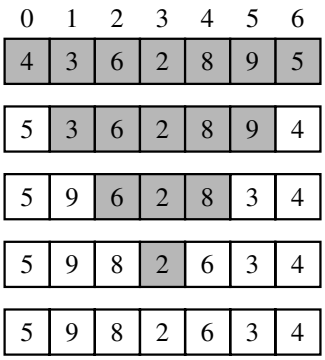


Figure 4.11: A trace of the recursion for reversing a sequence. The shaded portion has yet to be reversed.

Recursive Algorithms for Computing Powers

As another interesting example of the use of linear recursion, we consider the problem of raising a number x to an arbitrary nonnegative integer, n. That is, we wish to compute the ***power function***, defined as $power(x,n) = x^n$. (We use the name "power" for this discussion, to differentiate from the built-in function pow that provides such functionality.) We will consider two different recursive formulations for the problem that lead to algorithms with very different performance.

A trivial recursive definition follows from the fact that $x^n = x \cdot x^{n-1}$ for $n > 0$.

$$power(x,n) = \begin{cases} 1 & \text{if } n = 0 \\ x \cdot power(x, n-1) & \text{otherwise.} \end{cases}$$

This definition leads to a recursive algorithm shown in Code Fragment 4.11.

```
1  def power(x, n):
2    """Compute the value x**n for integer n."""
3    if n == 0:
4      return 1
5    else:
6      return x * power(x, n-1)
```

Code Fragment 4.11: Computing the power function using trivial recursion.

A recursive call to this version of $power(x,n)$ runs in $O(n)$ time. Its recursion trace has structure very similar to that of the factorial function from Figure 4.1, with the parameter decreasing by one with each call, and constant work performed at each of $n+1$ levels.

However, there is a much faster way to compute the power function using an alternative definition that employs a squaring technique. Let $k = \lfloor \frac{n}{2} \rfloor$ denote the floor of the division (expressed as n // 2 in Python). We consider the expression $\left(x^k\right)^2$. When n is even, $\lfloor \frac{n}{2} \rfloor = \frac{n}{2}$ and therefore $\left(x^k\right)^2 = \left(x^{\frac{n}{2}}\right)^2 = x^n$. When n is odd, $\lfloor \frac{n}{2} \rfloor = \frac{n-1}{2}$ and $\left(x^k\right)^2 = x^{n-1}$, and therefore $x^n = x \cdot \left(x^k\right)^2$, just as $2^{13} = 2 \cdot 2^6 \cdot 2^6$. This analysis leads to the following recursive definition:

$$power(x,n) = \begin{cases} 1 & \text{if } n = 0 \\ x \cdot \left(power\left(x, \lfloor \frac{n}{2} \rfloor\right)\right)^2 & \text{if } n > 0 \text{ is odd} \\ \left(power\left(x, \lfloor \frac{n}{2} \rfloor\right)\right)^2 & \text{if } n > 0 \text{ is even} \end{cases}$$

If we were to implement this recursion making *two* recursive calls to compute $power(x, \lfloor \frac{n}{2} \rfloor) \cdot power(x, \lfloor \frac{n}{2} \rfloor)$, a trace of the recursion would demonstrate $O(n)$ calls. We can perform significantly fewer operations by computing $power(x, \lfloor \frac{n}{2} \rfloor)$ as a partial result, and then multiplying it by itself. An implementation based on this recursive definition is given in Code Fragment 4.12.

```
1  def power(x, n):
2    """Compute the value x**n for integer n."""
3    if n == 0:
4      return 1
5    else:
6      partial = power(x, n // 2)          # rely on truncated division
7      result = partial * partial
8      if n % 2 == 1:                      # if n odd, include extra factor of x
9        result *= x
10     return result
```

Code Fragment 4.12: Computing the power function using repeated squaring.

To illustrate the execution of our improved algorithm, Figure 4.12 provides a recursion trace of the computation power(2, 13).

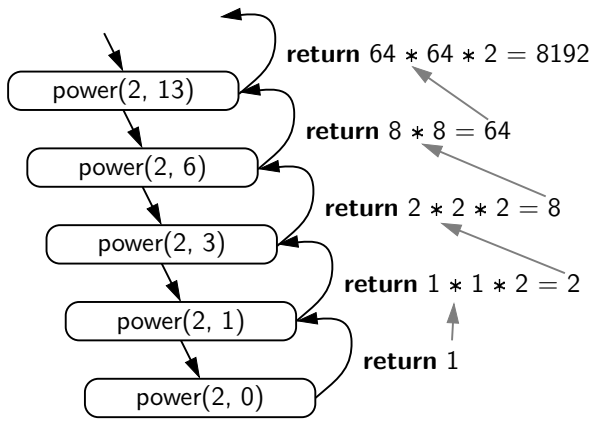


Figure 4.12: Recursion trace for an execution of power(2, 13).

To analyze the running time of the revised algorithm, we observe that the exponent in each recursive call of function power(x,n) is at most half of the preceding exponent. As we saw with the analysis of binary search, the number of times that we can divide n in half before getting to one or less is $O(\log n)$. Therefore, our new formulation of the power function results in $O(\log n)$ recursive calls. Each individual activation of the function uses $O(1)$ operations (excluding the recursive calls), and so the total number of operations for computing power(x,n) is $O(\log n)$. This is a significant improvement over the original $O(n)$-time algorithm.

The improved version also provides significant saving in reducing the memory usage. The first version has a recursive depth of $O(n)$, and therefore $O(n)$ activation records are simultaneous stored in memory. Because the recursive depth of the improved version is $O(\log n)$, its memory usages is $O(\log n)$ as well.

4.4.2 Binary Recursion

When a function makes two recursive calls, we say that it uses ***binary recursion***. We have already seen several examples of binary recursion, most notably when drawing the English ruler (Section 4.1.2), or in the bad_fibonacci function of Section 4.3. As another application of binary recursion, let us revisit the problem of summing the n elements of a sequence, S, of numbers. Computing the sum of one or zero elements is trivial. With two or more elements, we can recursively compute the sum of the first half, and the sum of the second half, and add these sums together. Our implementation of such an algorithm, in Code Fragment 4.13, is initially invoked as binary_sum(A, 0, len(A)).

```
1  def binary_sum(S, start, stop):
2    """Return the sum of the numbers in implicit slice S[start:stop]."""
3    if start >= stop:                          # zero elements in slice
4      return 0
5    elif start == stop-1:                      # one element in slice
6      return S[start]
7    else:                                      # two or more elements in slice
8      mid = (start + stop) // 2
9      return binary_sum(S, start, mid) + binary_sum(S, mid, stop)
```

Code Fragment 4.13: Summing the elements of a sequence using binary recursion.

To analyze algorithm binary_sum, we consider, for simplicity, the case where n is a power of two. Figure 4.13 shows the recursion trace of an execution of binary_sum(0, 8). We label each box with the values of parameters start:stop for that call. The size of the range is divided in half at each recursive call, and so the depth of the recursion is $1+\log_2 n$. Therefore, binary_sum uses $O(\log n)$ amount of additional space, which is a big improvement over the $O(n)$ space used by the linear_sum function of Code Fragment 4.9. However, the running time of binary_sum is $O(n)$, as there are $2n-1$ function calls, each requiring constant time.

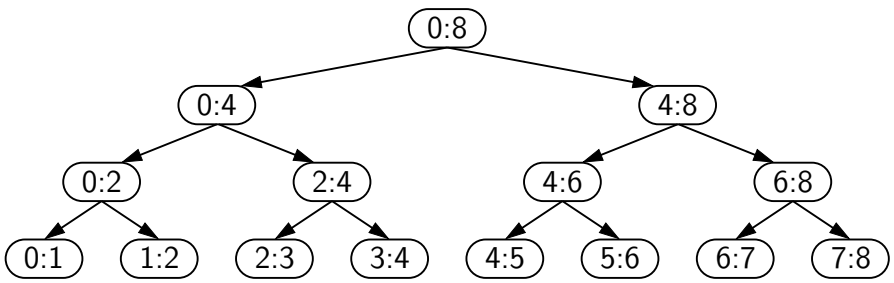


Figure 4.13: Recursion trace for the execution of binary_sum(0, 8).

4.4.3 Multiple Recursion

Generalizing from binary recursion, we define ***multiple recursion*** as a process in which a function may make more than two recursive calls. Our recursion for analyzing the disk space usage of a file system (see Section 4.1.4) is an example of multiple recursion, because the number of recursive calls made during one invocation was equal to the number of entries within a given directory of the file system.

Another common application of multiple recursion is when we want to enumerate various configurations in order to solve a combinatorial puzzle. For example, the following are all instances of what are known as ***summation puzzles***:

$$\begin{aligned} pot + pan &= bib \\ dog + cat &= pig \\ boy + girl &= baby \end{aligned}$$

To solve such a puzzle, we need to assign a unique digit (that is, $0, 1, \ldots, 9$) to each letter in the equation, in order to make the equation true. Typically, we solve such a puzzle by using our human observations of the particular puzzle we are trying to solve to eliminate configurations (that is, possible partial assignments of digits to letters) until we can work though the feasible configurations left, testing for the correctness of each one.

If the number of possible configurations is not too large, however, we can use a computer to simply enumerate all the possibilities and test each one, without employing any human observations. In addition, such an algorithm can use multiple recursion to work through the configurations in a systematic way. We show pseudocode for such an algorithm in Code Fragment 4.14. To keep the description general enough to be used with other puzzles, the algorithm enumerates and tests all k-length sequences without repetitions of the elements of a given universe U. We build the sequences of k elements by the following steps:

1. Recursively generating the sequences of $k-1$ elements
2. Appending to each such sequence an element not already contained in it.

Throughout the execution of the algorithm, we use a set U to keep track of the elements not contained in the current sequence, so that an element e has not been used yet if and only if e is in U.

Another way to look at the algorithm of Code Fragment 4.14 is that it enumerates every possible size-k ordered subset of U, and tests each subset for being a possible solution to our puzzle.

For summation puzzles, $U = \{0,1,2,3,4,5,6,7,8,9\}$ and each position in the sequence corresponds to a given letter. For example, the first position could stand for b, the second for o, the third for y, and so on.

```
Algorithm PuzzleSolve(k,S,U):
    Input: An integer k, sequence S, and set U
    Output: An enumeration of all k-length extensions to S using elements in U
      without repetitions
    for each e in U do
      Add e to the end of S
      Remove e from U                                   {e is now being used}
      if k == 1 then
        Test whether S is a configuration that solves the puzzle
        if S solves the puzzle then
          return "Solution found: " S
      else
        PuzzleSolve(k−1,S,U)                            {a recursive call}
      Remove e from the end of S
      Add e back to U                                   {e is now considered as unused}
```

Code Fragment 4.14: Solving a combinatorial puzzle by enumerating and testing all possible configurations.

In Figure 4.14, we show a recursion trace of a call to PuzzleSolve$(3,S,U)$, where S is empty and $U = \{a,b,c\}$. During the execution, all the permutations of the three characters are generated and tested. Note that the initial call makes three recursive calls, each of which in turn makes two more. If we had executed PuzzleSolve$(3,S,U)$ on a set U consisting of four elements, the initial call would have made four recursive calls, each of which would have a trace looking like the one in Figure 4.14.

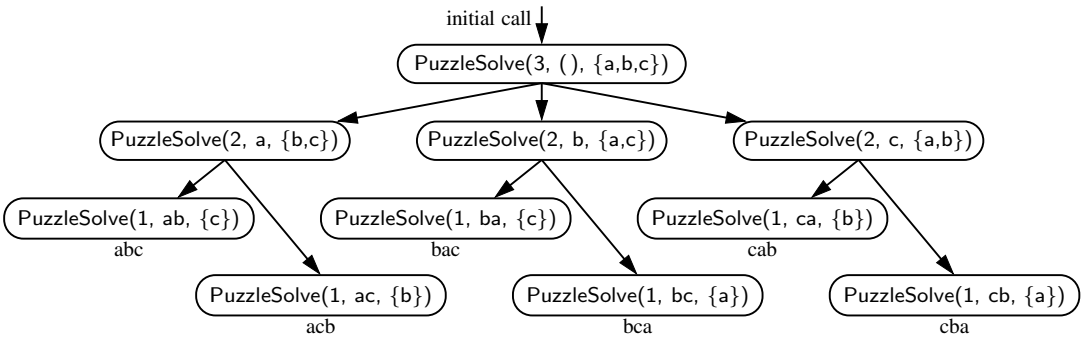


Figure 4.14: Recursion trace for an execution of PuzzleSolve$(3,S,U)$, where S is empty and $U = \{a,b,c\}$. This execution generates and tests all permutations of a, b, and c. We show the permutations generated directly below their respective boxes.

4.5 Designing Recursive Algorithms

In general, an algorithm that uses recursion typically has the following form:

- ***Test for base cases.*** We begin by testing for a set of base cases (there should be at least one). These base cases should be defined so that every possible chain of recursive calls will eventually reach a base case, and the handling of each base case should not use recursion.
- ***Recur.*** If not a base case, we perform one or more recursive calls. This recursive step may involve a test that decides which of several possible recursive calls to make. We should define each possible recursive call so that it makes progress towards a base case.

Parameterizing a Recursion

To design a recursive algorithm for a given problem, it is useful to think of the different ways we might define subproblems that have the same general structure as the original problem. If one has difficulty finding the repetitive structure needed to design a recursive algorithm, it is sometimes useful to work out the problem on a few concrete examples to see how the subproblems should be defined.

A successful recursive design sometimes requires that we redefine the original problem to facilitate similar-looking subproblems. Often, this involved reparameterizing the signature of the function. For example, when performing a binary search in a sequence, a natural function signature for a caller would appear as binary_search(data, target). However, in Section 4.1.3, we defined our function with calling signature binary_search(data, target, low, high), using the additional parameters to demarcate sublists as the recursion proceeds. This change in parameterization is critical for binary search. If we had insisted on the cleaner signature, binary_search(data, target), the only way to invoke a search on half the list would have been to make a new list instance with only those elements to send as the first parameter. However, making a copy of half the list would already take $O(n)$ time, negating the whole benefit of the binary search algorithm.

If we wished to provide a cleaner public interface to an algorithm like binary search, without bothering a user with the extra parameters, a standard technique is to make one function for public use with the cleaner interface, such as binary_search(data, target), and then having its body invoke a nonpublic utility function having the desired recursive parameters.

You will see that we similarly reparameterized the recursion in several other examples of this chapter (e.g., reverse, linear_sum, binary_sum). We saw a different approach to redefining a recursion in our good_fibonacci implementation, by intentionally strengthening the expectation of what is returned (in that case, returning a pair of numbers rather than a single number).

4.6 Eliminating Tail Recursion

The main benefit of a recursive approach to algorithm design is that it allows us to succinctly take advantage of a repetitive structure present in many problems. By making our algorithm description exploit the repetitive structure in a recursive way, we can often avoid complex case analyses and nested loops. This approach can lead to more readable algorithm descriptions, while still being quite efficient.

However, the usefulness of recursion comes at a modest cost. In particular, the Python interpreter must maintain activation records that keep track of the state of each nested call. When computer memory is at a premium, it is useful in some cases to be able to derive nonrecursive algorithms from recursive ones.

In general, we can use the stack data structure, which we will introduce in Section 6.1, to convert a recursive algorithm into a nonrecursive algorithm by managing the nesting of the recursive structure ourselves, rather than relying on the interpreter to do so. Although this only shifts the memory usage from the interpreter to our stack, we may be able to reduce the memory usage by storing only the minimal information necessary.

Even better, some forms of recursion can be eliminated without any use of axillary memory. A notable such form is known as ***tail recursion***. A recursion is a tail recursion if any recursive call that is made from one context is the very last operation in that context, with the return value of the recursive call (if any) immediately returned by the enclosing recursion. By necessity, a tail recursion must be a linear recursion (since there is no way to make a second recursive call if you must immediately return the result of the first).

Of the recursive functions demonstrated in this chapter, the binary_search function of Code Fragment 4.3 and the reverse function of Code Fragment 4.10 are examples of tail recursion. Several others of our linear recursions are almost like tail recursion, but not technically so. For example, our factorial function of Code Fragment 4.1 is *not* a tail recursion. It concludes with the command:

```
return n * factorial(n-1)
```

This is not a tail recursion because an additional multiplication is performed after the recursive call is completed. For similar reasons, the linear_sum function of Code Fragment 4.9 and the good_fibonacci function of Code Fragment 4.7 fail to be tail recursions.

Any tail recursion can be reimplemented nonrecursively by enclosing the body in a loop for repetition, and replacing a recursive call with new parameters by a reassignment of the existing parameters to those values. As a tangible example, our binary_search function can be reimplemented as shown in Code Fragment 4.15. We initialize variables low and high, just prior to our while loop, to represent the full extent of the sequence. Then, during each pass of the loop, we either find

```
1  def binary_search_iterative(data, target):
2    """Return True if target is found in the given Python list."""
3    low = 0
4    high = len(data)-1
5    while low <= high:
6      mid = (low + high) // 2
7      if target == data[mid]:                # found a match
8        return True
9      elif target < data[mid]:
10       high = mid - 1                       # only consider values left of mid
11     else:
12       low = mid + 1                        # only consider values right of mid
13   return False                             # loop ended without success
```

Code Fragment 4.15: A nonrecursive implementation of binary search.

the target, or we narrow the range of the candidate subsequence. Where we made the recursive call binary_search(data, target, low, mid -1) in the original version, we simply replace high = mid $-$ 1 in our new version and then continue to the next iteration of the loop. Our original base case condition of low $>$ high has simply been replaced by the opposite loop condition **while** low $<=$ high. In our new implementation, we return False to designate a failed search if the while loop ends (that is, without having ever returned True from within).

We can similarly develop a nonrecursive implementation (Code Fragment 4.16) of the original recursive reverse method of Code Fragment 4.10.

```
1  def reverse_iterative(S):
2    """Reverse elements in sequence S."""
3    start, stop = 0, len(S)
4    while start < stop - 1:
5      S[start], S[stop-1] = S[stop-1], S[start]      # swap first and last
6      start, stop = start + 1, stop - 1              # narrow the range
```

Code Fragment 4.16: Reversing the elements of a sequence using iteration.

In this new version, we update the values start and stop during each pass of the loop, exiting once we reach the case of having one or less elements in that range.

Many other linear recursions can be expressed quite efficiently with iteration, even if they were not formally tail recursions. For example, there are trivial nonrecursive implementations for computing factorials, summing elements of a sequence, or computing Fibonacci numbers efficiently. In fact, our implementation of a Fibonacci generator, from Section 1.8, produces each subsequent value in $O(1)$ time, and thus takes $O(n)$ time to generate the nth entry in the series.

4.7 Exercises

For help with exercises, please visit the site, www.wiley.com/college/goodrich.

Reinforcement

R-4.1 Describe a recursive algorithm for finding the maximum element in a sequence, S, of n elements. What is your running time and space usage?

R-4.2 Draw the recursion trace for the computation of $power(2,5)$, using the traditional function implemented in Code Fragment 4.11.

R-4.3 Draw the recursion trace for the computation of $power(2,18)$, using the repeated squaring algorithm, as implemented in Code Fragment 4.12.

R-4.4 Draw the recursion trace for the execution of function reverse(S, 0, 5) (Code Fragment 4.10) on S = [4, 3, 6, 2, 6].

R-4.5 Draw the recursion trace for the execution of function PuzzleSolve$(3,S,U)$ (Code Fragment 4.14), where S is empty and $U = \{a,b,c,d\}$.

R-4.6 Describe a recursive function for computing the nth ***Harmonic number***, $H_n = \sum_{i=1}^{n} 1/i$.

R-4.7 Describe a recursive function for converting a string of digits into the integer it represents. For example, `'13531'` represents the integer 13,531.

R-4.8 Isabel has an interesting way of summing up the values in a sequence A of n integers, where n is a power of two. She creates a new sequence B of half the size of A and sets $B[i] = A[2i] + A[2i+1]$, for $i = 0,1,\ldots,(n/2)-1$. If B has size 1, then she outputs $B[0]$. Otherwise, she replaces A with B, and repeats the process. What is the running time of her algorithm?

Creativity

C-4.9 Write a short recursive Python function that finds the minimum and maximum values in a sequence without using any loops.

C-4.10 Describe a recursive algorithm to compute the integer part of the base-two logarithm of n using only addition and integer division.

C-4.11 Describe an efficient recursive function for solving the element uniqueness problem, which runs in time that is at most $O(n^2)$ in the worst case without using sorting.

C-4.12 Give a recursive algorithm to compute the product of two positive integers, m and n, using only addition and subtraction.

C-4.13 In Section 4.2 we prove by induction that the number of *lines* printed by a call to draw_interval(c) is $2^c - 1$. Another interesting question is how many *dashes* are printed during that process. Prove by induction that the number of dashes printed by draw_interval(c) is $2^{c+1} - c - 2$.

C-4.14 In the ***Towers of Hanoi*** puzzle, we are given a platform with three pegs, a, b, and c, sticking out of it. On peg a is a stack of n disks, each larger than the next, so that the smallest is on the top and the largest is on the bottom. The puzzle is to move all the disks from peg a to peg c, moving one disk at a time, so that we never place a larger disk on top of a smaller one. See Figure 4.15 for an example of the case $n = 4$. Describe a recursive algorithm for solving the Towers of Hanoi puzzle for arbitrary n. (Hint: Consider first the subproblem of moving all but the nth disk from peg a to another peg using the third as "temporary storage.")

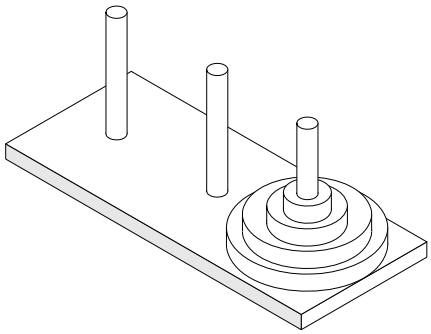


Figure 4.15: An illustration of the Towers of Hanoi puzzle.

C-4.15 Write a recursive function that will output all the subsets of a set of n elements (without repeating any subsets).

C-4.16 Write a short recursive Python function that takes a character string s and outputs its reverse. For example, the reverse of `'pots&pans'` would be `'snap&stop'`.

C-4.17 Write a short recursive Python function that determines if a string s is a palindrome, that is, it is equal to its reverse. For example, `'racecar'` and `'gohangasalamiimalasagnahog'` are palindromes.

C-4.18 Use recursion to write a Python function for determining if a string s has more vowels than consonants.

C-4.19 Write a short recursive Python function that rearranges a sequence of integer values so that all the even values appear before all the odd values.

C-4.20 Given an unsorted sequence, S, of integers and an integer k, describe a recursive algorithm for rearranging the elements in S so that all elements less than or equal to k come before any elements larger than k. What is the running time of your algorithm on a sequence of n values?

C-4.21 Suppose you are given an n-element sequence, S, containing distinct integers that are listed in increasing order. Given a number k, describe a recursive algorithm to find two integers in S that sum to k, if such a pair exists. What is the running time of your algorithm?

C-4.22 Develop a nonrecursive implementation of the version of power from Code Fragment 4.12 that uses repeated squaring.

Projects

P-4.23 Implement a recursive function with signature find(path, filename) that reports all entries of the file system rooted at the given path having the given file name.

P-4.24 Write a program for solving summation puzzles by enumerating and testing all possible configurations. Using your program, solve the three puzzles given in Section 4.4.3.

P-4.25 Provide a nonrecursive implementation of the draw_interval function for the English ruler project of Section 4.1.2. There should be precisely $2^c - 1$ lines of output if c represents the length of the center tick. If incrementing a counter from 0 to $2^c - 2$, the number of dashes for each tick line should be exactly one more than the number of consecutive 1's at the end of the binary representation of the counter.

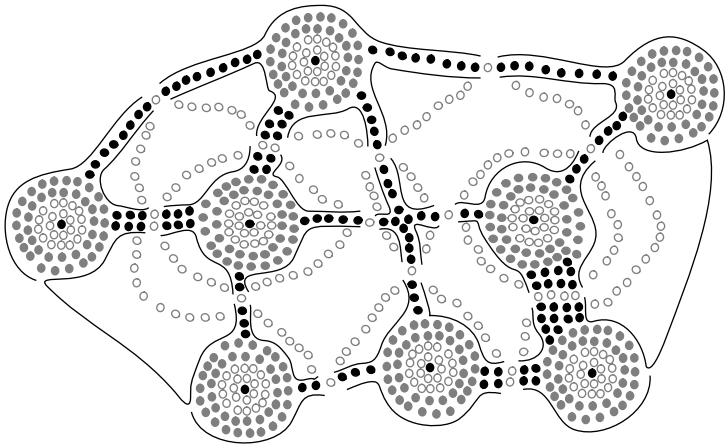
P-4.26 Write a program that can solve instances of the Tower of Hanoi problem (from Exercise C-4.14).

P-4.27 Python's os module provides a function with signature walk(path) that is a generator yielding the tuple (dirpath, dirnames, filenames) for each subdirectory of the directory identified by string path, such that string dirpath is the full path to the subdirectory, dirnames is a list of the names of the subdirectories within dirpath, and filenames is a list of the names of non-directory entries of dirpath. For example, when visiting the cs016 subdirectory of the file system shown in Figure 4.6, the walk would yield ('/user/rt/courses/cs016', ['homeworks', 'programs'], ['grades']). Give your own implementation of such a walk function.

Chapter Notes

The use of recursion in programs belongs to the folkore of computer science (for example, see the article of Dijkstra [36]). It is also at the heart of functional programming languages (for example, see the classic book by Abelson, Sussman, and Sussman [1]). Interestingly, binary search was first published in 1946, but was not published in a fully correct form until 1962. For further discussions on lessons learned, please see papers by Bentley [14] and Lesuisse [68].

Chapter 5 Array-Based Sequences



Contents

5.1 Python's Sequence Types 184
5.2 Low-Level Arrays 185
5.2.1 Referential Arrays 187
5.2.2 Compact Arrays in Python 190
5.3 Dynamic Arrays and Amortization 192
5.3.1 Implementing a Dynamic Array 195
5.3.2 Amortized Analysis of Dynamic Arrays 197
5.3.3 Python's List Class 201
5.4 Efficiency of Python's Sequence Types 202
5.4.1 Python's List and Tuple Classes 202
5.4.2 Python's String Class 208
5.5 Using Array-Based Sequences 210
5.5.1 Storing High Scores for a Game 210
5.5.2 Sorting a Sequence 214
5.5.3 Simple Cryptography 216
5.6 Multidimensional Data Sets 219
5.7 Exercises 224

5.1 Python's Sequence Types

In this chapter, we explore Python's various "sequence" classes, namely the built-in **list**, **tuple**, and **str** classes. There is significant commonality between these classes, most notably: each supports indexing to access an individual element of a sequence, using a syntax such as seq[k], and each uses a low-level concept known as an ***array*** to represent the sequence. However, there are significant differences in the abstractions that these classes represent, and in the way that instances of these classes are represented internally by Python. Because these classes are used so widely in Python programs, and because they will become building blocks upon which we will develop more complex data structures, it is imperative that we establish a clear understanding of both the public behavior and inner workings of these classes.

Public Behaviors

A proper understanding of the outward semantics for a class is a necessity for a good programmer. While the basic usage of lists, strings, and tuples may seem straightforward, there are several important subtleties regarding the behaviors associated with these classes (such as what it means to make a copy of a sequence, or to take a slice of a sequence). Having a misunderstanding of a behavior can easily lead to inadvertent bugs in a program. Therefore, we establish an accurate mental model for each of these classes. These images will help when exploring more advanced usage, such as representing a multidimensional data set as a list of lists.

Implementation Details

A focus on the internal implementations of these classes seems to go against our stated principles of object-oriented programming. In Section 2.1.2, we emphasized the principle of ***encapsulation***, noting that the user of a class need not know about the internal details of the implementation. While it is true that one only needs to understand the syntax and semantics of a class's public interface in order to be able to write legal and correct code that uses instances of the class, the efficiency of a program depends greatly on the efficiency of the components upon which it relies.

Asymptotic and Experimental Analyses

In describing the efficiency of various operations for Python's sequence classes, we will rely on the formal ***asymptotic analysis*** notations established in Chapter 3. We will also perform experimental analyses of the primary operations to provide empirical evidence that is consistent with the more theoretical asymptotic analyses.

5.2 Low-Level Arrays

To accurately describe the way in which Python represents the sequence types, we must first discuss aspects of the low-level computer architecture. The primary memory of a computer is composed of bits of information, and those bits are typically grouped into larger units that depend upon the precise system architecture. Such a typical unit is a ***byte***, which is equivalent to 8 bits.

A computer system will have a huge number of bytes of memory, and to keep track of what information is stored in what byte, the computer uses an abstraction known as a ***memory address***. In effect, each byte of memory is associated with a unique number that serves as its address (more formally, the binary representation of the number serves as the address). In this way, the computer system can refer to the data in "byte #2150" versus the data in "byte #2157," for example. Memory addresses are typically coordinated with the physical layout of the memory system, and so we often portray the numbers in sequential fashion. Figure 5.1 provides such a diagram, with the designated memory address for each byte.

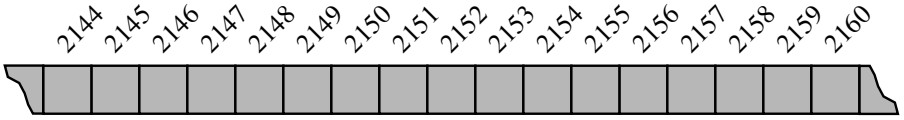


Figure 5.1: A representation of a portion of a computer's memory, with individual bytes labeled with consecutive memory addresses.

Despite the sequential nature of the numbering system, computer hardware is designed, in theory, so that any byte of the main memory can be efficiently accessed based upon its memory address. In this sense, we say that a computer's main memory performs as ***random access memory (RAM)***. That is, it is just as easy to retrieve byte #8675309 as it is to retrieve byte #309. (In practice, there are complicating factors including the use of caches and external memory; we address some of those issues in Chapter 15.) Using the notation for asymptotic analysis, we say that any individual byte of memory can be stored or retrieved in $O(1)$ time.

In general, a programming language keeps track of the association between an identifier and the memory address in which the associated value is stored. For example, identifier `x` might be associated with one value stored in memory, while `y` is associated with another value stored in memory. A common programming task is to keep track of a sequence of related objects. For example, we may want a video game to keep track of the top ten scores for that game. Rather than use ten different variables for this task, we would prefer to use a single name for the group and use index numbers to refer to the high scores in that group.

A group of related variables can be stored one after another in a contiguous portion of the computer's memory. We will denote such a representation as an ***array***. As a tangible example, a text string is stored as an ordered sequence of individual characters. In Python, each character is represented using the Unicode character set, and on most computing systems, Python internally represents each Unicode character with 16 bits (i.e., 2 bytes). Therefore, a six-character string, such as `'SAMPLE'`, would be stored in 12 consecutive bytes of memory, as diagrammed in Figure 5.2.

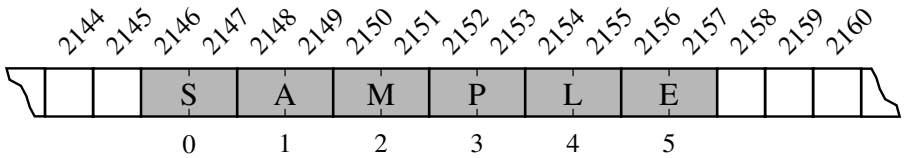


Figure 5.2: A Python string embedded as an array of characters in the computer's memory. We assume that each Unicode character of the string requires two bytes of memory. The numbers below the entries are indices into the string.

We describe this as an *array of six characters*, even though it requires 12 bytes of memory. We will refer to each location within an array as a ***cell***, and will use an integer ***index*** to describe its location within the array, with cells numbered starting with 0, 1, 2, and so on. For example, in Figure 5.2, the cell of the array with index 4 has contents `L` and is stored in bytes 2154 and 2155 of memory.

Each cell of an array must use the same number of bytes. This requirement is what allows an arbitrary cell of the array to be accessed in constant time based on its index. In particular, if one knows the memory address at which an array starts (e.g., 2146 in Figure 5.2), the number of bytes per element (e.g., 2 for a Unicode character), and a desired index within the array, the appropriate memory address can be computed using the calculation, $\mathsf{start} + \mathsf{cellsize} * \mathsf{index}$. By this formula, the cell at index 0 begins precisely at the start of the array, the cell at index 1 begins precisely cellsize bytes beyond the start of the array, and so on. As an example, cell 4 of Figure 5.2 begins at memory location $2146 + 2 \cdot 4 = 2146 + 8 = 2154$.

Of course, the arithmetic for calculating memory addresses within an array can be handled automatically. Therefore, a programmer can envision a more typical high-level abstraction of an array of characters as diagrammed in Figure 5.3.

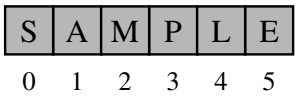


Figure 5.3: A higher-level abstraction for the string portrayed in Figure 5.2.

5.2.1 Referential Arrays

As another motivating example, assume that we want a medical information system to keep track of the patients currently assigned to beds in a certain hospital. If we assume that the hospital has 200 beds, and conveniently that those beds are numbered from 0 to 199, we might consider using an array-based structure to maintain the names of the patients currently assigned to those beds. For example, in Python we might use a list of names, such as:

```
['Rene', 'Joseph', 'Janet', 'Jonas', 'Helen', 'Virginia', ... ]
```

To represent such a list with an array, Python must adhere to the requirement that each cell of the array use the same number of bytes. Yet the elements are strings, and strings naturally have different lengths. Python could attempt to reserve enough space for each cell to hold the *maximum* length string (not just of currently stored strings, but of any string we might ever want to store), but that would be wasteful.

Instead, Python represents a list or tuple instance using an internal storage mechanism of an array of object ***references***. At the lowest level, what is stored is a consecutive sequence of memory addresses at which the elements of the sequence reside. A high-level diagram of such a list is shown in Figure 5.4.

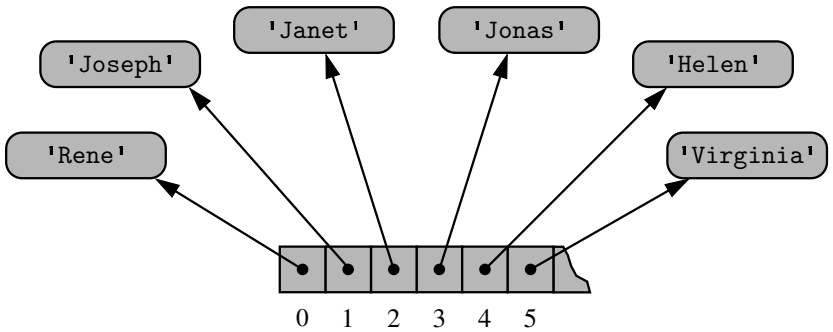


Figure 5.4: An array storing references to strings.

Although the relative size of the individual elements may vary, the number of bits used to store the memory address of each element is fixed (e.g., 64-bits per address). In this way, Python can support constant-time access to a list or tuple element based on its index.

In Figure 5.4, we characterize a list of strings that are the names of the patients in a hospital. It is more likely that a medical information system would manage more comprehensive information on each patient, perhaps represented as an instance of a Patient class. From the perspective of the list implementation, the same principle applies: The list will simply keep a sequence of references to those objects. Note as well that a reference to the **None** object can be used as an element of the list to represent an empty bed in the hospital.

The fact that lists and tuples are referential structures is significant to the semantics of these classes. A single list instance may include multiple references to the same object as elements of the list, and it is possible for a single object to be an element of two or more lists, as those lists simply store references back to that object. As an example, when you compute a slice of a list, the result is a new list instance, but that new list has references to the same elements that are in the original list, as portrayed in Figure 5.5.

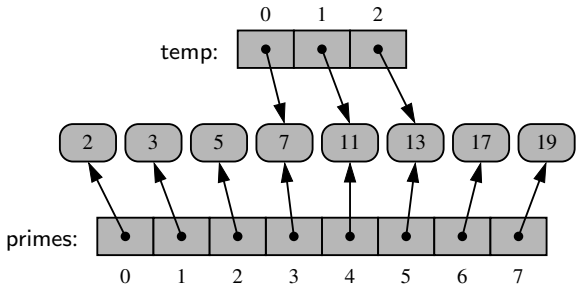


Figure 5.5: The result of the command temp = primes[3:6].

When the elements of the list are immutable objects, as with the integer instances in Figure 5.5, the fact that the two lists share elements is not that significant, as neither of the lists can cause a change to the shared object. If, for example, the command temp[2] = 15 were executed from this configuration, that does not change the existing integer object; it changes the reference in cell 2 of the temp list to reference a different object. The resulting configuration is shown in Figure 5.6.

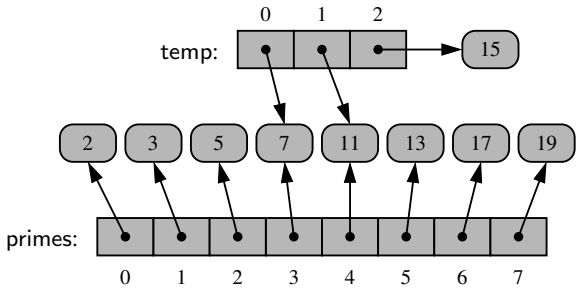


Figure 5.6: The result of the command temp[2] = 15 upon the configuration portrayed in Figure 5.5.

The same semantics is demonstrated when making a new list as a copy of an existing one, with a syntax such as backup = **list**(primes). This produces a new list that is a ***shallow copy*** (see Section 2.6), in that it references the same elements as in the first list. With immutable elements, this point is moot. If the contents of the list were of a mutable type, a ***deep copy***, meaning a new list with *new* elements, can be produced by using the deepcopy function from the copy module.

As a more striking example, it is a common practice in Python to initialize an array of integers using a syntax such as counters $=$ [0] $*$ 8. This syntax produces a list of length eight, with all eight elements being the value zero. Technically, all eight cells of the list reference the *same* object, as portrayed in Figure 5.7.

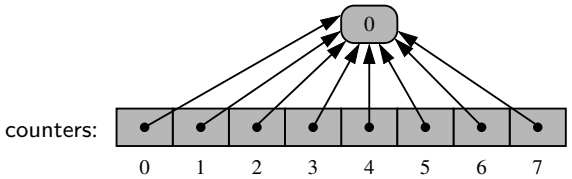


Figure 5.7: The result of the command data $=$ [0] $*$ 8.

At first glance, the extreme level of aliasing in this configuration may seem alarming. However, we rely on the fact that the referenced integer is immutable. Even a command such as counters[2] $+=$ 1 does not technically change the value of the existing integer instance. This computes a new integer, with value $0+1$, and sets cell 2 to reference the newly computed value. The resulting configuration is shown in Figure 5.8.

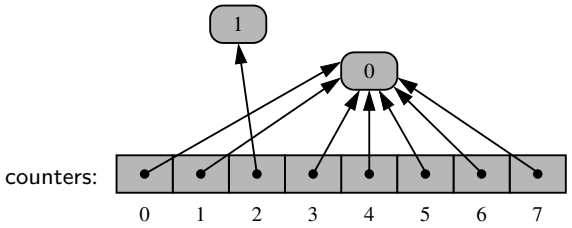


Figure 5.8: The result of command data[2] $+=$ 1 upon the list from Figure 5.7.

As a final manifestation of the referential nature of lists, we note that the extend command is used to add all elements from one list to the end of another list. The extended list does not receive copies of those elements, it receives references to those elements. Figure 5.9 portrays the effect of a call to extend.

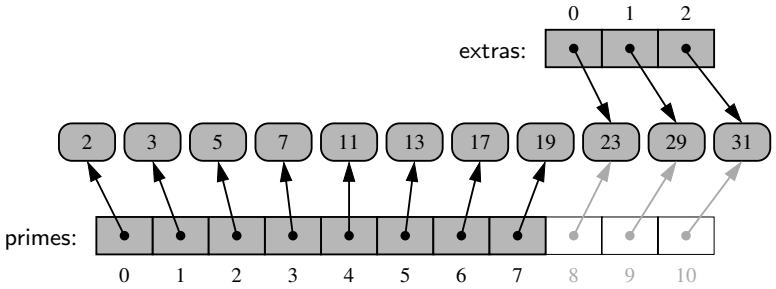
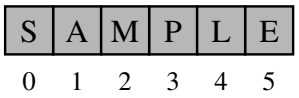


Figure 5.9: The effect of command primes.extend(extras), shown in light gray.

5.2.2 Compact Arrays in Python

In the introduction to this section, we emphasized that strings are represented using an array of characters (not an array of references). We will refer to this more direct representation as a ***compact array*** because the array is storing the bits that represent the primary data (characters, in the case of strings).



Compact arrays have several advantages over referential structures in terms of computing performance. Most significantly, the overall memory usage will be much lower for a compact structure because there is no overhead devoted to the explicit storage of the sequence of memory references (in addition to the primary data). That is, a referential structure will typically use 64-bits for the memory address stored in the array, on top of whatever number of bits are used to represent the object that is considered the element. Also, each Unicode character stored in a compact array within a string typically requires 2 bytes. If each character were stored independently as a one-character string, there would be significantly more bytes used.

As another case study, suppose we wish to store a sequence of one million, 64-bit integers. In theory, we might hope to use only 64 million bits. However, we estimate that a Python list will use *four to five times as much memory*. Each element of the list will result in a 64-bit memory address being stored in the primary array, and an **int** instance being stored elsewhere in memory. Python allows you to query the actual number of bytes being used for the primary storage of any object. This is done using the getsizeof function of the sys module. On our system, the size of a typical int object requires 14 bytes of memory (well beyond the 4 bytes needed for representing the actual 64-bit number). In all, the list will be using 18 bytes per entry, rather than the 4 bytes that a compact list of integers would require.

Another important advantage to a compact structure for high-performance computing is that the primary data are stored consecutively in memory. Note well that this is not the case for a referential structure. That is, even though a list maintains careful ordering of the sequence of memory addresses, where those elements reside in memory is not determined by the list. Because of the workings of the cache and memory hierarchies of computers, it is often advantageous to have data stored in memory near other data that might be used in the same computations.

Despite the apparent inefficiencies of referential structures, we will generally be content with the convenience of Python's lists and tuples in this book. The only place in which we consider alternatives will be in Chapter 15, which focuses on the impact of memory usage on data structures and algorithms. Python provides several means for creating compact arrays of various types.

Primary support for compact arrays is in a module named array. That module defines a class, also named array, providing compact storage for arrays of primitive data types. A portrayal of such an array of integers is shown in Figure 5.10.

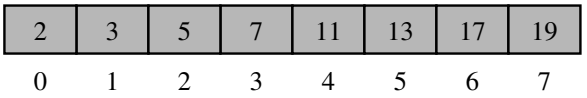


Figure 5.10: Integers stored compactly as elements of a Python array.

The public interface for the array class conforms mostly to that of a Python list. However, the constructor for the array class requires a ***type code*** as a first parameter, which is a character that designates the type of data that will be stored in the array. As a tangible example, the type code, `'i'`, designates an array of (signed) integers, typically represented using at least 16-bits each. We can declare the array shown in Figure 5.10 as,

```
primes = array('i', [2, 3, 5, 7, 11, 13, 17, 19])
```

The type code allows the interpreter to determine precisely how many bits are needed per element of the array. The type codes supported by the array module, as shown in Table 5.1, are formally based upon the native data types used by the C programming language (the language in which the the most widely used distribution of Python is implemented). The precise number of bits for the C data types is system-dependent, but typical ranges are shown in the table.

Code	C Data Type	Typical Number of Bytes
`'b'`	signed char	1
`'B'`	unsigned char	1
`'u'`	Unicode char	2 or 4
`'h'`	signed short int	2
`'H'`	unsigned short int	2
`'i'`	signed int	2 or 4
`'I'`	unsigned int	2 or 4
`'l'`	signed long int	4
`'L'`	unsigned long int	4
`'f'`	float	4
`'d'`	float	8

Table 5.1: Type codes supported by the array module.

The array module does not provide support for making compact arrays of user-defined data types. Compact arrays of such structures can be created with the lower-level support of a module named ctypes. (See Section 5.3.1 for more discussion of the ctypes module.)

5.3 Dynamic Arrays and Amortization

When creating a low-level array in a computer system, the precise size of that array must be explicitly declared in order for the system to properly allocate a consecutive piece of memory for its storage. For example, Figure 5.11 displays an array of 12 bytes that might be stored in memory locations 2146 through 2157.

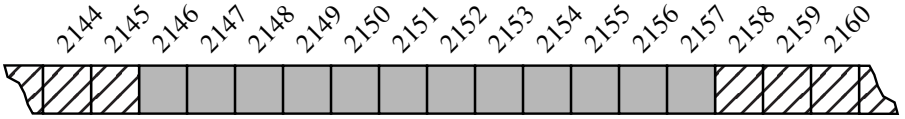


Figure 5.11: An array of 12 bytes allocated in memory locations 2146 through 2157.

Because the system might dedicate neighboring memory locations to store other data, the capacity of an array cannot trivially be increased by expanding into subsequent cells. In the context of representing a Python `tuple` or `str` instance, this constraint is no problem. Instances of those classes are immutable, so the correct size for an underlying array can be fixed when the object is instantiated.

Python's `list` class presents a more interesting abstraction. Although a list has a particular length when constructed, the class allows us to add elements to the list, with no apparent limit on the overall capacity of the list. To provide this abstraction, Python relies on an algorithmic sleight of hand known as a ***dynamic array***.

The first key to providing the semantics of a dynamic array is that a list instance maintains an underlying array that often has greater capacity than the current length of the list. For example, while a user may have created a list with five elements, the system may have reserved an underlying array capable of storing eight object references (rather than only five). This extra capacity makes it easy to append a new element to the list by using the next available cell of the array.

If a user continues to append elements to a list, any reserved capacity will eventually be exhausted. In that case, the class requests a new, larger array from the system, and initializes the new array so that its prefix matches that of the existing smaller array. At that point in time, the old array is no longer needed, so it is reclaimed by the system. Intuitively, this strategy is much like that of the hermit crab, which moves into a larger shell when it outgrows its previous one.

We give empirical evidence that Python's `list` class is based upon such a strategy. The source code for our experiment is displayed in Code Fragment 5.1, and a sample output of that program is given in Code Fragment 5.2. We rely on a function named `getsizeof` that is available from the `sys` module. This function reports the number of bytes that are being used to store an object in Python. For a list, it reports the number of bytes devoted to the array and other instance variables of the list, but *not* any space devoted to elements referenced by the list.

```
1  import sys                                  # provides getsizeof function
2  data = [ ]
3  for k in range(n):                          # NOTE: must fix choice of n
4    a = len(data)                             # number of elements
5    b = sys.getsizeof(data)                   # actual size in bytes
6    print('Length: {0:3d}; Size in bytes: {1:4d}'.format(a, b))
7    data.append(None)                         # increase length by one
```

Code Fragment 5.1: An experiment to explore the relationship between a list's length and its underlying size in Python.

```
Length:   0; Size in bytes:   72
Length:   1; Size in bytes:  104
Length:   2; Size in bytes:  104
Length:   3; Size in bytes:  104
Length:   4; Size in bytes:  104
Length:   5; Size in bytes:  136
Length:   6; Size in bytes:  136
Length:   7; Size in bytes:  136
Length:   8; Size in bytes:  136
Length:   9; Size in bytes:  200
Length:  10; Size in bytes:  200
Length:  11; Size in bytes:  200
Length:  12; Size in bytes:  200
Length:  13; Size in bytes:  200
Length:  14; Size in bytes:  200
Length:  15; Size in bytes:  200
Length:  16; Size in bytes:  200
Length:  17; Size in bytes:  272
Length:  18; Size in bytes:  272
Length:  19; Size in bytes:  272
Length:  20; Size in bytes:  272
Length:  21; Size in bytes:  272
Length:  22; Size in bytes:  272
Length:  23; Size in bytes:  272
Length:  24; Size in bytes:  272
Length:  25; Size in bytes:  272
Length:  26; Size in bytes:  352
```

Code Fragment 5.2: Sample output from the experiment of Code Fragment 5.1.

In evaluating the results of the experiment, we draw attention to the first line of output from Code Fragment 5.2. We see that an empty list instance already requires a certain number of bytes of memory (72 on our system). In fact, each object in Python maintains some state, for example, a reference to denote the class to which it belongs. Although we cannot directly access private instance variables for a list, we can speculate that in some form it maintains state information akin to:

_n	The number of actual elements currently stored in the list.
_capacity	The maximum number of elements that could be stored in the currently allocated array.
_A	The reference to the currently allocated array (initially None).

As soon as the first element is inserted into the list, we detect a change in the underlying size of the structure. In particular, we see the number of bytes jump from 72 to 104, an increase of exactly 32 bytes. Our experiment was run on a 64-bit machine architecture, meaning that each memory address is a 64-bit number (i.e., 8 bytes). We speculate that the increase of 32 bytes reflects the allocation of an underlying array capable of storing four object references. This hypothesis is consistent with the fact that we do not see any underlying change in the memory usage after inserting the second, third, or fourth element into the list.

After the fifth element has been added to the list, we see the memory usage jump from 104 bytes to 136 bytes. If we assume the original base usage of 72 bytes for the list, the total of 136 suggests an additional $64 = 8 \times 8$ bytes that provide capacity for up to eight object references. Again, this is consistent with the experiment, as the memory usage does not increase again until the ninth insertion. At that point, the 200 bytes can be viewed as the original 72 plus an additional 128-byte array to store 16 object references. The 17th insertion pushes the overall memory usage to $272 = 72 + 200 = 72 + 25 \times 8$, hence enough to store up to 25 element references.

Because a list is a referential structure, the result of getsizeof for a list instance only includes the size for representing its primary structure; it does not account for memory used by the *objects* that are elements of the list. In our experiment, we repeatedly append None to the list, because we do not care about the contents, but we could append any type of object without affecting the number of bytes reported by getsizeof(data).

If we were to continue such an experiment for further iterations, we might try to discern the pattern for how large of an array Python creates each time the capacity of the previous array is exhausted (see Exercises R-5.2 and C-5.13). Before exploring the precise sequence of capacities used by Python, we continue in this section by describing a general approach for implementing dynamic arrays and for performing an asymptotic analysis of their performance.

5.3.1 Implementing a Dynamic Array

Although the Python list class provides a highly optimized implementation of dynamic arrays, upon which we rely for the remainder of this book, it is instructive to see how such a class might be implemented.

The key is to provide means to grow the array A that stores the elements of a list. Of course, we cannot actually grow that array, as its capacity is fixed. If an element is appended to a list at a time when the underlying array is full, we perform the following steps:

1. Allocate a new array B with larger capacity.
2. Set $B[i] = A[i]$, for $i = 0, \ldots, n-1$, where n denotes current number of items.
3. Set $A = B$, that is, we henceforth use B as the array supporting the list.
4. Insert the new element in the new array.

An illustration of this process is shown in Figure 5.12.

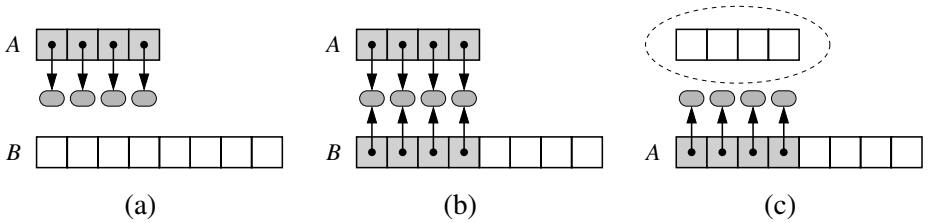


Figure 5.12: An illustration of the three steps for "growing" a dynamic array: (a) create new array B; (b) store elements of A in B; (c) reassign reference A to the new array. Not shown is the future garbage collection of the old array, or the insertion of the new element.

The remaining issue to consider is how large of a new array to create. A commonly used rule is for the new array to have twice the capacity of the existing array that has been filled. In Section 5.3.2, we will provide a mathematical analysis to justify such a choice.

In Code Fragment 5.3, we offer a concrete implementation of dynamic arrays in Python. Our DynamicArray class is designed using ideas described in this section. While consistent with the interface of a Python list class, we provide only limited functionality in the form of an append method, and accessors __len__ and __getitem__. Support for creating low-level arrays is provided by a module named ctypes. Because we will not typically use such a low-level structure in the remainder of this book, we omit a detailed explanation of the ctypes module. Instead, we wrap the necessary command for declaring the raw array within a private utility method _make_array. The hallmark expansion procedure is performed in our nonpublic _resize method.

```
1  import ctypes                                    # provides low-level arrays
2
3  class DynamicArray:
4    """A dynamic array class akin to a simplified Python list."""
5
6    def __init__(self):
7      """Create an empty array."""
8      self._n = 0                                  # count actual elements
9      self._capacity = 1                           # default array capacity
10     self._A = self._make_array(self._capacity)   # low-level array
11
12   def __len__(self):
13     """Return number of elements stored in the array."""
14     return self._n
15
16   def __getitem__(self, k):
17     """Return element at index k."""
18     if not 0 <= k < self._n:
19       raise IndexError('invalid index')
20     return self._A[k]                            # retrieve from array
21
22   def append(self, obj):
23     """Add object to end of the array."""
24     if self._n == self._capacity:                # not enough room
25       self._resize(2 * self._capacity)           # so double capacity
26     self._A[self._n] = obj
27     self._n += 1
28
29   def _resize(self, c):                          # nonpublic utitity
30     """Resize internal array to capacity c."""
31     B = self._make_array(c)                      # new (bigger) array
32     for k in range(self._n):                     # for each existing value
33       B[k] = self._A[k]
34     self._A = B                                  # use the bigger array
35     self._capacity = c
36
37   def _make_array(self, c):                      # nonpublic utitity
38      """Return new array with capacity c."""
39      return (c * ctypes.py_object)( )           # see ctypes documentation
```

Code Fragment 5.3: An implementation of a DynamicArray class, using a raw array from the ctypes module as storage.

5.3.2 Amortized Analysis of Dynamic Arrays

In this section, we perform a detailed analysis of the running time of operations on dynamic arrays. We use the big-Omega notation introduced in Section 3.3.1 to give an asymptotic lower bound on the running time of an algorithm or step within it.

The strategy of replacing an array with a new, larger array might at first seem slow, because a single append operation may require $\Omega(n)$ time to perform, where n is the current number of elements in the array. However, notice that by doubling the capacity during an array replacement, our new array allows us to add n new elements before the array must be replaced again. In this way, there are many simple append operations for each expensive one (see Figure 5.13). This fact allows us to show that performing a series of operations on an initially empty dynamic array is efficient in terms of its total running time.

Using an algorithmic design pattern called ***amortization***, we can show that performing a sequence of such append operations on a dynamic array is actually quite efficient. To perform an ***amortized analysis***, we use an accounting technique where we view the computer as a coin-operated appliance that requires the payment of one ***cyber-dollar*** for a constant amount of computing time. When an operation is executed, we should have enough cyber-dollars available in our current "bank account" to pay for that operation's running time. Thus, the total amount of cyber-dollars spent for any computation will be proportional to the total time spent on that computation. The beauty of using this analysis method is that we can overcharge some operations in order to save up cyber-dollars to pay for others.

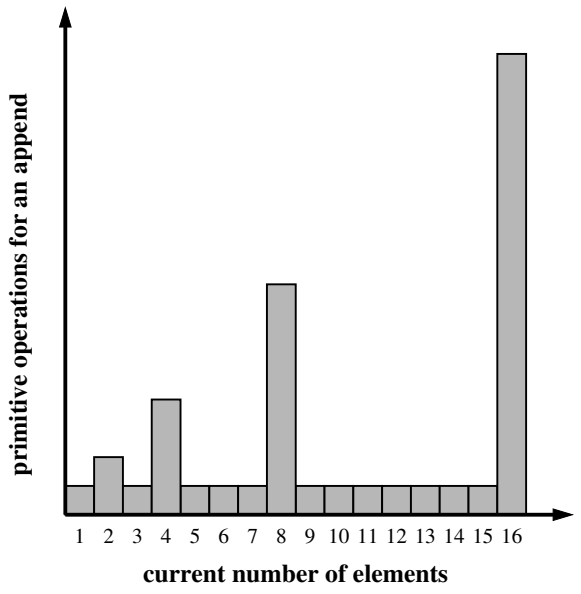


Figure 5.13: Running times of a series of append operations on a dynamic array.

Proposition 5.1: *Let S be a sequence implemented by means of a dynamic array with initial capacity one, using the strategy of doubling the array size when full. The total time to perform a series of n append operations in S, starting from S being empty, is $O(n)$.*

Justification: Let us assume that one cyber-dollar is enough to pay for the execution of each append operation in S, excluding the time spent for growing the array. Also, let us assume that growing the array from size k to size $2k$ requires k cyber-dollars for the time spent initializing the new array. We shall charge each append operation three cyber-dollars. Thus, we overcharge each append operation that does not cause an overflow by two cyber-dollars. Think of the two cyber-dollars profited in an insertion that does not grow the array as being "stored" with the cell in which the element was inserted. An overflow occurs when the array S has 2^i elements, for some integer $i \geq 0$, and the size of the array used by the array representing S is 2^i. Thus, doubling the size of the array will require 2^i cyber-dollars. Fortunately, these cyber-dollars can be found stored in cells 2^{i-1} through $2^i - 1$. (See Figure 5.14.) Note that the previous overflow occurred when the number of elements became larger than 2^{i-1} for the first time, and thus the cyber-dollars stored in cells 2^{i-1} through $2^i - 1$ have not yet been spent. Therefore, we have a valid amortization scheme in which each operation is charged three cyber-dollars and all the computing time is paid for. That is, we can pay for the execution of n append operations using $3n$ cyber-dollars. In other words, the amortized running time of each append operation is $O(1)$; hence, the total running time of n append operations is $O(n)$. ■

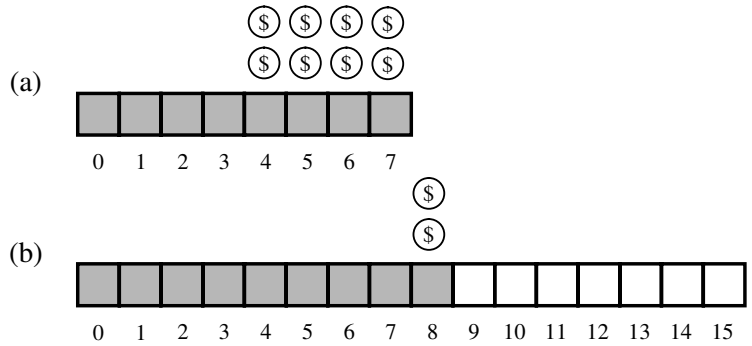


Figure 5.14: Illustration of a series of append operations on a dynamic array: (a) an 8-cell array is full, with two cyber-dollars "stored" at cells 4 through 7; (b) an append operation causes an overflow and a doubling of capacity. Copying the eight old elements to the new array is paid for by the cyber-dollars already stored in the table. Inserting the new element is paid for by one of the cyber-dollars charged to the current append operation, and the two cyber-dollars profited are stored at cell 8.

Geometric Increase in Capacity

Although the proof of Proposition 5.1 relies on the array being doubled each time we expand, the $O(1)$ amortized bound per operation can be proven for any geometrically increasing progression of array sizes (see Section 2.4.2 for discussion of geometric progressions). When choosing the geometric base, there exists a trade-off between run-time efficiency and memory usage. With a base of 2 (i.e., doubling the array), if the last insertion causes a resize event, the array essentially ends up twice as large as it needs to be. If we instead increase the array by only 25% of its current size (i.e., a geometric base of 1.25), we do not risk wasting as much memory in the end, but there will be more intermediate resize events along the way. Still it is possible to prove an $O(1)$ amortized bound, using a constant factor greater than the 3 cyber-dollars per operation used in the proof of Proposition 5.1 (see Exercise C-5.15). The key to the performance is that the amount of additional space is proportional to the current size of the array.

Beware of Arithmetic Progression

To avoid reserving too much space at once, it might be tempting to implement a dynamic array with a strategy in which a constant number of additional cells are reserved each time an array is resized. Unfortunately, the overall performance of such a strategy is significantly worse. At an extreme, an increase of only one cell causes each append operation to resize the array, leading to a familiar $1+2+3+\cdots+n$ summation and $\Omega(n^2)$ overall cost. Using increases of 2 or 3 at a time is slightly better, as portrayed in Figure 5.13, but the overall cost remains quadratic.

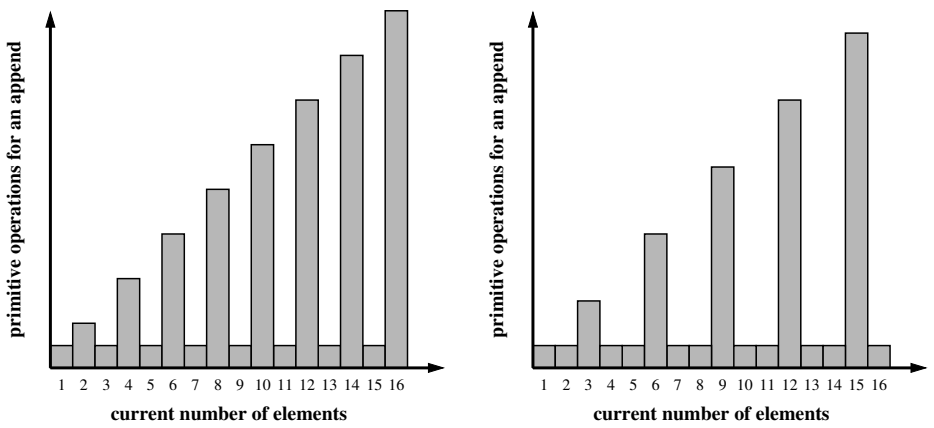


Figure 5.15: Running times of a series of append operations on a dynamic array using arithmetic progression of sizes. (a) Assumes increase of 2 in size of the array, while (b) assumes increase of 3.

Using a *fixed* increment for each resize, and thus an arithmetic progression of intermediate array sizes, results in an overall time that is quadratic in the number of operations, as shown in the following proposition. Intuitively, even an increase in 1000 cells per resize will become insignificant for large data sets.

Proposition 5.2: *Performing a series of n append operations on an initially empty dynamic array using a fixed increment with each resize takes $\Omega(n^2)$ time.*

Justification: Let $c > 0$ represent the fixed increment in capacity that is used for each resize event. During the series of n append operations, time will have been spent initializing arrays of size $c, 2c, 3c, \ldots, mc$ for $m = \lceil n/c \rceil$, and therefore, the overall time would be proportional to $c + 2c + 3c + \cdots + mc$. By Proposition 3.3, this sum is

$$\sum_{i=1}^{m} ci = c \cdot \sum_{i=1}^{m} i = c\frac{m(m+1)}{2} \geq c\frac{\frac{n}{c}(\frac{n}{c}+1)}{2} \geq \frac{n^2}{2c}.$$

Therefore, performing the n append operations takes $\Omega(n^2)$ time. ■

A lesson to be learned from Propositions 5.1 and 5.2 is that a subtle difference in an algorithm design can produce drastic differences in the asymptotic performance, and that a careful analysis can provide important insights into the design of a data structure.

Memory Usage and Shrinking an Array

Another consequence of the rule of a geometric increase in capacity when appending to a dynamic array is that the final array size is guaranteed to be proportional to the overall number of elements. That is, the data structure uses $O(n)$ memory. This is a very desirable property for a data structure.

If a container, such as a Python list, provides operations that cause the removal of one or more elements, greater care must be taken to ensure that a dynamic array guarantees $O(n)$ memory usage. The risk is that repeated insertions may cause the underlying array to grow arbitrarily large, and that there will no longer be a proportional relationship between the actual number of elements and the array capacity after many elements are removed.

A robust implementation of such a data structure will shrink the underlying array, on occasion, while maintaining the $O(1)$ amortized bound on individual operations. However, care must be taken to ensure that the structure cannot rapidly oscillate between growing and shrinking the underlying array, in which case the amortized bound would not be achieved. In Exercise C-5.16, we explore a strategy in which the array capacity is halved whenever the number of actual element falls below one fourth of that capacity, thereby guaranteeing that the array capacity is at most four times the number of elements; we explore the amortized analysis of such a strategy in Exercises C-5.17 and C-5.18.

5.3.3 Python's List Class

The experiments of Code Fragment 5.1 and 5.2, at the beginning of Section 5.3, provide empirical evidence that Python's list class is using a form of dynamic arrays for its storage. Yet, a careful examination of the intermediate array capacities (see Exercises R-5.2 and C-5.13) suggests that Python is not using a pure geometric progression, nor is it using an arithmetic progression.

With that said, it is clear that Python's implementation of the append method exhibits amortized constant-time behavior. We can demonstrate this fact experimentally. A single append operation typically executes so quickly that it would be difficult for us to accurately measure the time elapsed at that granularity, although we should notice some of the more expensive operations in which a resize is performed. We can get a more accurate measure of the amortized cost per operation by performing a series of n append operations on an initially empty list and determining the *average* cost of each. A function to perform that experiment is given in Code Fragment 5.4.

```
1  from time import time                  # import time function from time module
2  def compute_average(n):
3    """Perform n appends to an empty list and return average time elapsed."""
4    data = []
5    start = time( )                      # record the start time (in seconds)
6    for k in range(n):
7      data.append(None)
8    end = time( )                        # record the end time (in seconds)
9    return (end − start) / n             # compute average per operation
```

Code Fragment 5.4: Measuring the amortized cost of append for Python's list class.

Technically, the time elapsed between the start and end includes the time to manage the iteration of the for loop, in addition to the append calls. The empirical results of the experiment, for increasingly large values of n, are shown in Table 5.2. We see higher average cost for the smaller data sets, perhaps in part due to the overhead of the loop range. There is also natural variance in measuring the amortized cost in this way, because of the impact of the final resize event relative to n. Taken as a whole, there seems clear evidence that the amortized time for each append is independent of n.

n	100	1,000	10,000	100,000	1,000,000	10,000,000	100,000,000
μs	0.219	0.158	0.164	0.151	0.147	0.147	0.149

Table 5.2: Average running time of append, measured in microseconds, as observed over a sequence of n calls, starting with an empty list.

5.4 Efficiency of Python's Sequence Types

In the previous section, we began to explore the underpinnings of Python's list class, in terms of implementation strategies and efficiency. We continue in this section by examining the performance of all of Python's sequence types.

5.4.1 Python's List and Tuple Classes

The *nonmutating* behaviors of the list class are precisely those that are supported by the tuple class. We note that tuples are typically more memory efficient than lists because they are immutable; therefore, there is no need for an underlying dynamic array with surplus capacity. We summarize the asymptotic efficiency of the nonmutating behaviors of the list and tuple classes in Table 5.3. An explanation of this analysis follows.

Operation	Running Time
len(data)	$O(1)$
data[j]	$O(1)$
data.count(value)	$O(n)$
data.index(value)	$O(k+1)$
value **in** data	$O(k+1)$
data1 == data2 (similarly !=, <, <=, >, >=)	$O(k+1)$
data[j:k]	$O(k-j+1)$
data1 + data2	$O(n_1+n_2)$
c * data	$O(cn)$

Table 5.3: Asymptotic performance of the nonmutating behaviors of the list and tuple classes. Identifiers data, data1, and data2 designate instances of the list or tuple class, and n, n_1, and n_2 their respective lengths. For the containment check and index method, k represents the index of the leftmost occurrence (with $k = n$ if there is no occurrence). For comparisons between two sequences, we let k denote the leftmost index at which they disagree or else $k = \min(n_1, n_2)$.

Constant-Time Operations

The length of an instance is returned in constant time because an instance explicitly maintains such state information. The constant-time efficiency of syntax data[j] is assured by the underlying access into an array.

Searching for Occurrences of a Value

Each of the count, index, and __contains__ methods proceed through iteration of the sequence from left to right. In fact, Code Fragment 2.14 of Section 2.4.3 demonstrates how those behaviors might be implemented. Notably, the loop for computing the count must proceed through the entire sequence, while the loops for checking containment of an element or determining the index of an element immediately exit once they find the leftmost occurrence of the desired value, if one exists. So while count always examines the n elements of the sequence, index and __contains__ examine n elements in the worst case, but may be faster. Empirical evidence can be found by setting data = list(range(10000000)) and then comparing the relative efficiency of the test, 5 **in** data, relative to the test, 9999995 **in** data, or even the failed test, -5 **in** data.

Lexicographic Comparisons

Comparisons between two sequences are defined lexicographically. In the worst case, evaluating such a condition requires an iteration taking time proportional to the length of the *shorter* of the two sequences (because when one sequence ends, the lexicographic result can be determined). However, in some cases the result of the test can be evaluated more efficiently. For example, if evaluating [7, 3, ...] $<$ [7, 5, ...], it is clear that the result is True without examining the remainders of those lists, because the second element of the left operand is strictly less than the second element of the right operand.

Creating New Instances

The final three behaviors in Table 5.3 are those that construct a new instance based on one or more existing instances. In all cases, the running time depends on the construction and initialization of the new result, and therefore the asymptotic behavior is proportional to the *length* of the result. Therefore, we find that slice data[6000000:6000008] can be constructed almost immediately because it has only eight elements, while slice data[6000000:7000000] has one million elements, and thus is more time-consuming to create.

Mutating Behaviors

The efficiency of the mutating behaviors of the list class are described in Table 5.3. The simplest of those behaviors has syntax data[j] = val, and is supported by the special __setitem__ method. This operation has worst-case $O(1)$ running time because it simply replaces one element of a list with a new value. No other elements are affected and the size of the underlying array does not change. The more interesting behaviors to analyze are those that add or remove elements from the list.

Operation	Running Time
data[j] = val	$O(1)$
data.append(value)	$O(1)^*$
data.insert(k, value)	$O(n-k+1)^*$
data.pop( )	$O(1)^*$
data.pop(k) **del** data[k]	$O(n-k)^*$
data.remove(value)	$O(n)^*$
data1.extend(data2) data1 += data2	$O(n_2)^*$
data.reverse( )	$O(n)$
data.sort( )	$O(n \log n)$

*amortized

Table 5.4: Asymptotic performance of the mutating behaviors of the list class. Identifiers data, data1, and data2 designate instances of the list class, and n, n_1, and n_2 their respective lengths.

Adding Elements to a List

In Section 5.3 we fully explored the append method. In the worst case, it requires $\Omega(n)$ time because the underlying array is resized, but it uses $O(1)$ time in the amortized sense. Lists also support a method, with signature insert(k, value), that inserts a given value into the list at index $0 \leq k \leq n$ while shifting all subsequent elements back one slot to make room. For the purpose of illustration, Code Fragment 5.5 provides an implementation of that method, in the context of our DynamicArray class introduced in Code Fragment 5.3. There are two complicating factors in analyzing the efficiency of such an operation. First, we note that the addition of one element may require a resizing of the dynamic array. That portion of the work requires $\Omega(n)$ worst-case time but only $O(1)$ amortized time, as per append. The other expense for insert is the shifting of elements to make room for the new item. The time for

```
1  def insert(self, k, value):
2    """Insert value at index k, shifting subsequent values rightward."""
3    # (for simplicity, we assume 0 <= k <= n in this verion)
4    if self._n == self._capacity:                # not enough room
5      self._resize(2 * self._capacity)           # so double capacity
6    for j in range(self._n, k, -1):              # shift rightmost first
7      self._A[j] = self._A[j-1]
8    self._A[k] = value                           # store newest element
9    self._n += 1
```

Code Fragment 5.5: Implementation of insert for our DynamicArray class.

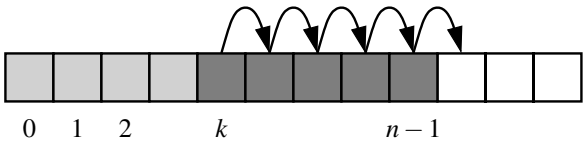


Figure 5.16: Creating room to insert a new element at index k of a dynamic array.

that process depends upon the index of the new element, and thus the number of other elements that must be shifted. That loop copies the reference that had been at index $n-1$ to index n, then the reference that had been at index $n-2$ to $n-1$, continuing until copying the reference that had been at index k to $k+1$, as illustrated in Figure 5.16. Overall this leads to an amortized $O(n-k+1)$ performance for inserting at index k.

When exploring the efficiency of Python's append method in Section 5.3.3, we performed an experiment that measured the average cost of repeated calls on varying sizes of lists (see Code Fragment 5.4 and Table 5.2). We have repeated that experiment with the insert method, trying three different access patterns:

- In the first case, we repeatedly insert at the beginning of a list,

```
for n in range(N):
  data.insert(0, None)
```

- In a second case, we repeatedly insert near the middle of a list,

```
for n in range(N):
  data.insert(n // 2, None)
```

- In a third case, we repeatedly insert at the end of the list,

```
for n in range(N):
  data.insert(n, None)
```

The results of our experiment are given in Table 5.5, reporting the *average* time per operation (not the total time for the entire loop). As expected, we see that inserting at the beginning of a list is most expensive, requiring linear time per operation. Inserting at the middle requires about half the time as inserting at the beginning, yet is still $\Omega(n)$ time. Inserting at the end displays $O(1)$ behavior, akin to append.

	N				
	100	1,000	10,000	100,000	1,000,000
k = 0	0.482	0.765	4.014	36.643	351.590
k = n // 2	0.451	0.577	2.191	17.873	175.383
k = n	0.420	0.422	0.395	0.389	0.397

Table 5.5: Average running time of insert(k, val), measured in microseconds, as observed over a sequence of N calls, starting with an empty list. We let n denote the size of the current list (as opposed to the final list).

Removing Elements from a List

Python's list class offers several ways to remove an element from a list. A call to pop( ) removes the last element from a list. This is most efficient, because all other elements remain in their original location. This is effectively an $O(1)$ operation, but the bound is amortized because Python will occasionally shrink the underlying dynamic array to conserve memory.

The parameterized version, pop(k), removes the element that is at index $k < n$ of a list, shifting all subsequent elements leftward to fill the gap that results from the removal. The efficiency of this operation is $O(n-k)$, as the amount of shifting depends upon the choice of index k, as illustrated in Figure 5.17. Note well that this implies that pop(0) is the most expensive call, using $\Omega(n)$ time. (see experiments in Exercise R-5.8.)

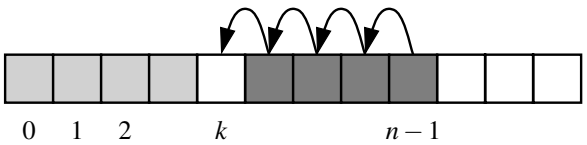


Figure 5.17: Removing an element at index k of a dynamic array.

The list class offers another method, named remove, that allows the caller to specify the *value* that should be removed (not the *index* at which it resides). Formally, it removes only the first occurrence of such a value from a list, or raises a ValueError if no such value is found. An implementation of such behavior is given in Code Fragment 5.6, again using our DynamicArray class for illustration.

Interestingly, there is no "efficient" case for remove; every call requires $\Omega(n)$ time. One part of the process searches from the beginning until finding the value at index k, while the rest iterates from k to the end in order to shift elements leftward. This linear behavior can be observed experimentally (see Exercise C-5.24).

```
1   def remove(self, value):
2     """Remove first occurrence of value (or raise ValueError)."""
3     # note: we do not consider shrinking the dynamic array in this version
4     for k in range(self._n):
5       if self._A[k] == value:                    # found a match!
6         for j in range(k, self._n - 1):          # shift others to fill gap
7           self._A[j] = self._A[j+1]
8         self._A[self._n - 1] = None              # help garbage collection
9         self._n -= 1                             # we have one less item
10        return                                   # exit immediately
11    raise ValueError('value not found')          # only reached if no match
```

Code Fragment 5.6: Implementation of remove for our DynamicArray class.

Extending a List

Python provides a method named extend that is used to add all elements of one list to the end of a second list. In effect, a call to data.extend(other) produces the same outcome as the code,

```
for element in other:
  data.append(element)
```

In either case, the running time is proportional to the length of the other list, and amortized because the underlying array for the first list may be resized to accommodate the additional elements.

In practice, the extend method is preferable to repeated calls to append because the constant factors hidden in the asymptotic analysis are significantly smaller. The greater efficiency of extend is threefold. First, there is always some advantage to using an appropriate Python method, because those methods are often implemented natively in a compiled language (rather than as interpreted Python code). Second, there is less overhead to a single function call that accomplishes all the work, versus many individual function calls. Finally, increased efficiency of extend comes from the fact that the resulting size of the updated list can be calculated in advance. If the second data set is quite large, there is some risk that the underlying dynamic array might be resized multiple times when using repeated calls to append. With a single call to extend, at most one resize operation will be performed. Exercise C-5.22 explores the relative efficiency of these two approaches experimentally.

Constructing New Lists

There are several syntaxes for constructing new lists. In almost all cases, the asymptotic efficiency of the behavior is linear in the length of the list that is created. However, as with the case in the preceding discussion of extend, there are significant differences in the practical efficiency.

Section 1.9.2 introduces the topic of ***list comprehension***, using an example such as squares = [k*k **for** k **in** range(1, n+1)] as a shorthand for

```
squares = [ ]
for k in range(1, n+1):
  squares.append(k*k)
```

Experiments should show that the list comprehension syntax is significantly faster than building the list by repeatedly appending (see Exercise C-5.23).

Similarly, it is a common Python idiom to initialize a list of constant values using the multiplication operator, as in [0] * n to produce a list of length n with all values equal to zero. Not only is this succinct for the programmer; it is more efficient than building such a list incrementally.

5.4.2 Python's String Class

Strings are very important in Python. We introduced their use in Chapter 1, with a discussion of various operator syntaxes in Section 1.3. A comprehensive summary of the named methods of the class is given in Tables A.1 through A.4 of Appendix A. We will not formally analyze the efficiency of each of those behaviors in this section, but we do wish to comment on some notable issues. In general, we let n denote the length of a string. For operations that rely on a second string as a pattern, we let m denote the length of that pattern string.

The analysis for many behaviors is quite intuitive. For example, methods that produce a new string (e.g., capitalize, center, strip) require time that is linear in the length of the string that is produced. Many of the behaviors that test Boolean conditions of a string (e.g., islower) take $O(n)$ time, examining all n characters in the worst case, but short circuiting as soon as the answer becomes evident (e.g., islower can immediately return False if the first character is uppercased). The comparison operators (e.g., $==$, $<$) fall into this category as well.

Pattern Matching

Some of the most interesting behaviors, from an algorithmic point of view, are those that in some way depend upon finding a string pattern within a larger string; this goal is at the heart of methods such as __contains__, find, index, count, replace, and split. String algorithms will be the topic of Chapter 13, and this particular problem known as ***pattern matching*** will be the focus of Section 13.2. A naive implementation runs in $O(mn)$ time case, because we consider the $n-m+1$ possible starting indices for the pattern, and we spend $O(m)$ time at each starting position, checking if the pattern matches. However, in Section 13.2, we will develop an algorithm for finding a pattern of length m within a longer string of length n in $O(n)$ time.

Composing Strings

Finally, we wish to comment on several approaches for composing large strings. As an academic exercise, assume that we have a large string named document, and our goal is to produce a new string, letters, that contains only the alphabetic characters of the original string (e.g., with spaces, numbers, and punctuation removed). It may be tempting to compose a result through repeated concatenation, as follows.

```
# WARNING: do not do this
letters = ''                          # start with empty string
for c in document:
  if c.isalpha( ):
    letters += c                      # concatenate alphabetic character
```

While the preceding code fragment accomplishes the goal, it may be terribly inefficient. Because strings are immutable, the command, letters += c, would presumably compute the concatenation, letters + c, as a new string instance and then reassign the identifier, letters, to that result. Constructing that new string would require time proportional to its length. If the final result has n characters, the series of concatenations would take time proportional to the familiar sum $1+2+3+\cdots+n$, and therefore $O(n^2)$ time.

Inefficient code of this type is widespread in Python, perhaps because of the somewhat natural appearance of the code, and mistaken presumptions about how the += operator is evaluated with strings. Some later implementations of the Python interpreter have developed an optimization to allow such code to complete in linear time, but this is not guaranteed for all Python implementations. The optimization is as follows. The reason that a command, letters += c, causes a new string instance to be created is that the original string must be left unchanged if another variable in a program refers to that string. On the other hand, if Python knew that there were no other references to the string in question, it could implement += more efficiently by directly mutating the string (as a dynamic array). As it happens, the Python interpreter already maintains what are known as ***reference counts*** for each object; this count is used in part to determine if an object can be garbage collected. (See Section 15.1.2.) But in this context, it provides a means to detect when no other references exist to a string, thereby allowing the optimization.

A more standard Python idiom to guarantee linear time composition of a string is to use a temporary list to store individual pieces, and then to rely on the join method of the str class to compose the final result. Using this technique with our previous example would appear as follows:

```
temp = [ ]                          # start with empty list
for c in document:
  if c.isalpha( ):
    temp.append(c)                  # append alphabetic character
letters = ''.join(temp)             # compose overall result
```

This approach is guaranteed to run in $O(n)$ time. First, we note that the series of up to n append calls will require a total of $O(n)$ time, as per the definition of the amortized cost of that operation. The final call to join also guarantees that it takes time that is linear in the final length of the composed string.

As we discussed at the end of the previous section, we can further improve the practical execution time by using a list comprehension syntax to build up the temporary list, rather than by repeated calls to append. That solution appears as,

```
letters = ''.join([c for c in document if c.isalpha( )])
```

Better yet, we can entirely avoid the temporary list with a generator comprehension:

```
letters = ''.join(c for c in document if c.isalpha( ))
```

5.5 Using Array-Based Sequences

5.5.1 Storing High Scores for a Game

The first application we study is storing a sequence of high score entries for a video game. This is representative of many applications in which a sequence of objects must be stored. We could just as easily have chosen to store records for patients in a hospital or the names of players on a football team. Nevertheless, let us focus on storing high score entries, which is a simple application that is already rich enough to present some important data-structuring concepts.

To begin, we consider what information to include in an object representing a high score entry. Obviously, one component to include is an integer representing the score itself, which we identify as _score. Another useful thing to include is the name of the person earning this score, which we identify as _name. We could go on from here, adding fields representing the date the score was earned or game statistics that led to that score. However, we omit such details to keep our example simple. A Python class, GameEntry, representing a game entry, is given in Code Fragment 5.7.

```
1  class GameEntry:
2    """Represents one entry of a list of high scores."""
3
4    def __init__(self, name, score):
5      self._name = name
6      self._score = score
7
8    def get_name(self):
9      return self._name
10
11   def get_score(self):
12     return self._score
13
14   def __str__(self):
15     return '({0}, {1})'.format(self._name, self._score) # e.g., '(Bob, 98)'
```

Code Fragment 5.7: Python code for a simple GameEntry class. We include methods for returning the name and score for a game entry object, as well as a method for returning a string representation of this entry.

A Class for High Scores

To maintain a sequence of high scores, we develop a class named Scoreboard. A scoreboard is limited to a certain number of high scores that can be saved; once that limit is reached, a new score only qualifies for the scoreboard if it is strictly higher than the lowest "high score" on the board. The length of the desired scoreboard may depend on the game, perhaps 10, 50, or 500. Since that limit may vary depending on the game, we allow it to be specified as a parameter to our Scoreboard constructor.

Internally, we will use a Python list named _board in order to manage the GameEntry instances that represent the high scores. Since we expect the scoreboard to eventually reach full capacity, we initialize the list to be large enough to hold the maximum number of scores, but we initially set all entries to None. By allocating the list with maximum capacity initially, it never needs to be resized. As entries are added, we will maintain them from highest to lowest score, starting at index 0 of the list. We illustrate a typical state of the data structure in Figure 5.18.

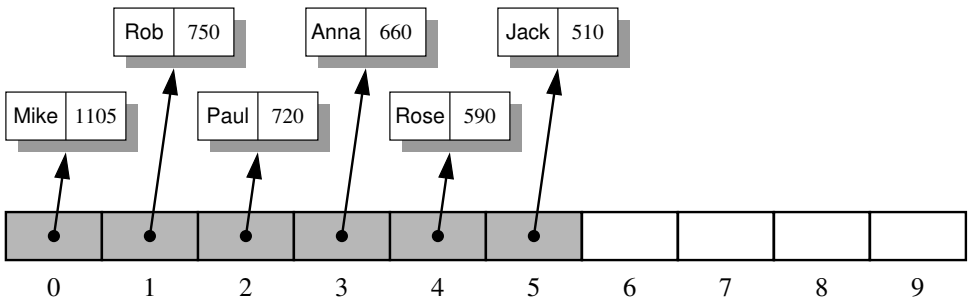


Figure 5.18: An illustration of an ordered list of length ten, storing references to six GameEntry objects in the cells from index 0 to 5, with the rest being None.

A complete Python implementation of the Scoreboard class is given in Code Fragment 5.8. The constructor is rather simple. The command

self._board = [**None**] * capacity

creates a list with the desired length, yet all entries equal to None. We maintain an additional instance variable, _n, that represents the number of actual entries currently in our table. For convenience, our class supports the __getitem__ method to retrieve an entry at a given index with a syntax board[i] (or None if no such entry exists), and we support a simple __str__ method that returns a string representation of the entire scoreboard, with one entry per line.

```
1  class Scoreboard:
2    """Fixed-length sequence of high scores in nondecreasing order."""
3
4    def __init__(self, capacity=10):
5      """Initialize scoreboard with given maximum capacity.
6
7      All entries are initially None.
8      """
9      self._board = [None] * capacity          # reserve space for future scores
10     self._n = 0                              # number of actual entries
11
12   def __getitem__(self, k):
13     """Return entry at index k."""
14     return self._board[k]
15
16   def __str__(self):
17     """Return string representation of the high score list."""
18     return '\n'.join(str(self._board[j]) for j in range(self._n))
19
20   def add(self, entry):
21     """Consider adding entry to high scores."""
22     score = entry.get_score( )
23
24     # Does new entry qualify as a high score?
25     # answer is yes if board not full or score is higher than last entry
26     good = self._n < len(self._board) or score > self._board[-1].get_score( )
27
28     if good:
29       if self._n < len(self._board):         # no score drops from list
30         self._n += 1                         # so overall number increases
31
32       # shift lower scores rightward to make room for new entry
33       j = self._n - 1
34       while j > 0 and self._board[j-1].get_score( ) < score:
35         self._board[j] = self._board[j-1]    # shift entry from j-1 to j
36         j -= 1                               # and decrement j
37       self._board[j] = entry                 # when done, add new entry
```

Code Fragment 5.8: Python code for a Scoreboard class that maintains an ordered series of scores as GameEntry objects.

Adding an Entry

The most interesting method of the Scoreboard class is add, which is responsible for considering the addition of a new entry to the scoreboard. Keep in mind that every entry will not necessarily qualify as a high score. If the board is not yet full, any new entry will be retained. Once the board is full, a new entry is only retained if it is strictly better than one of the other scores, in particular, the last entry of the scoreboard, which is the lowest of the high scores.

When a new score is considered, we begin by determining whether it qualifies as a high score. If so, we increase the count of active scores, _n, unless the board is already at full capacity. In that case, adding a new high score causes some other entry to be dropped from the scoreboard, so the overall number of entries remains the same.

To correctly place a new entry within the list, the final task is to shift any inferior scores one spot lower (with the least score being dropped entirely when the scoreboard is full). This process is quite similar to the implementation of the insert method of the list class, as described on pages 204–205. In the context of our scoreboard, there is no need to shift any None references that remain near the end of the array, so the process can proceed as diagrammed in Figure 5.19.

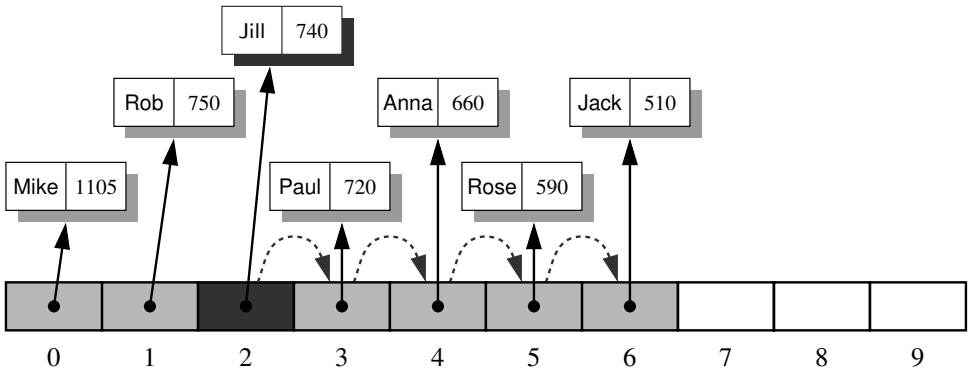


Figure 5.19: Adding a new GameEntry for Jill to the scoreboard. In order to make room for the new reference, we have to shift the references for game entries with smaller scores than the new one to the right by one cell. Then we can insert the new entry with index 2.

To implement the final stage, we begin by considering index j = self._n − 1, which is the index at which the last GameEntry instance will reside, after completing the operation. Either j is the correct index for the newest entry, or one or more immediately before it will have lesser scores. The while loop at line 34 checks the compound condition, shifting references rightward and decrementing j, as long as there is another entry at index $j-1$ with a score less than the new score.

5.5.2 Sorting a Sequence

In the previous subsection, we considered an application for which we added an object to a sequence at a given position while shifting other elements so as to keep the previous order intact. In this section, we use a similar technique to solve the ***sorting*** problem, that is, starting with an unordered sequence of elements and rearranging them into nondecreasing order.

The Insertion-Sort Algorithm

We study several sorting algorithms in this book, most of which are described in Chapter 12. As a warm-up, in this section we describe a nice, simple sorting algorithm known as ***insertion-sort***. The algorithm proceeds as follows for an array-based sequence. We start with the first element in the array. One element by itself is already sorted. Then we consider the next element in the array. If it is smaller than the first, we swap them. Next we consider the third element in the array. We swap it leftward until it is in its proper order with the first two elements. We then consider the fourth element, and swap it leftward until it is in the proper order with the first three. We continue in this manner with the fifth element, the sixth, and so on, until the whole array is sorted. We can express the insertion-sort algorithm in pseudo-code, as shown in Code Fragment 5.9.

Algorithm InsertionSort(A):
Input: An array A of n comparable elements
Output: The array A with elements rearranged in nondecreasing order
for k from 1 to n − 1 **do**
Insert A[k] at its proper location within A[0], A[1], ..., A[k].

Code Fragment 5.9: High-level description of the insertion-sort algorithm.

This is a simple, high-level description of insertion-sort. If we look back to Code Fragment 5.8 of Section 5.5.1, we see that the task of inserting a new entry into the list of high scores is almost identical to the task of inserting a newly considered element in insertion-sort (except that game scores were ordered from high to low). We provide a Python implementation of insertion-sort in Code Fragment 5.10, using an outer loop to consider each element in turn, and an inner loop that moves a newly considered element to its proper location relative to the (sorted) subarray of elements that are to its left. We illustrate an example run of the insertion-sort algorithm in Figure 5.20.

The nested loops of insertion-sort lead to an $O(n^2)$ running time in the worst case. The most work is done if the array is initially in reverse order. On the other hand, if the initial array is nearly sorted or perfectly sorted, insertion-sort runs in $O(n)$ time because there are few or no iterations of the inner loop.

```
1  def insertion_sort(A):
2    """Sort list of comparable elements into nondecreasing order."""
3    for k in range(1, len(A)):                # from 1 to n-1
4      cur = A[k]                              # current element to be inserted
5      j = k                                   # find correct index j for current
6      while j > 0 and A[j−1] > cur:           # element A[j-1] must be after current
7        A[j] = A[j−1]
8        j −= 1
9      A[j] = cur                              # cur is now in the right place
```

Code Fragment 5.10: Python code for performing insertion-sort on a list.

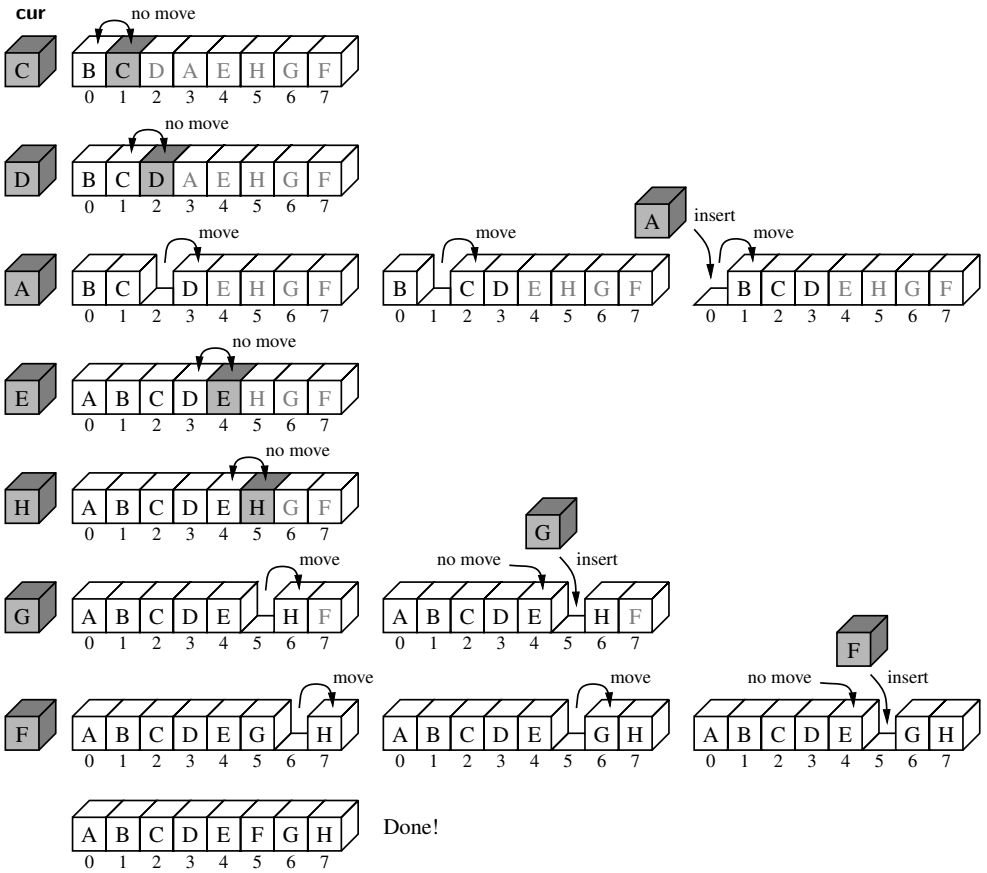


Figure 5.20: Execution of the insertion-sort algorithm on an array of eight characters. Each row corresponds to an iteration of the outer loop, and each copy of the sequence in a row corresponds to an iteration of the inner loop. The current element that is being inserted is highlighted in the array, and shown as the cur value.

5.5.3 Simple Cryptography

An interesting application of strings and lists is ***cryptography***, the science of secret messages and their applications. This field studies ways of performing ***encryption***, which takes a message, called the ***plaintext***, and converts it into a scrambled message, called the ***ciphertext***. Likewise, cryptography also studies corresponding ways of performing ***decryption***, which takes a ciphertext and turns it back into its original plaintext.

Arguably the earliest encryption scheme is the ***Caesar cipher***, which is named after Julius Caesar, who used this scheme to protect important military messages. (All of Caesar's messages were written in Latin, of course, which already makes them unreadable for most of us!) The Caesar cipher is a simple way to obscure a message written in a language that forms words with an alphabet.

The Caesar cipher involves replacing each letter in a message with the letter that is a certain number of letters after it in the alphabet. So, in an English message, we might replace each A with D, each B with E, each C with F, and so on, if shifting by three characters. We continue this approach all the way up to W, which is replaced with Z. Then, we let the substitution pattern ***wrap around***, so that we replace X with A, Y with B, and Z with C.

Converting Between Strings and Character Lists

Given that strings are immutable, we cannot directly edit an instance to encrypt it. Instead, our goal will be to generate a new string. A convenient technique for performing string transformations is to create an equivalent list of characters, edit the list, and then reassemble a (new) string based on the list. The first step can be performed by sending the string as a parameter to the constructor of the list class. For example, the expression list('bird') produces the result ['b', 'i', 'r', 'd']. Conversely, we can use a list of characters to build a string by invoking the join method on an empty string, with the list of characters as the parameter. For example, the call ''.join(['b', 'i', 'r', 'd']) returns the string 'bird'.

Using Characters as Array Indices

If we were to number our letters like array indices, so that A is 0, B is 1, C is 2, and so on, then we can write the Caesar cipher with a rotation of r as a simple formula: Replace each letter i with the letter $(i + r)$ mod 26, where mod is the ***modulo*** operator, which returns the remainder after performing an integer division. This operator is denoted with % in Python, and it is exactly the operator we need to easily perform the wrap around at the end of the alphabet. For 26 mod 26 is 0, 27 mod 26 is 1, and 28 mod 26 is 2. The decryption algorithm for the Caesar cipher is just the opposite—we replace each letter with the one r places before it, with wrap around (that is, letter i is replaced by letter $(i - r)$ mod 26).

We can represent a replacement rule using another string to describe the translation. As a concrete example, suppose we are using a Caesar cipher with a three-character rotation. We can precompute a string that represents the replacements that should be used for each character from A to Z. For example, A should be replaced by D, B replaced by E, and so on. The 26 replacement characters in order are `'DEFGHIJKLMNOPQRSTUVWXYZABC'`. We can subsequently use this translation string as a guide to encrypt a message. The remaining challenge is how to quickly locate the replacement for each character of the original message.

Fortunately, we can rely on the fact that characters are represented in Unicode by integer code points, and the code points for the uppercase letters of the Latin alphabet are consecutive (for simplicity, we restrict our encryption to uppercase letters). Python supports functions that convert between integer code points and one-character strings. Specifically, the function ord(c) takes a one-character string as a parameter and returns the integer code point for that character. Conversely, the function chr(j) takes an integer and returns its associated one-character string.

In order to find a replacement for a character in our Caesar cipher, we need to map the characters `'A'` to `'Z'` to the respective numbers 0 to 25. The formula for doing that conversion is j = ord(c) − ord(`'A'`). As a sanity check, if character c is `'A'`, we have that $j = 0$. When c is `'B'`, we will find that its ordinal value is precisely one more than that for `'A'`, so their difference is 1. In general, the integer j that results from such a calculation can be used as an index into our precomputed translation string, as illustrated in Figure 5.21.

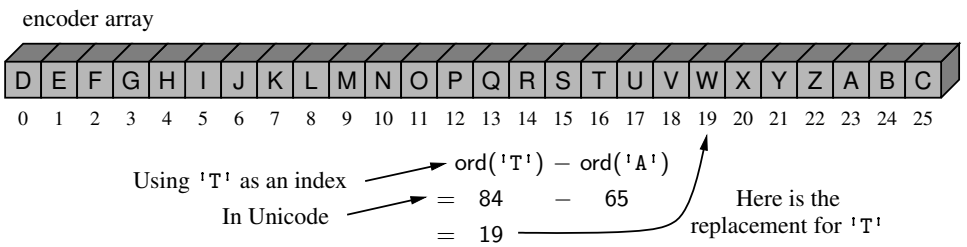


Figure 5.21: Illustrating the use of uppercase characters as indices, in this case to perform the replacement rule for Caesar cipher encryption.

In Code Fragment 5.11, we develop a Python class for performing the Caesar cipher with an arbitrary rotational shift, and demonstrate its use. When we run this program (to perform a simple test), we get the following output.

```
Secret:  WKH HDJOH LV LQ SODB; PHHW DW MRH'V.
Message: THE EAGLE IS IN PLAY; MEET AT JOE'S.
```

The constructor for the class builds the forward and backward translation strings for the given rotation. With those in hand, the encryption and decryption algorithms are essentially the same, and so we perform both by means of a nonpublic utility method named _transform.

```
class CaesarCipher:
  """Class for doing encryption and decryption using a Caesar cipher."""

  def __init__(self, shift):
    """Construct Caesar cipher using given integer shift for rotation."""
    encoder = [None] * 26                       # temp array for encryption
    decoder = [None] * 26                       # temp array for decryption
    for k in range(26):
      encoder[k] = chr((k + shift) % 26 + ord('A'))
      decoder[k] = chr((k - shift) % 26 + ord('A'))
    self._forward = ''.join(encoder)            # will store as string
    self._backward = ''.join(decoder)           # since fixed

  def encrypt(self, message):
    """Return string representing encripted message."""
    return  self._transform(message, self._forward)

  def decrypt(self, secret):
    """Return decrypted message given encrypted secret."""
    return  self._transform(secret, self._backward)

  def _transform(self, original, code):
    """Utility to perform transformation based on given code string."""
    msg = list(original)
    for k in range(len(msg)):
      if msg[k].isupper( ):
        j = ord(msg[k]) - ord('A')              # index from 0 to 25
        msg[k] = code[j]                        # replace this character
    return ''.join(msg)

if __name__ == '__main__':
  cipher = CaesarCipher(3)
  message = "THE EAGLE IS IN PLAY; MEET AT JOE'S."
  coded = cipher.encrypt(message)
  print('Secret: ', coded)
  answer = cipher.decrypt(coded)
  print('Message:', answer)
```

Code Fragment 5.11: A complete Python class for the Caesar cipher.

5.6 Multidimensional Data Sets

Lists, tuples, and strings in Python are one-dimensional. We use a single index to access each element of the sequence. Many computer applications involve multidimensional data sets. For example, computer graphics are often modeled in either two or three dimensions. Geographic information may be naturally represented in two dimensions, medical imaging may provide three-dimensional scans of a patient, and a company's valuation is often based upon a high number of independent financial measures that can be modeled as multidimensional data. A two-dimensional array is sometimes also called a ***matrix***. We may use two indices, say i and j, to refer to the cells in the matrix. The first index usually refers to a row number and the second to a column number, and these are traditionally zero-indexed in computer science. Figure 5.22 illustrates a two-dimensional data set with integer values. This data might, for example, represent the number of stores in various regions of Manhattan.

	0	1	2	3	4	5	6	7	8	9
0	22	18	709	5	33	10	4	56	82	440
1	45	32	830	120	750	660	13	77	20	105
2	4	880	45	66	61	28	650	7	510	67
3	940	12	36	3	20	100	306	590	0	500
4	50	65	42	49	88	25	70	126	83	288
5	398	233	5	83	59	232	49	8	365	90
6	33	58	632	87	94	5	59	204	120	829
7	62	394	3	4	102	140	183	390	16	26

Figure 5.22: Illustration of a two-dimensional integer data set, which has 8 rows and 10 columns. The rows and columns are zero-indexed. If this data set were named stores, the value of stores[3][5] is 100 and the value of stores[6][2] is 632.

A common representation for a two-dimensional data set in Python is as a list of lists. In particular, we can represent a two-dimensional array as a list of rows, with each row itself being a list of values. For example, the two-dimensional data

$$\begin{array}{ccccc} 22 & 18 & 709 & 5 & 33 \\ 45 & 32 & 830 & 120 & 750 \\ 4 & 880 & 45 & 66 & 61 \end{array}$$

might be stored in Python as follows.

```
data = [ [22, 18, 709, 5, 33], [45, 32, 830, 120, 750], [4, 880, 45, 66, 61] ]
```

An advantage of this representation is that we can naturally use a syntax such as data[1][3] to represent the value that has row index 1 and column index 3, as data[1], the second entry in the outer list, is itself a list, and thus indexable.

Constructing a Multidimensional List

To quickly initialize a one-dimensional list, we generally rely on a syntax such as `data = [0] * n` to create a list of n zeros. On page 189, we emphasized that from a technical perspective, this creates a list of length n with all entries referencing the same integer instance, but that there was no meaningful consequence of such aliasing because of the immutability of the `int` class in Python.

We have to be considerably more careful when creating a list of lists. If our goal were to create the equivalent of a two-dimensional list of integers, with r rows and c columns, and to initialize all values to zero, a flawed approach might be to try the command

```
data = ([0] * c) * r                # Warning: this is a mistake
```

While `([0] * c)` is indeed a list of c zeros, multiplying that list by r unfortunately creates a single list with length $r \cdot c$, just as `[2,4,6] * 2` results in list `[2, 4, 6, 2, 4, 6]`.

A better, yet still flawed attempt is to make a list that contains the list of c zeros as its only element, and then to multiply that list by r. That is, we could try the command

```
data = [ [0] * c ] * r              # Warning: still a mistake
```

This is much closer, as we actually do have a structure that is formally a list of lists. The problem is that all r entries of the list known as `data` are references to the same instance of a list of c zeros. Figure 5.23 provides a portrayal of such aliasing.

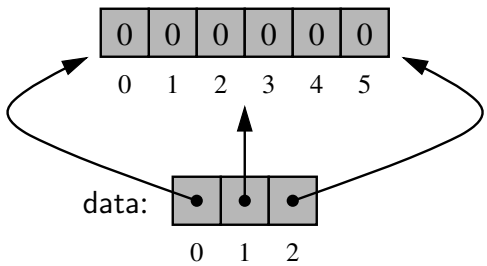


Figure 5.23: A flawed representation of a 3×6 data set as a list of lists, created with the command `data = [ [0] * 6 ] * 3`. (For simplicity, we overlook the fact that the values in the secondary list are referential.)

This is truly a problem. Setting an entry such as `data[2][0] = 100` would change the first entry of the secondary list to reference a new value, 100. Yet that cell of the secondary list also represents the value `data[0][0]`, because "row" `data[0]` and "row" `data[2]` refer to the same secondary list.

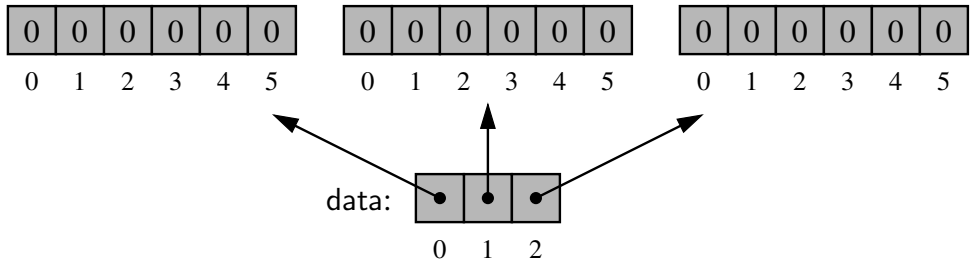


Figure 5.24: A valid representation of a 3×6 data set as a list of lists. (For simplicity, we overlook the fact that the values in the secondary lists are referential.)

To properly initialize a two-dimensional list, we must ensure that each cell of the primary list refers to an *independent* instance of a secondary list. This can be accomplished through the use of Python's list comprehension syntax.

```
data = [ [0] * c for j in range(r) ]
```

This command produces a valid configuration, similar to the one shown in Figure 5.24. By using list comprehension, the expression `[0] * c` is reevaluated for each pass of the embedded for loop. Therefore, we get r distinct secondary lists, as desired. (We note that the variable j in that command is irrelevant; we simply need a for loop that iterates r times.)

Two-Dimensional Arrays and Positional Games

Many computer games, be they strategy games, simulation games, or first-person conflict games, involve objects that reside in a two-dimensional space. Software for such ***positional games*** need a way of representing such a two-dimensional "board," and in Python the list of lists is a natural choice.

Tic-Tac-Toe

As most school children know, ***Tic-Tac-Toe*** is a game played in a three-by-three board. Two players—X and O—alternate in placing their respective marks in the cells of this board, starting with player X. If either player succeeds in getting three of his or her marks in a row, column, or diagonal, then that player wins.

This is admittedly not a sophisticated positional game, and it's not even that much fun to play, since a good player O can always force a tie. Tic-Tac-Toe's saving grace is that it is a nice, simple example showing how two-dimensional arrays can be used for positional games. Software for more sophisticated positional games, such as checkers, chess, or the popular simulation games, are all based on the same approach we illustrate here for using a two-dimensional array for Tic-Tac-Toe.

Our representation of a 3×3 board will be a list of lists of characters, with `'X'` or `'O'` designating a player's move, or `' '` designating an empty space. For example, the board configuration

O	X	O
	X	
	O	X

will be stored internally as

```
[ ['O', 'X', 'O'], [' ', 'X', ' '], [' ', 'O', 'X'] ]
```

We develop a complete Python class for maintaining a Tic-Tac-Toe board for two players. That class will keep track of the moves and report a winner, but it does not perform any strategy or allow someone to play Tic-Tac-Toe against the computer. The details of such a program are beyond the scope of this chapter, but it might nonetheless make a good course project (see Exercise P-8.68).

Before presenting the implementation of the class, we demonstrate its public interface with a simple test in Code Fragment 5.12.

```
1  game = TicTacToe( )
2  # X moves:            # O moves:
3  game.mark(1, 1);      game.mark(0, 2)
4  game.mark(2, 2);      game.mark(0, 0)
5  game.mark(0, 1);      game.mark(2, 1)
6  game.mark(1, 2);      game.mark(1, 0)
7  game.mark(2, 0)
8
9  print(game)
10 winner = game.winner( )
11 if winner is None:
12   print('Tie')
13 else:
14   print(winner, 'wins')
```

Code Fragment 5.12: A simple test for our Tic-Tac-Toe class.

The basic operations are that a new game instance represents an empty board, that the mark(i,j) method adds a mark at the given position for the current player (with the software managing the alternating of turns), and that the game board can be printed and the winner determined. The complete source code for the TicTacToe class is given in Code Fragment 5.13. Our mark method performs error checking to make sure that valid indices are sent, that the position is not already occupied, and that no further moves are made after someone wins the game.

```
class TicTacToe:
  """Management of a Tic-Tac-Toe game (does not do strategy)."""

  def __init__(self):
    """Start a new game."""
    self._board = [ [' '] * 3 for j in range(3) ]
    self._player = 'X'

  def mark(self, i, j):
    """Put an X or O mark at position (i,j) for next player's turn."""
    if not (0 <= i <= 2 and 0 <= j <= 2):
      raise ValueError('Invalid board position')
    if self._board[i][j] != ' ':
      raise ValueError('Board position occupied')
    if self.winner( ) is not None:
      raise ValueError('Game is already complete')
    self._board[i][j] = self._player
    if self._player == 'X':
      self._player = 'O'
    else:
      self._player = 'X'

  def _is_win(self, mark):
    """Check whether the board configuration is a win for the given player."""
    board = self._board                                  # local variable for shorthand
    return (mark == board[0][0] == board[0][1] == board[0][2] or     # row 0
            mark == board[1][0] == board[1][1] == board[1][2] or     # row 1
            mark == board[2][0] == board[2][1] == board[2][2] or     # row 2
            mark == board[0][0] == board[1][0] == board[2][0] or     # column 0
            mark == board[0][1] == board[1][1] == board[2][1] or     # column 1
            mark == board[0][2] == board[1][2] == board[2][2] or     # column 2
            mark == board[0][0] == board[1][1] == board[2][2] or     # diagonal
            mark == board[0][2] == board[1][1] == board[2][0])       # rev diag

  def winner(self):
    """Return mark of winning player, or None to indicate a tie."""
    for mark in 'XO':
      if self._is_win(mark):
        return mark
    return None

  def __str__(self):
    """Return string representation of current game board."""
    rows = ['|'.join(self._board[r]) for r in range(3)]
    return '\n-----\n'.join(rows)
```

Code Fragment 5.13: A complete Python class for managing a Tic-Tac-Toe game.

5.7 Exercises

For help with exercises, please visit the site, www.wiley.com/college/goodrich.

Reinforcement

R-5.1 Execute the experiment from Code Fragment 5.1 and compare the results on your system to those we report in Code Fragment 5.2.

R-5.2 In Code Fragment 5.1, we perform an experiment to compare the length of a Python list to its underlying memory usage. Determining the sequence of array sizes requires a manual inspection of the output of that program. Redesign the experiment so that the program outputs only those values of k at which the existing capacity is exhausted. For example, on a system consistent with the results of Code Fragment 5.2, your program should output that the sequence of array capacities are 0, 4, 8, 16, 25, ...

R-5.3 Modify the experiment from Code Fragment 5.1 in order to demonstrate that Python's list class occasionally shrinks the size of its underlying array when elements are popped from a list.

R-5.4 Our DynamicArray class, as given in Code Fragment 5.3, does not support use of negative indices with __getitem__. Update that method to better match the semantics of a Python list.

R-5.5 Redo the justification of Proposition 5.1 assuming that the the cost of growing the array from size k to size $2k$ is $3k$ cyber-dollars. How much should each append operation be charged to make the amortization work?

R-5.6 Our implementation of insert for the DynamicArray class, as given in Code Fragment 5.5, has the following inefficiency. In the case when a resize occurs, the resize operation takes time to copy all the elements from an old array to a new array, and then the subsequent loop in the body of insert shifts many of those elements. Give an improved implementation of the insert method, so that, in the case of a resize, the elements are shifted into their final position during that operation, thereby avoiding the subsequent shifting.

R-5.7 Let A be an array of size $n \geq 2$ containing integers from 1 to $n-1$, inclusive, with exactly one repeated. Describe a fast algorithm for finding the integer in A that is repeated.

R-5.8 Experimentally evaluate the efficiency of the pop method of Python's list class when using varying indices as a parameter, as we did for insert on page 205. Report your results akin to Table 5.5.

R-5.9 Explain the changes that would have to be made to the program of Code Fragment 5.11 so that it could perform the Caesar cipher for messages that are written in an alphabet-based language other than English, such as Greek, Russian, or Hebrew.

R-5.10 The constructor for the CaesarCipher class in Code Fragment 5.11 can be implemented with a two-line body by building the forward and backward strings using a combination of the join method and an appropriate comprehension syntax. Give such an implementation.

R-5.11 Use standard control structures to compute the sum of all numbers in an $n \times n$ data set, represented as a list of lists.

R-5.12 Describe how the built-in sum function can be combined with Python's comprehension syntax to compute the sum of all numbers in an $n \times n$ data set, represented as a list of lists.

Creativity

C-5.13 In the experiment of Code Fragment 5.1, we begin with an empty list. If data were initially constructed with nonempty length, does this affect the sequence of values at which the underlying array is expanded? Perform your own experiments, and comment on any relationship you see between the initial length and the expansion sequence.

C-5.14 The shuffle method, supported by the random module, takes a Python list and rearranges it so that every possible ordering is equally likely. Implement your own version of such a function. You may rely on the randrange(n) function of the random module, which returns a random number between 0 and $n-1$ inclusive.

C-5.15 Consider an implementation of a dynamic array, but instead of copying the elements into an array of double the size (that is, from N to $2N$) when its capacity is reached, we copy the elements into an array with $\lceil N/4 \rceil$ additional cells, going from capacity N to capacity $N + \lceil N/4 \rceil$. Prove that performing a sequence of n append operations still runs in $O(n)$ time in this case.

C-5.16 Implement a pop method for the DynamicArray class, given in Code Fragment 5.3, that removes the last element of the array, and that shrinks the capacity, N, of the array by half any time the number of elements in the array goes below $N/4$.

C-5.17 Prove that when using a dynamic array that grows and shrinks as in the previous exercise, the following series of $2n$ operations takes $O(n)$ time: n append operations on an initially empty array, followed by n pop operations.

C-5.18 Give a formal proof that any sequence of n append or pop operations on an initially empty dynamic array takes $O(n)$ time, if using the strategy described in Exercise C-5.16.

C-5.19 Consider a variant of Exercise C-5.16, in which an array of capacity N is resized to capacity precisely that of the number of elements, any time the number of elements in the array goes strictly below $N/4$. Give a formal proof that any sequence of n append or pop operations on an initially empty dynamic array takes $O(n)$ time.

C-5.20 Consider a variant of Exercise C-5.16, in which an array of capacity N, is resized to capacity precisely that of the number of elements, any time the number of elements in the array goes strictly below $N/2$. Show that there exists a sequence of n operations that requires $\Omega(n^2)$ time to execute.

C-5.21 In Section 5.4.2, we described four different ways to compose a long string: (1) repeated concatenation, (2) appending to a temporary list and then joining, (3) using list comprehension with join, and (4) using generator comprehension with join. Develop an experiment to test the efficiency of all four of these approaches and report your findings.

C-5.22 Develop an experiment to compare the relative efficiency of the extend method of Python's list class versus using repeated calls to append to accomplish the equivalent task.

C-5.23 Based on the discussion of page 207, develop an experiment to compare the efficiency of Python's list comprehension syntax versus the construction of a list by means of repeated calls to append.

C-5.24 Perform experiments to evaluate the efficiency of the remove method of Python's list class, as we did for insert on page 205. Use known values so that all removals occur either at the beginning, middle, or end of the list. Report your results akin to Table 5.5.

C-5.25 The syntax data.remove(value) for Python list data removes only the first occurrence of element value from the list. Give an implementation of a function, with signature remove_all(data, value), that removes *all* occurrences of value from the given list, such that the worst-case running time of the function is $O(n)$ on a list with n elements. Not that it is not efficient enough in general to rely on repeated calls to remove.

C-5.26 Let B be an array of size $n \geq 6$ containing integers from 1 to $n-5$, inclusive, with exactly five repeated. Describe a good algorithm for finding the five integers in B that are repeated.

C-5.27 Given a Python list L of n positive integers, each represented with $k = \lceil \log n \rceil + 1$ bits, describe an $O(n)$-time method for finding a k-bit integer not in L.

C-5.28 Argue why any solution to the previous problem must run in $\Omega(n)$ time.

C-5.29 A useful operation in databases is the ***natural join***. If we view a database as a list of *ordered* pairs of objects, then the natural join of databases A and B is the list of all ordered triples (x,y,z) such that the pair (x,y) is in A and the pair (y,z) is in B. Describe and analyze an efficient algorithm for computing the natural join of a list A of n pairs and a list B of m pairs.

C-5.30 When Bob wants to send Alice a message M on the Internet, he breaks M into n ***data packets***, numbers the packets consecutively, and injects them into the network. When the packets arrive at Alice's computer, they may be out of order, so Alice must assemble the sequence of n packets in order before she can be sure she has the entire message. Describe an efficient scheme for Alice to do this, assuming that she knows the value of n. What is the running time of this algorithm?

C-5.31 Describe a way to use recursion to add all the numbers in an $n \times n$ data set, represented as a list of lists.

Projects

P-5.32 Write a Python function that takes two three-dimensional numeric data sets and adds them componentwise.

P-5.33 Write a Python program for a matrix class that can add and multiply two-dimensional arrays of numbers, assuming the dimensions agree appropriately for the operation.

P-5.34 Write a program that can perform the Caesar cipher for English messages that include both upper- and lowercase characters.

P-5.35 Implement a class, SubstitutionCipher, with a constructor that takes a string with the 26 uppercase letters in an arbitrary order and uses that for the forward mapping for encryption (akin to the self._forward string in our CaesarCipher class of Code Fragment 5.11). You should derive the backward mapping from the forward version.

P-5.36 Redesign the CaesarCipher class as a subclass of the SubstitutionCipher from the previous problem.

P-5.37 Design a RandomCipher class as a subclass of the SubstitutionCipher from Exercise P-5.35, so that each instance of the class relies on a random permutation of letters for its mapping.

Chapter Notes

The fundamental data structures of arrays belong to the folklore of computer science. They were first chronicled in the computer science literature by Knuth in his seminal book on *Fundamental Algorithms* [64].